Development of an Embedded HPEC Telemetry Unit and Dashboard Integration for the TUKZIE Rev 0 Platform



Presented by:
Samson Okuthe
Student number: OKTSAM001

Prepared for:
Simon Winberg
Co-supervisor: Sampath Jayalath
Department of Electrical Engineering
University of Cape Town

Submitted to the Department of Electrical Engineering at the University of Cape Town in partial fulfilment of the academic requirements for a Bachelor of Science degree in Electrical and Computer Engineering.

October 22, 2026

Declaration

1. I know that plagiarism is wrong. Plagiarism is to use another's work and pretend that it is one's own.
2. I have used the IEEE convention for citation and referencing. Each contribution to, and quotation in, this report from the work(s) of other people has been attributed, and has been cited and referenced.
3. This report is my own work.
4. I have not allowed, and will not allow, anyone to copy my work with the intention of passing it off as their own work or part thereof.

Signature: 

Samson Okuthe (OKTSAM001)

Date: October 22, 2026

Word count of the body of the report (Chapters 1 to 8, text only, excluding headings, tables, figures, captions and code listings; measured with TeXcount after removing tables and figures, Report/wordcount.py, 21 October 2026): 23 872.

Terms of Reference

This project (SW-7) is to develop a high-performance embedded computing (HPEC) telemetry unit for the TUKZIE Rev 0 electric cargo trike, integrated with the vehicle's existing dashboard, capable of acquiring synchronised inertial, battery, positioning and camera data, characterising the vehicle's dynamic response at the edge, preserving telemetry under intermittent cellular connectivity, and presenting a minimal set of safety-relevant information to the driver on that dashboard.

Acknowledgments

I thank my supervisor, A/Prof. Simon Winberg, and co-supervisor, Sampath Jayalath, for their guidance, and the TUKZIE Flagship Programme and its vacation-work teams, whose earlier work on the vehicle's dashboard, sensing and drivetrain this project builds on.

Abstract

This report describes an embedded telemetry unit for the TUKZIE Rev 0, a 72 V electric cargo trike, and its integration with the vehicle's existing dashboard, which replaced the windshield head-up display of the original brief. An ESP32-S3 with an integrated LTE modem samples two inertial measurement units at 200 Hz, reads the battery-management system over Bluetooth Low Energy and GNSS, computes time-domain ride features at the edge, logs every record to flash and publishes it over MQTT; after an outage it replays the records the broker did not acknowledge, and a program on the dashboard's Raspberry Pi 5 stores each record once in a database. Two camera units each run a pretrained object detector fused with three time-of-flight sensors and warn of objects closing on the vehicle from their time to contact: the Raspberry Pi 5 beside the display faces forward, and a Raspberry Pi 4 beside the battery, aligned in time with the inertial stream by a pulse wire, faces backward. Pages added to the dashboard switch between the two views and show live data only, with missing data marked as absent. On the bench, both inertial sensors held 200 Hz with no lost samples, although each flash write, about once a minute, delays sampling by up to 44 ms; calibration brought both to 9.80 to 9.81 m/s^2 at rest in the orientation calibrated; GNSS positions were a median of 2.7 m from a phone reference; after a 10-minute radio outage the unit reconnected in 5.2 s and cleared its 60-record backlog within 43 s, and every one of the 111 records in its log reached the database, 24 of them by a backfill from the log; and with the whole system running, the detector on the Raspberry Pi 4 processed 14.6 frames/s with a median of 118 to 122 ms from camera readout to detection (19.5 frames/s and 102 ms for the detector alone), against 19.2 frames/s and 51 to 54 ms on the Raspberry Pi 5 beside the running dashboard; mounted on the vehicle, the front detector returned results a median of 53 to 102 ms after readout and the rear one 126 to 151 ms. On a first road session with about 16 minutes of driving the battery link held under loads of up to 46.9 A and no inertial samples were dropped, but a third of the per-second records did not reach the recorder over the USB link, which has since been replaced by a wired serial link. With no GNSS antenna fitted and no raw samples kept, the ride-characterisation and glance-time questions remain open.

Contents

1 Introduction **1**

1.1 The vehicle and the problem 1

1.2 Deliverables and scope change 2

1.3 Research questions 2

1.4 Engineering requirements and acceptance criteria 2

1.5 Scope and limitations 2

1.6 Terminology 4

1.7 Report outline 5

2 Literature Review **6**

2.1 Scope and evaluation framework 6

2.2 Electric cargo trikes and the case for telemetry 6

2.3 Road roughness and ride response 8

2.4 Inertial signal processing and vibration separation 8

2.5 Edge computing and real-time telemetry architecture 9

2.6 Driver-display design 9

2.7 Cellular synchronisation and data integrity 10

2.8 Lightweight detection and monocular ranging 10

2.9 Proximity sensing on low-speed vehicles . . . 10

2.10 Platform options for acquisition and detection . . . 11

2.11 Synthesis and research gap . . . 11

3 Methodology **13**

3.1 Research approach . . . 13

3.2 Phases as run . . . 14

3.3 Site selection . . . 14

3.4 Experiments . . . 15

3.4.1 Start-up and interface checks . . . 16

3.4.2 Integration and power . . . 16

3.4.3 Vehicle tests . . . 16

3.5 Data collection . . . 17

3.6 Data analysis . . . 18

3.7 Ethics and safety . . . 19

3.8 Summary of methods . . . 20

4 Design **22**

4.1 System overview and architecture selection . . . 22

4.2 Hardware . . . 24

4.3 Firmware architecture . . . 25

4.4 Ride-characterisation signal processing . . . 26

4.4.1 Per-sensor calibration . . . 27

4.4.2 Speed-normalised index . . . 27

4.4.3 Dual-IMU fusion . . . 28
4.5 Battery-management interface . . . 28
4.6 Positioning . . . 28
4.7 Other vehicle inputs . . . 29
4.8 Camera subsystem . . . 29
4.8.1 Capture and synchronisation . . . 29
4.8.2 Detection . . . 30
4.8.3 Range sensing and fusion . . . 31
4.8.4 Dashboard integration . . . 32
4.8.5 Approaching objects: time to contact . . . 33
4.8.6 Two-sided layout . . . 34
4.9 Communications and local logging . . . 34
4.10 Fault investigations . . . 36
4.10.1 Boot loop after enabling the full flash . . . 37
4.10.2 Modem power key, GNSS warm-up and reply layout . . . 37
4.10.3 Battery-management connection and checksum . . . 37
4.10.4 Invalid JSON found only by reading the log back . . . 38
4.10.5 Camera colour cast . . . 38
4.10.6 Time-of-flight start-up hang . . . 39
4.10.7 Raspberry Pi 4 under-voltage and reboots . . . 39
4.11 Testing infrastructure . . . 41
4.12 Maintenance design . . . 42

5 Results **44**

5.1 Inertial acquisition (R1) . . . 44
5.2 Calibration (R2) . . . 47
5.3 Ride features and fusion (R3) . . . 47
5.4 Position (R4) . . . 50
5.5 Battery telemetry (R5) . . . 50
5.6 Hazard sensing (R6 to R8) . . . 51
5.7 Telemetry, logging and recovery (R9, R10) . . . 54
5.8 Driver display (R11, R12) . . . 55
5.9 Integration and power . . . 56
5.10 Summary against the requirements . . . 57

6 Discussion **58**

6.1 Acquisition integrity and what it costs . . . 58
6.2 Ride characterisation (RQ1, RQ4) . . . 58
6.3 Hazard sensing . . . 59
6.4 Data integrity and connectivity (RQ3) . . . 60
6.5 Driver display (RQ2) . . . 60
6.6 Integration and power . . . 60
6.7 Validity and limitations . . . 61

7 Conclusions **62**

8 Recommendations **64**

8.1 Next steps for this system . . . 64
8.2 Towards autonomous operation . . . 65

8.3 The bigger picture . . . 66

8.4 Handing the project on . . . 67

A Graduate Attributes, AI Use and Project Repository **73**

A.1 Graduate attribute evidence . . . 73

A.2 Project repository . . . 75

A.3 Declaration of Artificial Intelligence Use . . . 75

B Supporting Figures **76**

C Wiring and Connections **79**

C.1 Pin references . . . 79

C.2 Raspberry Pi 4 to the time-of-flight sensor board . . . 79

C.3 Telemetry unit (ESP32-S3) to the Raspberry Pi 4 . . . 82

C.4 Telemetry unit sensors and antennas . . . 82

C.5 Camera, displays and power . . . 84

C.6 Wiring checks . . . 84

D Test Evidence **85**

E Configuration and Development Record **92**

E.1 Firmware configuration . . . 93

E.2 Development faults . . . 94

E.3 Diagnosis records . . . 96

E.4 Supporting detail . . . 97

F Enclosure and Mount Design **101**

F.1 Requirements and measurements . . . 101
F.2 Design choices . . . 101
F.3 Dashboard display mount . . . 104
F.4 Rear sensor pod (Raspberry Pi 4 and telemetry unit) . . . 104
F.5 IMU pods . . . 106
F.6 Front sensor pod for the Raspberry Pi 5 . . . 108
F.7 Ventilated lids . . . 109
F.8 Verification of the geometry . . . 112
F.9 Manufacture . . . 113
F.10 Outstanding . . . 115

G Dashboard Interface Design **116**

G.1 Starting point and constraints . . . 116
G.2 Design process . . . 116
G.3 Design rules . . . 118
G.4 The pages . . . 119
G.5 Verification . . . 119

H Ethics Approval **123**

I Photographs of the First Road Session **126**

I.1 Rear sensor pod . . . 126
I.2 The system on the vehicle . . . 128
I.3 The dashboard on the handlebar . . . 129
I.4 Front sensor board on the vehicle . . . 130

Chapter 1

Introduction

1.1 The vehicle and the problem

The TUKZIE Flagship Programme is developing an energy-aware, secure and autonomy-ready platform for last-mile logistics and urban freight, a market that is growing as cities grow [1]. Its testbed is the TUKZIE Rev 0, a 72 V electric cargo trike manufactured by Jinpeng [2]. Inspection of the motor and controller nameplates recorded a three-phase, permanent-magnet, sensored hub motor rated at 72 V DC and 35 A with a controller current limit of 90 A, about 2.5 kW of rated input power [3]; the vehicle's own plate (Figure B.1, Appendix B) gives model L2e-U, a rated output of 3.9 kW and a rated speed of 42 km/h. Electric tricycles of this kind already carry urban goods in African cities at lower operating cost than motorcycles and light cars (Section 2.2).

At the start of this project the vehicle had a touchscreen dashboard on a Raspberry Pi 5, written by earlier vacation-work teams, but no live data: every page was driven by simulated values (Section 4.8.4). Nothing measured how the trike responds to the road and its own powertrain, the battery state was not read, and there was no record of a journey. A vehicle intended for fleet use and later autonomy needs all of these, and it needs them on a platform that vibrates strongly, has limited power and computing, and loses cellular coverage. Road traffic injuries are also a leading cause of death, mostly in low- and middle-income countries [4], so hazard alerts for the driver are worth having, provided that the display does not itself become a distraction: NHTSA guidance limits single glances away from the road to 2 s and a task to 12 s in total [5].

The engineering problem is therefore to acquire synchronised data reliably on a high-vibration, power-constrained vehicle; to distinguish road-induced motion from vehicle-specific dynamics; to keep processing and display latency bounded; to preserve telemetry through intermittent connectivity; and to present the result to the driver without obstructing the view or making safety-relevant behaviour depend on a remote service [6].

The objective of this project is to develop a high-performance embedded computing telemetry unit for the TUKZIE Rev 0, integrated with the vehicle's existing dashboard, that acquires synchronised inertial, battery, positioning and camera data, characterises the vehicle's dynamic response at the edge, preserves telemetry through intermittent cellular connectivity, and presents a minimal set of safety-relevant information to the driver on that dashboard.

1.2 Deliverables and scope change

The brief [6] asked for: (1) an embedded high-performance embedded computing (HPEC) telemetry unit with a deterministic multi-threaded architecture for sensor polling, signal processing, display rendering and networking; (2) a driver interface, originally a windshield head-up display (HUD); (3) synchronisation of telemetry with a central database over 4G/LTE; and (4) a technical report validating the data and the system. During the project the windshield HUD was replaced, with the supervisor's agreement on 1 October 2026 and a title change recorded by the course coordinator on 2 October, by integration with the vehicle's existing dashboard: that dashboard already carries the information a HUD would show, a second display would compete for the same attention [7], and the camera's hazard alerts serve the programme's move towards autonomy better than a second screen (Section 2.6). The camera hazard detector was added to the scope on the supervisor's direction, and time-of-flight proximity sensing after a later request for a live nearby-object alert. A second camera and time-of-flight board were added at the rear in October, with a warning for objects closing on the vehicle.

The intended outcome was an integrated prototype demonstrated on the vehicle, Technology Readiness Level 6 [8], not a product qualified for unrestricted road use.

1.3 Research questions

RQ1 How accurately and repeatably can a low-cost edge-processing system characterise the trike's dynamic response to road- and powertrain-induced excitation in real time, using inertial sensors on a 72 V electric cargo trike?

RQ2 Which information hierarchy, layout and update strategy can present safety-critical telemetry and hazard alerts on the vehicle's existing dashboard while limiting visual clutter and driver distraction?

RQ3 What end-to-end latency, data-loss and recovery performance can be achieved across sensor acquisition, edge processing, local logging and 4G/LTE synchronisation?

RQ4 Which time- and frequency-domain features most effectively separate road-induced vibration from repeatable motor and powertrain vibration?

1.4 Engineering requirements and acceptance criteria

The research questions are answered through the engineering requirements in Table 1.1. Each requirement has an acceptance criterion fixed before the test, taken from the project's test procedures (`BenchTest/TEST_PROCEDURES.md`, test identifiers in the last column) or, for removability, from the vehicle integration and test plan (SW7-VIT-001). Chapter 5 reports each against its criterion (Table 5.6).

1.5 Scope and limitations

In scope and delivered, tested on the bench and run once on the vehicle: the ESP32-S3 telemetry unit (two IMUs, GNSS, battery-management system over Bluetooth Low Energy, LTE, local logging, and replay

Table 1.1: Engineering requirements, acceptance criteria and the research question each serves.

ID	Requirement	Acceptance criterion	RQ	Test
R1	Fixed-rate dual-IMU acquisition above 160 Hz, the Nyquist rate of the 80 Hz ISO 2631-1 band	Both IMUs at 195 Hz or more; zero dropped samples; with logging off no inter-sample gap over 10 ms, with buffered logging gaps over 10 ms only at a flush	1, 3	T1, T2, T9
R2	Per-sensor calibration	Resting magnitude 9.80 to 9.82 m/s^2 on both IMUs	1	T4
R3	Correct on-board features and fusion	Synthetic-sine self-test within 5%; all fusion self-tests pass; every fused window from both sensors	1, 4	T1
R4	Position	3D fix; parsed position within 10 m of a phone reference; no damaged reply accepted	1, 3	T3
R5	Battery telemetry	Connection and checksum-valid frames; pack voltage matches a meter	3	T7
R6	Camera to IMU time alignment	About 2 edges/s; median interval 500 ms; none outside 500 ± 20 ms	1	C6
R7	Hazard detection	Runs a full minute at 5 frames/s or more; person logged as person; alert onset (three frames plus latency) under about 0.5 s	2, 3	C1, C2, C8
R8	Proximity range	Three sensors answer and respond to the matching hand; within about 5% of a tape below 1 m; fused distance median error 5 cm or less at 0.5 and 1.0 m	2	C7, C9
R9	Store-and-forward telemetry	Acknowledged publishes; loss reported and reconnection without intervention after coverage returns; every stored record read back intact and parsing; every record stored once in the database after an outage	3	T5, T6, T10, T15
R10	Local dashboard link	One line per second, no missing sequence numbers, valid JSON; record under 3 s old at the dashboard; IMUs unaffected	2, 3	T11, T12
R11	Driver display shows only valid data	Only live values, "--" when missing, never simulated values; camera alerts listed and cleared; offline state shown	2	B2, B3
R12	Glance demand	Single glances within the 2 s NHTSA guidance [5]	2	on vehicle
R13	Removable installation	No drilling or permanent change; vehicle back to standard in about 20 min	all	VIT-001

into a database); two camera units with a pretrained object detector, facing forward on the dashboard's Raspberry Pi 5 and backward on a Raspberry Pi 4, the rear one time-synchronised to the inertial stream; three time-of-flight proximity sensors on each, fused with the camera detections; and the integration of all of these with the existing PySide6 dashboard on the Raspberry Pi 5. In scope but not delivered by the time of writing: the Sensirion SEN55 air-quality sensor (received, cable outstanding, Section 4.7); the 8×8-zone VL53L5CX time-of-flight sensor (three VL53L0X sensors stand in); and the motor Hall tap. The windshield HUD is not built.

The limitations are: one vehicle and one set of hardware; a pretrained, general-purpose detector that reports object class, a coarse distance band, confidence, timestamp and alert status only, stores no frames, identifies no individual, follows an object only from frame to frame to estimate its time to contact and does not recognise it again later, and controls no vehicle function; calibration under bench conditions, without the effect of temperature or humidity being measured; vehicle testing limited by time to urban routes, without rural or off-road conditions; and results that are mostly from the bench, with one road session.

1.6 Terminology

Table 1.2 lists the abbreviations and terms used in this report.

Table 1.2: Abbreviations and terms.

Term	**Meaning**
AT command	A text command sent to the modem over its serial line, such as `AT+CGNSSINFO`
BLE	Bluetooth Low Energy, the radio link to the battery-management system
BMS	Battery-management system of the traction pack
Bridge	The program on a Raspberry Pi (`telemetry_bridge.py`) that reads the telemetry unit and the time-of-flight sensors and serves them to the dashboard
Camera unit	A Raspberry Pi with a camera, the detector and three time-of-flight sensors: the Raspberry Pi 5 at the front, the Raspberry Pi 4 at the rear
COCO	Common Objects in Context, the image dataset on which the detector was trained
Dashboard line	The one-line JSON record the telemetry unit sends each second for the dashboard
DIO	Dual-I/O, a flash memory access mode
FreeRTOS	The real-time operating system kernel on which the ESP32-S3 firmware runs its tasks
Fusion	Matching camera detections with time-of-flight distances by bearing and time
GNSS	Global navigation satellite system: satellite positioning, such as GPS
GPIO	General-purpose input and output pin
HPEC	High-performance embedded computing
HTTP	Hypertext Transfer Protocol, used between the Raspberry Pis
HUD	Head-up display, which places information in the driver's forward view, in the original brief on the windshield
I^2C	Inter-Integrated Circuit, a two-wire bus for the IMUs and time-of-flight sensors
IMU	Inertial measurement unit: accelerometer and gyroscope (MPU6050)
IRI	International Roughness Index
JSON	JavaScript Object Notation, the text format of every record
LiDAR	Light detection and ranging
LittleFS	The file system in the telemetry unit's internal flash
LTE	Long Term Evolution, the 4G cellular network; Cat-1 is its low-rate category
MQTT	Message Queuing Telemetry Transport, a publish and subscribe protocol through a broker

Term	Meaning
NHTSA	National Highway Traffic Safety Administration (United States)
NoIR	A camera module without an infrared-cut filter
PSRAM	Pseudo-static RAM, external memory on the ESP32-S3 module
QoS	Quality of service level of an MQTT publish; at QoS 1 the broker acknowledges each record
R1 to R13	The engineering requirements of Table 1.1
RMS	Root mean square
RQ1 to RQ4	The research questions of this chapter
SoC	State of charge of the battery, in per cent
Store-and-forward	Logging every record locally and sending, or replaying, it when coverage allows
T, C, B tests	Test identifiers in `BenchTest/TEST_PROCEDURES.md`: T for the telemetry unit, C for the camera units, B for the bench tools and dashboard
Telemetry unit	The ESP32-S3 board with its modem, GNSS and two IMUs
ToF	Time of flight: optical ranging by the round-trip time of light (VL53L0X, VL53L5CX)
TRL	Technology Readiness Level
TTC	Time to contact: the time until an object closing at constant speed reaches the vehicle
UART	Universal asynchronous receiver and transmitter, a serial link
USB	Universal Serial Bus
UTC	Coordinated Universal Time

1.7 Report outline

Chapter 2 reviews the literature on ride sensing, edge telemetry, driver displays and proximity sensing. Chapter 3 sets out the methodology: the research approach and its phases, the experiments, and how data were collected and analysed. Chapter 4 describes the design as built and how each part links to the requirements. Chapter 5 gives the results against the requirements, Chapter 6 interprets them, Chapter 7 answers the research questions and Chapter 8 sets out recommendations. Wiring, configuration, test evidence and the development record are in the appendices.

Chapter 2

Literature Review

2.1 Scope and evaluation framework

The system combines four fields that are usually studied apart: ride and road sensing, embedded signal processing, vehicular telemetry, and driver-display design. The literature is judged here by what the TUKZIE Rev 0 needs, not only by reported accuracy: computational cost, sampling requirements, sensitivity to speed and mounting, real-time latency, communication loss and driver workload. A technique that works on a passenger car, a smartphone or a simulator may not transfer to a light three-wheeled cargo vehicle.

Sources were found by an automated search of OpenAlex, arXiv and Crossref on a fixed query set (ride characterisation, three-wheeler dynamics, cellular vehicle telemetry, MQTT, IMU calibration and fusion, camera hazard detection on edge hardware, driver displays and BMS protocols), run three times on 19 September 2026 and holding 733 records (`Research/research.db`), and by a targeted search of the three-wheeler market and open-source projects. The gap claim in Section 2.11 is bounded by this search.

The chapter is ordered as follows. Section 2.2 sets out why an electric cargo trike needs on-vehicle measurement. Sections 2.3 and 2.4 review ride response and the processing of inertial signals. Section 2.5 covers edge computing and real-time architecture, Section 2.6 driver-display design and Section 2.7 cellular synchronisation. Sections 2.8 and 2.9 review camera detection and proximity sensing, and Section 2.10 compares the hardware platforms that can carry this work. Section 2.11 draws the findings together, states the gap and lists the methods carried forward. Figure 2.1 shows which research question (Chapter 1) each theme informs.

2.2 Electric cargo trikes and the case for telemetry

Light electric freight vehicles can reduce the cost and emissions of urban delivery, but by how much depends on route, payload, prices and the comparison vehicle. In Dar es Salaam an electric cargo tricycle had operating costs 45.5% below a motorcycle and 86% below a light car on the routes studied [9]; in Portland, Saenz et al. found greenhouse-gas reductions of 51 to 72% for an electric-tricycle logistics service [10]. Both figures were obtained by measuring energy, distance and operations on the vehicles in

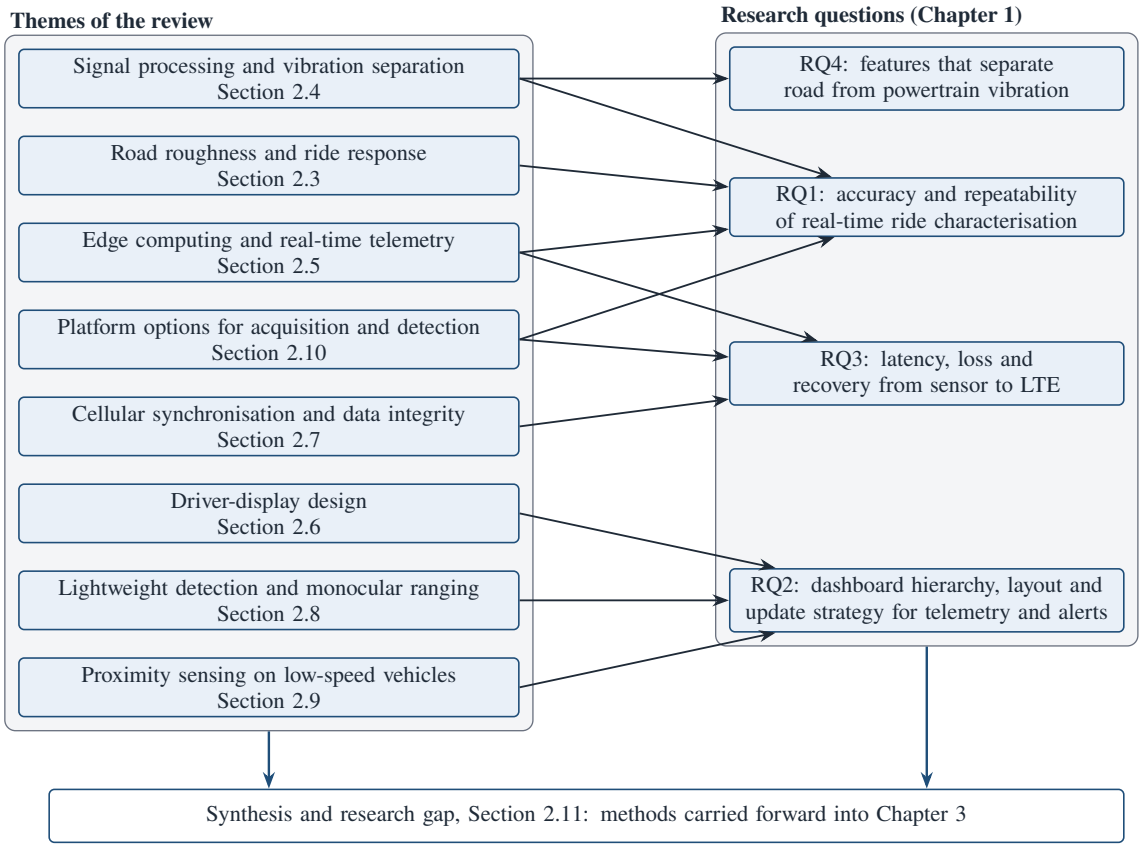


Figure 2.1: The themes of this review mapped onto the research questions of Chapter 1. An arrow means the section supplies a method, a limit or a test used for that question. Section 2.2 gives the context for all of them.

service, and an operator can confirm such benefits for its own fleet only by measuring the same quantities. On-vehicle telemetry is the instrument for this, which is the link between the logistics literature and this project. A three-wheeled chassis also makes vehicle response an engineering concern: payload position, tyre pressure, speed and surface affect both stability and measured vibration, so operating state should be recorded with the inertial data, and passenger-car roughness thresholds should not be transferred directly.

2.3 Road roughness and ride response

The International Roughness Index (IRI) is computed from a measured road profile through a standard quarter-car model [11]. An accelerometer on a vehicle measures something else: the combined response of road, tyres, suspension, chassis, payload, speed and mounting. González et al. classified road profiles from simulated vehicle accelerations [12], but the inference depends on the assumed vehicle model, which cannot be carried over to a trike. Smartphone and low-cost studies extend the response-based approach, and identify placement, orientation, speed, sampling rate and inconsistent ground truth as persistent sources of variation [13]; Alatoom and Smadi's microcontroller and single-board-computer framework is closer to this project because it allows controlled mounting and deterministic sampling [14]. None of these establishes a calibrated ride measure for a three-wheeled cargo vehicle. The defensible first claim for this project is therefore a repeatable, vehicle-specific response index and event log, validated by repeatability over traversals and sensitivity to speed and payload, not an IRI.

2.4 Inertial signal processing and vibration separation

An inertial signal contains gravity, orientation changes, road excitation, body motion, drivetrain vibration, noise and transients. Features from a nominally vertical axis are comparable only after orientation compensation, and become unreliable when samples are irregular or missing. Time-domain features (root-mean-square, standard deviation, peak-to-peak, crest factor, jerk, threshold counts) are cheap and interpretable; spectral energy and power spectral density separate sustained narrow-band vibration from broadband impacts. Bui et al. show rough-road detection running on an edge microcontroller [15]. Yiliguoqi et al. classified roughness with an SVM on vertical acceleration and roll features at only 10 Hz, correctly in 80 to 100% of trials across four vehicles [16]; that range is tied to their phones, routes, labels and thresholds and cannot be expected on the TUKZIE without retraining. Learned classifiers should therefore follow a transparent statistical baseline, not replace calibration.

Separating road excitation from powertrain vibration needs an empirical baseline: stationary runs at several motor speeds identify repeatable powertrain components, and coast-down and repeated route runs show which components remain without traction and which change with the surface. Windowed statistics should be normalised or stratified by speed, because the same road feature excites the vehicle at a different frequency at a different speed [17]. ISO 2631-1 defines the evaluation of whole-body vibration up to 80 Hz [18]; its weighted measures describe comfort, which should be reported separately from vehicle response and road geometry.

2.5 Edge computing and real-time telemetry architecture

Edge computing moves processing close to the sensors to reduce response time, bandwidth and dependence on a remote service [19], which suits a vehicle whose alerts must work without coverage and whose raw 200 Hz data would cost more to transmit than compact features. Embedded HPEC also allows several streams to be timestamped and fused before data leave the vehicle [20]. "High performance" here means predictable completion of time-critical work within a small power budget, not throughput. Threads alone do not give real-time behaviour: shared locks, blocking input and output, allocation and unbounded queues still cause missed samples or stale displays. A producer and consumer design with timestamped samples in bounded queues, explicit overload behaviour (an old display frame may be dropped, an inertial sample must not be lost silently) and reporting of jitter, queue occupancy, display age and end-to-end latency as distributions is therefore the appropriate pattern.

Pérez-González et al. give a close comparison: an electric-racing telemetry system with ESP32 acquisition, Raspberry Pi 4B analytics and LoRaWAN, reporting 99% packet delivery and sub-second latency on their track [21]. The division between deterministic acquisition and higher-level processing supports the architecture used here; the radio result is specific to their gateway and route. LTE gives wider reach at the cost of subscription, coverage dependence and less predictable delay, which argues for local logging and asynchronous transmission whichever radio is used.

2.6 Driver-display design

Any display, head-down or head-up, draws the driver's eyes or attention from the road. NHTSA's glance limits (Chapter 1) treat a vehicle stopped at a light as being driven [5]. A head-up position does not remove the problem: HUD use changes attention demand and driving performance [22], and head-down assessment methods do not fully capture HUD behaviour [23]. A windshield HUD also brings optical problems: a double image from the two glass surfaces that wedge glass corrects only for one viewing geometry, an eye box that must be measured per installation, and legibility that depends on ambient light [24, 25, 26]; misaligned or excessive graphics compete with hazards [27]. Each is harder on a removable, low-cost unit on a vibrating trike in daylight, and where a dashboard already carries speed and battery state a HUD adds a competing channel [7]. These are the reasons the project used the existing dashboard.

For a dashboard, the literature gives five design issues against which a page can be checked. Layout and clutter: symbol count was the most influential interface factor in a semi-automated driving study, which recommended fewer than six symbols at a time [28], and added channels raise mental demand [7], so a stable hierarchy (persistent speed and battery, brief prioritised alerts, navigation only when needed) is preferred. Legibility and contrast: ISO 15008 sets character height by visual angle, 20 arc minutes recommended and 16 acceptable, as reported in [29, 30]; at about 70 cm from a 10.1-inch, 1280 by 800 panel, 20 arc minutes is a capital height of about 24 pixels, and a humanist typeface cut total glance time by 10.6% among male drivers [29]. Interaction: touch keys improved performance up to about 17.5 mm [31], Euro NCAP's 2026 protocol sets a minimum touch-target size [32], and entering a destination took about 40 s on production systems [33], so such tasks belong to the parked state. Alert priority: safety alerts should be few, ordered and distinguishable from status. Data validity and update age: the reviewed guidance concerns legibility and workload and does not address stale data, but a display that keeps showing a value after its source has stopped presents an estimate as a measurement; this project therefore treats missing and out-of-date data as a display requirement in its own right (R11). Evaluation should combine technical measures (render time, update rate, contrast in representative light, age of the

data shown) with glance behaviour and workload in controlled trials.

2.7 Cellular synchronisation and data integrity

The brief names MQTT and HTTP over 4G/LTE and asks for local logging and synchronisation with a central database [6]. Since coverage and delay cannot be controlled, this project keeps the real-time path separate from synchronisation and measures delivery rather than assuming it: each record carries a timestamp, a session identifier and a sequence number so that gaps and duplicates can be detected, and the measures are completeness, backlog, delay, duplicates and recovery after an induced outage.

2.8 Lightweight detection and monocular ranging

A camera alert on a vehicle without an accelerator needs a detector that runs on a general-purpose processor. SSD predicts classes and boxes for a fixed set of default boxes in one pass [34]; MobileNet cuts the arithmetic of the feature extractor with depthwise separable convolutions [35]; and integer-only quantisation runs such networks in 8-bit arithmetic at a small cost in accuracy [36]. A model trained on COCO knows only its classes, which include people, vehicles and animals but no road-surface hazards [37]. In a benchmark on edge devices including the Raspberry Pi 4, SSD MobileNet V1 had the lowest latency and energy and the lowest accuracy [38], the trade made here. Monocular distance rests on an assumption: inverse perspective mapping, which assumes a flat road, performs poorly for distant objects and on curves [39], and the width heuristic used here assumes a width per class, so it is indicative only and is fused with time-of-flight ranging. Whether an object is closing matters as much as how far away it is, and that does not need the distance: for constant closing speed, the time to contact equals an object's image size divided by its rate of growth [58]. Single-camera forward-collision warning in production vehicles is built on this scale change rather than on an absolute range [59], which suits an uncalibrated camera. Objects outside the trained classes, such as debris, are a separate problem, approached with synthetic data [40], anomaly-segmentation benchmarks [41] and detectors trained on rare roadside obstacles [42].

2.9 Proximity sensing on low-speed vehicles

Nearby-object alerting is treated as a requirement even for slow vehicles: De Simone et al. classified ultrasonic ranges for obstacle avoidance on retrofitted agricultural machines [43], and Noomwongs et al. combined RTK-GNSS with a 2D LiDAR on a low-speed micro electric vehicle, because vision alone needed computing such vehicles lack [44]. A multi-zone time-of-flight sensor such as the VL53L5CX resolves direction from one device; an independent characterisation found it near-linear against a white target but with a standard deviation near 50 mm beyond 400 mm and only 18.8 to 34.4% valid readings against a black target, and a warm-up drift of about 15 minutes [45]. Range on dark targets and in sunlight therefore has to be measured on the vehicle, not taken from the datasheet.

2.10 Platform options for acquisition and detection

The reviewed systems use three classes of hardware, and each suits a different part of the workload. A microcontroller can sample at a fixed period and process short windows at low cost: Bui et al. run rough-road detection on an edge microcontroller [15], and Alatoom and Smadi's framework on microcontroller and single-board-computer platforms allows deterministic sampling [14]. A microcontroller cannot, however, run a camera network at a useful rate. A single-board computer such as the Raspberry Pi 4 can run a quantised detector on its general-purpose processor [36, 38], but its Linux scheduler gives no guarantee of a fixed sampling period. Pérez-González et al. divide the work between the two, with ESP32 acquisition and Raspberry Pi 4B analytics [21]. The third option is a neural accelerator beside a single-board computer, which would raise the detector's frame rate at the cost of a further device to integrate; without one, vision needs computing that small vehicles may lack, which is why Noomwongs et al. used LiDAR on a low-speed micro electric vehicle [44].

The literature therefore points to a split design: deterministic acquisition on a microcontroller, detection on a single-board computer, and an accelerator as an option to be measured rather than assumed. Table 4.1 in Chapter 4 compares these options with the figures measured in this project, and Section 4.1 gives the choice made.

2.11 Synthesis and research gap

The reviewed work shows that low-cost inertial sensors can detect road anomalies, that edge devices can process sensor windows locally, and that in-vehicle displays can be read within glance limits when sparse and legible. It also shows three limitations: response-based roughness depends strongly on vehicle, speed, mounting and calibration; edge architectures are usually reported by accuracy or mean latency, without jitter, queue behaviour or outage recovery; and display guidance comes almost entirely from passenger cars.

The gap claim is bounded by the search made for it (Section 2.1). The automated search of OpenAlex, arXiv and Crossref, run three times on 19 September 2026 and holding 733 records, and the targeted search of the three-wheeler market and open-source projects found no system that integrates these concerns for a light electric cargo trike: fixed-rate dual-IMU acquisition with measured jitter and loss, on-board ride features, camera detection fused with time-of-flight ranging, LTE telemetry with local logging and replay, and presentation on the vehicle's own dashboard with missing data shown as absent. One reported system combining these elements on a light electric vehicle would have falsified the claim; Table 2.1 compares the closest systems cited here.

The same search narrowed two earlier claims (`Research/gap_ledger.json`): open-architecture inertial, GNSS and cellular telemetry units already exist on two-wheeled racing vehicles, and local logging beside a cellular uplink appears in prior work, although its recovery is rarely measured. The contribution claimed is therefore the integration and its measurement on this vehicle class, not any single component.

Three observations close the review. Response-based ride measures belong to the vehicle, its speed and its mounting, so a trike needs its own baseline rather than a threshold transferred from a car. A real-time telemetry system is judged by its worst cases and its recovery, not by its mean latency. Display guidance from passenger cars bounds what a trike dashboard should show, but has to be checked on the vehicle.

Table 2.1: Closest cited systems against the elements of the gap claim. "-" means not described in the source as reviewed here (its abstract and the passages cited), not proven absent.

System	**Fixed-rate IMUs, jitter and loss measured**	**On-board ride features**	**Camera fused with ranging**	**Cellular, local log and replay**	**Vehicle's own dashboard**	**Missing data shown absent**
Pérez-González et al. [21]	Packet delivery measured, not sampling	-	-	LoRaWAN, not cellular	-	-
Alatoom and Smadi [14]	Deter-ministic sampling	Pavement condition	-	-	-	-
Bui et al. [15]	-	Rough-road detection	-	-	-	-
Yiliguoqi et al. [16]	Phone, 10 Hz	-	-	-	-	-
Noomwongs et al. [44]	-	-	2D LiDAR, no camera	-	-	-
This system	Yes, two IMUs	Yes, time domain	Yes (bench)	Yes (LTE)	Yes	Yes

The gap is the integration of these elements on a light electric cargo trike, measured as a whole. The following methods and theories are carried forward:

- a vehicle-specific response index and event log, validated by repeatability over traversals and sensitivity to speed and payload, in place of an IRI (Section 2.3);
- orientation compensation, windowed time-domain features, spectral features for narrow-band powertrain vibration, and speed-stratified comparison against stationary and coast-down baselines (Section 2.4);
- a producer and consumer design with timestamped samples in bounded queues, with jitter, queue occupancy, display age and latency reported as distributions (Section 2.5);
- records with a timestamp, session identifier and sequence number, local logging, and measured completeness, backlog and recovery after an outage (Section 2.7);
- a sparse, stable dashboard hierarchy checked against glance limits, ISO 15008 character height and touch-target size, with missing and out-of-date data shown as such (Section 2.6);
- a quantised single-pass detector, time to contact from image scale, and fusion with time-of-flight ranging measured on the vehicle (Sections 2.8 and 2.9);
- the split between deterministic acquisition on a microcontroller and detection on a single-board computer (Section 2.10).

Chapter 3 sets out how each of these was tested on the TUKZIE Rev 0, and Chapter 4 how it was implemented.

Chapter 3

Methodology

Chapter 2 ended with a bounded gap (Section 2.11, Table 2.1). No reported system combines fixed-rate dual-IMU acquisition with measured jitter and loss, on-board ride features, camera detection fused with ranging, cellular telemetry with local logging and replay, and presentation on the vehicle's own dashboard, for a light electric cargo trike. The review also found that edge architectures are usually reported by accuracy or mean latency, without jitter, queue behaviour or outage recovery, and that response-based ride measures depend on the vehicle, speed and mounting. This chapter sets out how the project answers RQ1 to RQ4 under those findings: the research approach and its phases, the sites, the experiments, how data were collected and analysed, and the ethics and safety conditions. It describes what was done to obtain the evidence. What was built is described in Chapter 4.

3.1 Research approach

The project is engineering design research. A prototype is built and then measured against criteria set before each test, and the measurements answer the research questions. The four questions are answered through the thirteen engineering requirements of Table 1.1, each with an acceptance criterion and a test identifier.

The work follows a V-model (Figure 3.1). The left arm defines the system: the problem and research questions, then the requirements and their criteria, then the design. The right arm checks it at the same levels, from the bottom up: unit and start-up checks confirm each interface, bench tests check each requirement against its criterion, and vehicle tests check what only the vehicle can show. The approach was chosen for three reasons. First, the gap is about measured behaviour (timing, loss, recovery), so each design stage needs a matching test that produces a number. Second, faults in a system of this kind often appear only when parts are combined, so each interface is checked before the next is added, and a fault can be traced to one change. Third, the target is a prototype demonstrated on the vehicle, Technology Readiness Level 6 [8], which requires bench validation of each part before testing in the relevant environment. The criteria come from the literature where it gives one: the 80 Hz upper limit of the ISO 2631-1 band for the sampling rate [18], the 2 s single-glance limit for the display [5], and the completeness, duplicate and recovery measures of Section 2.7 for synchronisation.

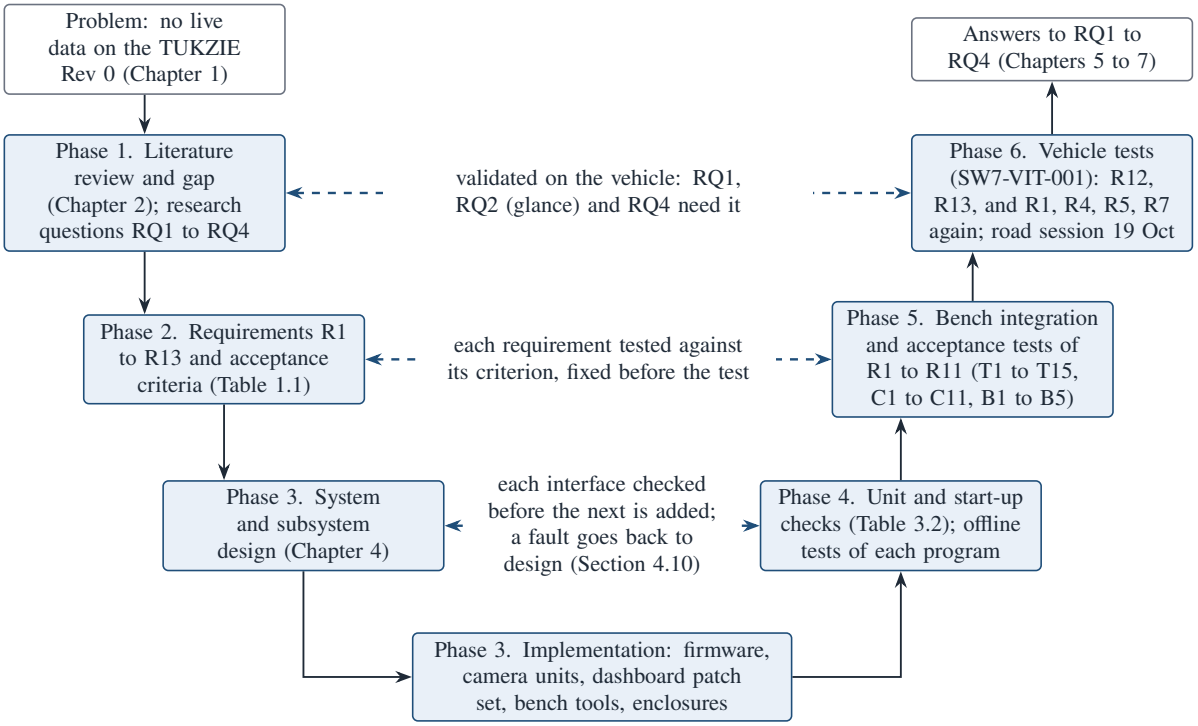


Figure 3.1: Research approach as a V-model. Phases 1 to 3 define and build the system; phases 4 to 6 check it at the matching level (dashed links). Phases 3 to 5 were repeated for each subsystem as it was added. The dates of each phase are in Table 3.1 and Figure E.1.

3.2 Phases as run

Table 3.1 gives the six phases of Figure 3.1 as they were actually run, with dates from the repository's commit history, the project log, the change log and the lab notebook. Figure E.1 (Appendix E) draws the same record as a timeline. The phases overlapped. Phases 3 to 5 ran as a loop for each subsystem in turn: for example, the rooftop GNSS test of 29 to 30 September (phase 5) exposed a reply layout other than the one assumed, which sent the parser back to design (phase 3) for firmware v0.5.1 to v0.5.3 and a retest. Two scope changes also fed back into phases 2 and 3: the windshield HUD was replaced by integration with the existing dashboard on 1 October, and the hardware was rearranged into front and rear units on 15 October.

3.3 Site selection

The bench was the laboratory at UCT, where the telemetry unit, both Raspberry Pis and the dashboard were set up together (Figure 4.2). Most tests were run there because the bench allows one change at a time, repeated runs and a laptop beside the hardware, which the vehicle does not. Two tests needed other places: the GNSS fix needs open sky, so it was tested on a rooftop (test T3), and the battery link needs the vehicle's own pack, so it was first tested on the vehicle (test T7). The vehicle is the TUKZIE Rev 0, the programme's only testbed, so the vehicle tests use one vehicle and one set of hardware (Chapter 1). It was in the laboratory, on the ground and raised on request. Stationary tests use rated stands with the drive wheel raised, and tests on the ground use a road. The first road session was run on 19 October 2026 (Section 3.4.3); its venue, route and conditions are still to be recorded there. No person takes part

Table 3.1: The phases of Figure 3.1 as run. Dates are from the git history, `PROJECT_LOG.md`, `CHANGELOG.md` and `Planning/lab-notebook.md`.

Phase	**What was done**	**Dates (2026)**	**Reported in**
1. Review and research questions	Papers collected; automated search of OpenAlex, arXiv and Crossref and the gap ledger (19 September); review aligned with the dashboard scope (7 October)	3 Aug to 7 Oct	Chapters 1 and 2
2. Requirements and criteria	Test procedures and pass criteria written (`BenchTest/TEST_PROCEDURES.md`, 29 to 30 September); tests added as subsystems were added (T15 on 11 October, C11 on 15 October)	29 Sep to 15 Oct	Table 1.1
3. Design and implementation	Firmware baseline (27 August); dual-IMU FreeRTOS firmware (16 September) to v0.7.4.1 (21 October); camera detector (16 September); dashboard integration (from 28 September); enclosures (12 to 19 October)	27 Aug to 21 Oct	Chapter 4
4. Unit and start-up checks	Each interface checked on the target board and the Raspberry Pis; faults diagnosed one change at a time (boot loop, 25 September, to the time-of-flight hang, 10 October, and the enclosed Raspberry Pi 4, 16 October)	25 Sep to 16 Oct	Table 3.2, Section 4.10
5. Bench integration and acceptance tests	T1 (25 September) to the Raspberry Pi 5 detector run (15 October); T15 outage test (11 October); wired serial link check (21 October)	25 Sep to 21 Oct	Chapter 5
6. Vehicle tests	First battery frames on the vehicle (30 September to 1 October); whole system mounted and first road session (19 October); data check (20 October)	30 Sep to 20 Oct	Section 3.4.3, Chapter 5

as a subject, so no participants were selected (Section 3.7).

3.4 Experiments

The experiments are of four kinds, matching phases 4 to 6 of Figure 3.1, and the checks in turn triggered fault investigations.

- *Unit and start-up checks* (phase 4): pass or fail checks of each interface, consolidated below.
- *Bench acceptance tests* (phase 5): one or more tests per requirement, identified in the last column of Table 1.1. Their procedures and pass criteria are in `BenchTest/TEST_PROCEDURES.md`: T1 to T15 for the telemetry unit, C1 to C11 for the camera units and B1 to B5 for the bench tools and the dashboard. Each run's raw output is indexed in Appendix D. Not every test has been run: T14 on the Raspberry Pis, C5 (begun on 19 October), C8, the tape measurements of C9 and C11 are planned or incomplete, and Chapter 5 marks them as outstanding.
- *Integration tests* (phase 5): the whole chain run together on the bench, described below.
- *Vehicle tests* (phase 6): the stationary and ground phases of the vehicle integration and test plan, described below.

- *Fault investigations*: where more than one cause could explain a fault, a deciding test was designed to separate them, changing one thing at a time (Section 4.10, Table 4.2).

3.4.1 Start-up and interface checks

The routine checks that establish each interface before the higher-level tests are consolidated in Table 3.2. They are pass or fail, and all passed; the faults they exposed are in Table E.2. The acceptance tests of Table 1.1 are reported in Chapter 5.

Table 3.2: Start-up and interface checks (bench).

Check	**Evidence required**	**Result**
Processor boot	Versioned banner, continuous status, no reset loop; bounded wait for a USB host	Pass
Flash configuration	16 MB flash in DIO mode with the 16 MB partition table; LittleFS mounts	Pass after the DIO fix
Serial routing	Application output on the target board's native USB port	Pass after the routing fix
Modem power-up	Modem answers `AT` without the button; key pulsed only if silent	Pass (v0.5.2)
UART, SIM, signal	`OK` to `AT`; `+CPIN: READY`; valid `AT+CSQ`	Pass; CSQ 22 to 30
LTE registration	`AT+CEREG?` and `AT+CPSI?` registered, `LTE,Online`	Pass; intermittent indoors
GNSS interface	Correctly framed `+CGNSSINFO` after warm-up	Pass
Manual bridge	Commands and replies passed both ways	Pass
I^2C levels	Idle bus at about 3.3 V (inputs not 5 V tolerant)	Pass, 3.3 V
Both IMUs on target board	Found, configured, calibrated, 400 to 401 samples per 2 s	Pass, 0 dropped
Raspberry Pi 4	Same OS release as the Pi 5; camera identified (OV5647); network and ssh reachable	Pass
Pre-test check	`BenchTest/sw7_check.py` reports every link before a session	16 offline tests pass; READY on 4 Oct except the Pi 4 supply

3.4.2 Integration and power

All telemetry-unit subsystems were first run together on the target board, and then the whole chain from the telemetry unit through the Raspberry Pi 4 to the dashboard on the Raspberry Pi 5. This exposed the Raspberry Pi 4's under-voltage, traced to the telemetry unit's modem (Section 4.10.7); the unit needs its own supply, and the Pi 4 now has a fan (Appendix C).

3.4.3 Vehicle tests

The vehicle integration and test plan (SW7-VIT-001) divides the vehicle work into a stationary phase with the drive wheel raised and a phase with the vehicle on the ground. The stationary phase covers mounting, calibration in the mounted position, acquisition timing, synchronisation, the Hall tap and

pole-pair count, a powertrain vibration baseline with the motor run at low, medium and high settings for about 60 s each, BMS voltage against a meter, GNSS and LTE static tests and an induced outage. The ground phase covers repeatability over repeated traversals, speed dependence, surface discrimination, sensor placement, coast-down separation of the powertrain, and payload. Sessions with the 72 V system energised require a second person, rated stands and chocks; the plan requires removable mounting, with no drilling or permanent modification, and a return to standard in about 20 minutes (R13), to be checked when the system is installed. *Recording on the vehicle.* A recorder runs on the Raspberry Pi 5 as a service that starts at boot (`BenchTest/road_logger.py`, `sw7-roadlog`). Twice a second it saves every reply of the four sources to one file per session: the front time-of-flight and fusion and the front alerts, read locally, and the rear unit's record (telemetry unit, rear time-of-flight, fusion) and rear alerts, over the Ethernet cable. So no laptop has to ride along. Runs are marked by lines in a marker file or by times on the run sheet (`Planning/road-test-2026-10-16-checklist.md`). Without internet on the road the computer's clock is not reliable after a restart, so GNSS time in the unit's record is the reference for aligning the logs and the video.

Vehicle state. No signal gives the gear or the direction of travel. Each second is labelled afterwards from three recorded signals: battery current (the motor is working), the vibration features (running or moving) and GNSS speed (moving, which separates driving from running on the stands). Reversing runs are identified from the markers and video. A direct gear signal would need the reverse switch tapped through an optocoupler into the telemetry unit, which touches the 72 V vehicle wiring and so needs the supervisor's approval.

A first road session was run on 19 October with the whole system mounted; photographs of the system on the vehicle are in Appendix efapp:road1. The vehicle was driven by Yusuf, not by the author. It followed the checklist in `Planning/road-test-2026-10-16-checklist.md` and was recorded on the vehicle from 17:15 to 19:04, with about 16 minutes of driving in 26 segments between 18:15 and 18:49. Driving was identified from the battery current and vibration: a second counts as driving when the current exceeds 1 A or the vibration value 0.3, and segments less than 10 s apart are joined. Because the recorder polls twice a second and the bridge repeats its last record, the analysis counts each record once, by its sequence number.

[Outstanding: road sessions: venue, route, conditions, mounting positions, the runs completed, and the second session.]

3.5 Data collection

Data collection was designed into the system so that each test leaves a record that can be checked later, on the bench or on the vehicle, without a separate measurement pass. The sources are as follows.

- *Inertial samples.* Each sample carries a sensor identifier, a per-sensor sequence number and a microsecond timestamp from a monotonic clock. A full queue drops and counts the new sample rather than block. Every 2 s a statistics task reports, per sensor, the achieved rate, the minimum, mean and maximum interval and the drop count (Section 4.3). Raw 200 Hz samples are not kept by firmware up to v0.7.4.1, only the 1 s features (Section 4.4).
- *Serial logs.* On the bench the unit's serial output is saved on the laptop by the bench console or a test script (Section 4.11), under `BenchTest/logs/`.

- *Local log.* Every record is written to the unit's flash whether or not there is coverage, and read back in full with an indexed, length-checked dump that repeats until every record has arrived intact (Sections 4.9 and 4.10.4).
- *Database.* Each record carries a boot counter, a sequence number and, with a fix, UTC time. The ingest program stores it once per device, boot and sequence number, and can backfill from a copy of the local log (Section 4.9).
- *Camera timing and alerts.* Every frame's latency is measured from its sensor timestamp and reported every 5 s as median, 95th percentile and maximum, optionally per frame (Section 4.8.2). Each alert is logged with its class, band, confidence, time and status, and with the side, time to contact and track number where they apply (Section 3.7).
- *Synchronisation pulse.* The ESP32 logs the microsecond time of each 500 ms edge, and the Raspberry Pi 4 logs its arrival from the kernel on the camera's clock (Section 4.8.1).
- *Fusion log.* Distances and bearings are logged only for a validation session, and then hold numbers only (Section 4.8.3).
- *Power and temperature.* A logger on the Raspberry Pi 4 records its power and temperature flags, I^2C addresses and USB devices every 2 s, and one on the Raspberry Pi 5 its supply, flags and temperature every 5 s (Section 4.10.7).
- *Road recorder.* On the vehicle, the recorder on the Raspberry Pi 5 saves every reply of the four sources twice a second, with a marker file for runs (Section 3.4.3).

Every record is kept unedited, under `Report/evidence/` or `BenchTest/logs/`, and indexed by date and test in Appendix D; each day's lab actions are also recorded in `Planning/lab-notebook.md`. The road-session files are too large for git and are kept on the project's cloud storage (`Planning/lab-notebook.md`, 20 October).

3.6 Data analysis

The analysis turns each record into a figure that can be compared with the acceptance criterion of Table 1.1, fixed before the test. Each requirement is then given a status in Table 5.6: met, partly met or outstanding, with the place of the test (bench or vehicle) stated. Results that were not measured are marked as outstanding rather than estimated.

- *Timing distributions.* Latency and interval distributions are skewed, and an alert is limited by its slow cases, so they are summarised by the median, the 95th percentile and the maximum rather than the mean. This also answers the review's finding that edge systems are usually reported by accuracy or mean latency, without jitter (Section 2.11). Acquisition is judged by the rate, the drop count and the number of gaps over 10 ms, each placed relative to the flash writes (R1).
- *Position.* Each fix is compared with a phone's position and summarised as the median, 95th percentile and maximum distance (R4).
- *Synchronisation pulse.* Intervals between logged edges are summarised by their median and range against 500 ± 20 ms (R6).

- *Store-and-forward.* Records are counted by device, boot and sequence number, giving missing records, duplicates, replays and the time to clear the backlog (`Server/sync_report.py`); the database is compared with the read-back of the unit's log (R9).
- *Road session.* The bridge repeats its last record until a new one arrives and the recorder polls twice a second, so records are first reduced to one per sequence number. Counting replies instead gave 19.5 rather than about 16 minutes of driving, and a 95th percentile of the driving vibration value of 2.0 rather than 3.1 (`Planning/lab-notebook.md`, 21 October). Each second is then labelled driving when the battery current exceeds 1 A or the vibration value 0.3, with segments less than 10 s apart joined, and parked otherwise. Medians, 95th percentiles and maxima are reported per state.
- *Fault investigations.* Each candidate cause is accepted or ruled out by its deciding test (Table 4.2).

RQ1 and RQ4 need analysis beyond this. Repeatability needs repeated traversals of the same road, and separating road from powertrain vibration needs frequency-domain features (Section 2.4) computed from raw samples, together with the motor speed. The first road session kept neither raw samples nor a GNSS fix, and the Hall tap is not wired, so these analyses have not yet been possible (Chapter 5).

3.7 Ethics and safety

The study was approved with conditions by the Engineering and Built Environment Research Ethics Committee on 6 October 2026, reference EBE/03305/2026, under the project's original title, which named the windshield head-up display (Appendix H). The approval rests on the study involving no human subjects and lapses on 31 December 2026. Its conditions are that a full proposal is given to the supervisor and that a physical risk assessment is completed to the satisfaction of the laboratory owner. Both were met: the full proposal was given to the supervisor, and the risk assessment was signed by the laboratory owner before the vehicle was driven for the road session of 19 October.

No person takes part as a subject: nobody is asked to do anything, and no glance or driver measurement has been made. On the road session the vehicle was driven by Yusuf, who operated it and was not measured. The cameras do see people on and beside the road, so what is kept was limited by design. Frames are processed in memory and never stored, and the live view is streamed only on the vehicle's own network. Each alert keeps its object class, distance band, confidence, time and status, with the side, the time to contact and a tracking number that restarts at every start-up. There is no face or number-plate recognition, the detector controls nothing on the vehicle and its warnings are advisory. Frames kept for calibration and as evidence show only the author.

The topic's ethics notes said that machine learning was not planned for the first version and that a revised application would be submitted before it was used. The application that was approved stated in its abstract that machine learning might be used once the baseline system was working, and the approval was granted on that application, so it covers this use. The detector is a pretrained neural network, and a second network was trained on public pothole datasets (Section 4.8.2); the project collects no training data of its own, and the limits above on what is kept apply to both.

During bench tests the telemetry records, including the unit's position on campus, went to a public test broker that anyone can read (Section 4.9). Before the vehicle is driven with a position fix, the position is

to be left out of the published record or a private broker with authentication used. On the vehicle, sessions with the 72 V system energised require a second person, rated stands and chocks, and the mounting is removable, with no drilling (Section 3.4.3).

[Outstanding: position removed from the published record, or a private broker, before a drive with a position fix.]

3.8 Summary of methods

Table 3.3 summarises the methods for each research question and its group of requirements.

Table 3.3: Summary of methods per research question and requirement group.

Research question / sub-problem	**Site / participant selection**	**Data collection methods**	**Data analysis methods**	**Appendix reference**
RQ1: ride response, with R1 to R4 and R6 (T1 to T4, T9, C6)	Bench at UCT; rooftop for GNSS; vehicle on stands and road; no participants	Per-sample sequence numbers and timestamps, 2 s statistics, 1 s features, local log, sync-pulse edges, GNSS replies	Rate, drops and gaps over 10 ms; resting magnitude; self-tests; distance to a phone (median, 95th percentile); pulse interval median and range	D, C, E (Table E.1)
RQ2: driver display and alerts, with R7, R8, R11, R12 (C1, C2, C7 to C9, B2, B3)	Bench, dashboard offscreen on a laptop and on the Raspberry Pi 5; vehicle; no participants (no glance study yet)	Per-frame latency, alert logs, time-of-flight readings, offscreen screenshots, touch-controller readings	Frame rate; latency median, 95th percentile and maximum; alert onset; distance against a tape (planned); pass or fail of the display rules	D, G, C
RQ3: latency, loss and recovery, with R1, R4, R5, R7, R9, R10 (T5 to T7, T10 to T12, T15)	Bench, with induced outages; vehicle for the battery link and the road session	Publish results, local log read-back, database records by boot and sequence number, dashboard lines, road recorder	Missing and duplicate records, drain time, reconnection time; lost lines; record age; records deduplicated by sequence number	D, E
RQ4: road against powertrain vibration, with R3	Vehicle: stationary powertrain baseline and repeated traversals (planned)	1 s features and disagreement; raw samples and motor speed needed (not yet collected)	Features per driving or parked second; frequency-domain features (planned)	D
All: installation and power, R13	Vehicle, 19 October	Mounting record, power and temperature logs	Return to standard timed (outstanding); power budget against the pack	F, C
Ethics and safety	No human subjects	Five alert fields; no frames stored	Not applicable	H

The next chapter describes the design that these methods test: the system architecture, the hardware and software of each unit, their integration, the testing infrastructure and the provision for maintenance, each linked to the requirements it serves.

Chapter 4

Design

Chapter 3 set out the approach: a V-model in which each design stage has a matching test, run in six phases. This chapter is the design and implementation of phase 3, the system as built on 21 October 2026, on which the experiments of Section 3.4 were run. It is structured as follows. Section 4.1 gives the system-level design and the choice of architecture. Section 4.2 covers the hardware. The software follows: the telemetry unit's firmware (Section 4.3), its ride-characterisation processing (Section 4.4), the battery-management interface (Section 4.5), positioning (Section 4.6), the other vehicle inputs (Section 4.7) and the camera subsystem (Section 4.8). The integration design is in the dashboard integration (Section 4.8.4), the two-sided layout (Section 4.8.6) and the communications and logging (Section 4.9). Section 4.10 gives the fault investigations that shaped the design, Section 4.11 the testing infrastructure and Section 4.12 the maintenance design. Each major section opens by naming the requirements of Table 1.1 and the research questions it serves, so that the chain from objectives to requirements, design, tests and conclusions can be followed.

Development followed a staged approach: each interface was tested on its own before higher-level functions were added, so that a fault could be traced to one change. The seven faults whose diagnosis shaped the design are worked through in Section 4.10; the full development record, with every fault, its cause and its fix, is in Appendix E, and the firmware configuration (versions, pins, bus speeds, task priorities, queue depths and partitions) is in Table E.1 of that appendix.

4.1 System overview and architecture selection

This section serves all thirteen requirements at system level, R1 and R7 in particular, and through them RQ1 to RQ4. Figure 4.1 shows the system. An ESP32-S3 telemetry unit samples two inertial measurement units (IMUs), reads GNSS and the battery-management system (BMS), computes ride features, logs locally and publishes over LTE. Two camera units each run the detector and fuse it with three time-of-flight sensors: the vehicle's existing dashboard computer, a Raspberry Pi 5 beside the display, faces forward, and a Raspberry Pi 4 beside the battery faces backward and also bridges the telemetry unit's data. The two are joined by an Ethernet cable (Section 4.8.6). In the first bench versions the Raspberry Pi 4 faced forward and the dashboard read it over Wi-Fi.

Table 4.1 compares the architectures that were feasible with the hardware available. The deciding

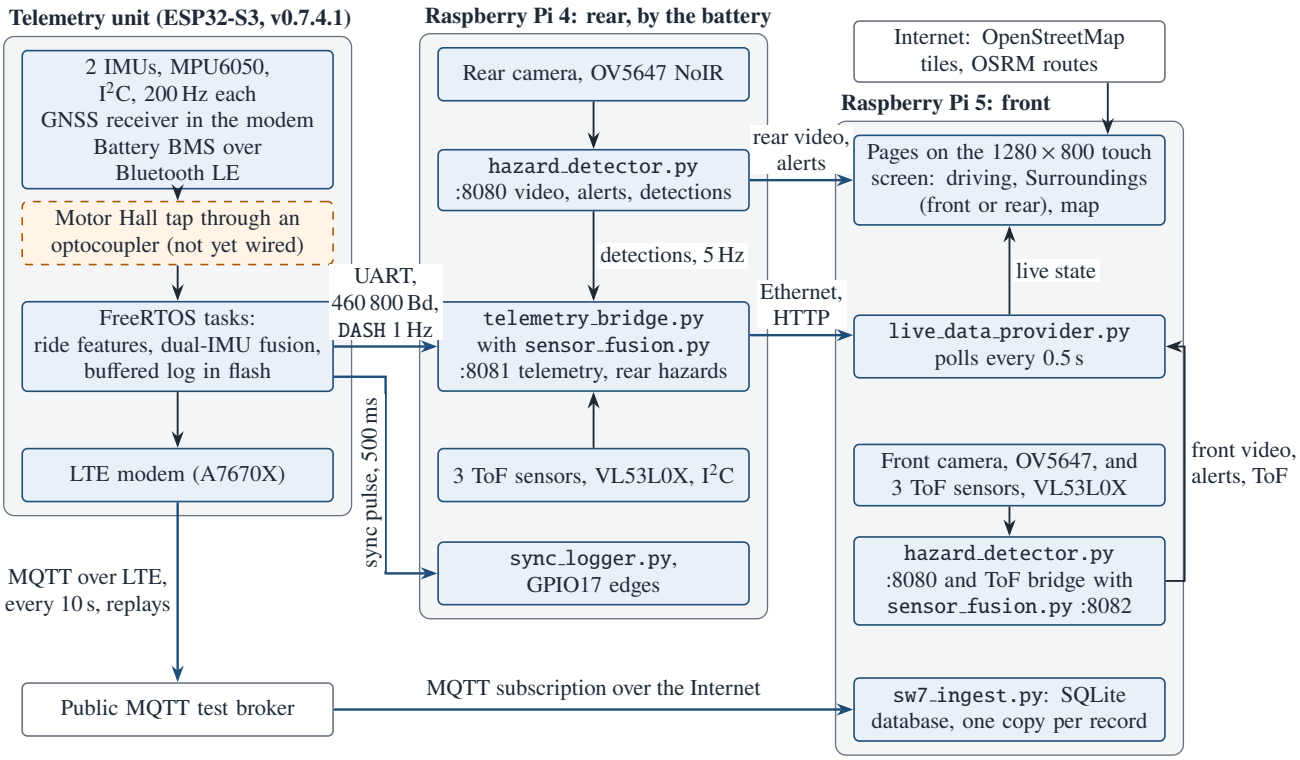


Figure 4.1: The system as built on 21 October 2026: the Raspberry Pi 5 beside the display runs the dashboard and the front camera unit, the Raspberry Pi 4 beside the battery runs the rear camera unit and reads the telemetry unit, and the two are joined by an Ethernet cable (Section 4.8.6). Programs are named by their files in the repository. Dashed outline: built but not yet connected on the vehicle.

requirements were R1 (a fixed 200 Hz sampling period with no loss) and R7 (camera detection at a useful frame rate). A microcontroller under FreeRTOS can hold a fixed sampling period but cannot run a camera network at a useful rate; a Raspberry Pi can run the network, but a general-purpose Linux scheduler gives no guarantee of a 5 ms period, so it was not relied on for acquisition. Splitting the work keeps each part where it is predictable. The camera went on a Raspberry Pi 4 rather than the Raspberry Pi 5 because the Pi 5 runs the inherited dashboard, and developing on it would have put a working deployment at risk. Power was not measured for any option; the one power finding is that the telemetry unit's modem must not be powered from a Raspberry Pi's USB port (Section 3.4.2).

Table 4.1: Feasible architectures compared. Figures are measured in this project unless marked; power was not measured.

Option	**Workload**	**Power**	**Memory**	**Interfaces**	**Latency, timing**
ESP32-S3 alone	Dual 200 Hz sampling, features, BLE, modem: yes. Camera network at a useful rate: no	Lowest of the options (not measured)	16 MB flash, 8 MB PSRAM (not used)	Two I^2C, UARTs, BLE, native USB; LTE and GNSS on the board	5 ms period held, 200.0 to 200.5 Hz
Raspberry Pi alone	Detector at 19.5 frames/s (Pi 4); fixed-period sampling not guaranteed	Higher; under-voltage on a weak supply	Not a constraint in testing	I^2C, CSI camera, Wi-Fi, USB; no LTE modem	Detector 102 ms median; sampling jitter not bounded by design
ESP32-S3 and Pi 4 (chosen)	Acquisition on the ESP32, vision and fusion on the Pi	Two supplies; modem not from the Pi's USB	As above	Pulse wire for timing, USB serial for data	5 ms acquisition; 102 ms detection alone, 118 to 122 ms with the whole system; 1 Hz dashboard line
Pi 5 with Hailo-8 accelerator (not tested)	Detector at camera rate expected; no Pi 4 link needed	Not measured	16 GB	Accelerator not yet enabled on the PCIe bus	Not measured

4.2 Hardware

This section serves R1, R4, R5, R6, R8 and R13, and RQ1 and RQ3. The telemetry unit is a Makerfabs ESP32-S3 4G LTE CAT1 A7670X board: an ESP32-S3-WROOM-1-N16R8 module (16 MB flash, 8 MB PSRAM) with an A7670X LTE Cat-1 modem that also contains the GNSS receiver [46] (Figure B.2, Appendix B). The integrated board lets processor, modem and GNSS be tested without a separate modem circuit. Two MPU6050 IMUs are on separate I^2C buses at 400 kHz, which avoids their shared address and bus contention. The modem is on a hardware UART at 115200 bit/s, with its power key and reset on GPIO 4 and 5. The board's microSD slot was not used: its chip-select line passes through a resistor (R23) that the schematic marks as not fitted, so local logging uses the internal flash (Section 4.9).

Each camera unit carries an OV5647 camera module without an infrared-cut filter and three VL53L0X time-of-flight sensors on its I^2C bus: the Raspberry Pi 5 beside the display faces forward, and the Raspberry Pi 4 beside the battery faces backward and also takes the sync-pulse input on GPIO17

(Section 4.8.6). The Raspberry Pi 4's camera is on a 15 cm ribbon. Each camera is housed in a three-part casing designed and 3D printed in this project (Figure B.5, Appendix B), not yet fixed to the vehicle. The enclosures and mounts designed to put the system on the vehicle (the dashboard display mount, the two sensor pods and the IMU pods), the choices behind them and the checks on their geometry are in Appendix F; none had been printed by 12 October. Every connection is listed in Appendix C, and Figure 4.2 shows the bench setup.

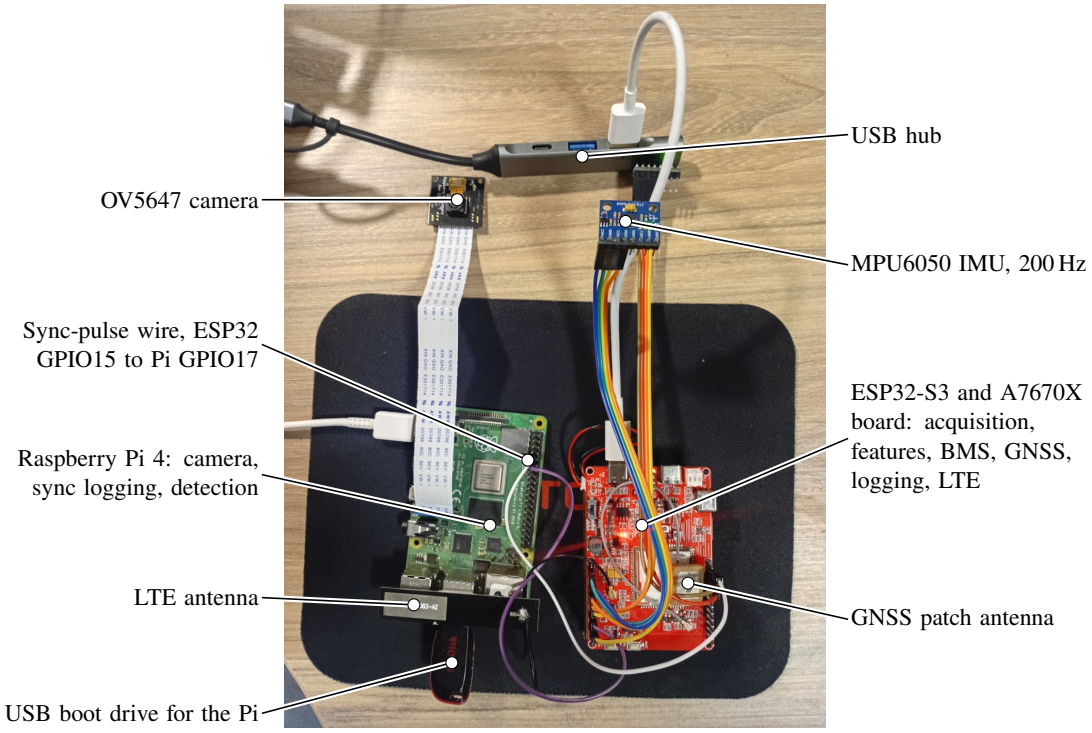


Figure 4.2: Bench setup of the telemetry system before the two-sided layout, each component labelled by its role. At this stage the Raspberry Pi 4 carried the only camera; in the final layout it is the rear unit and the Raspberry Pi 5 carries the front camera (Figure 4.5).

4.3 Firmware architecture

This section serves R1 and R6, and RQ1 and RQ3. The firmware is written with PlatformIO and the Arduino framework on FreeRTOS (Table E.1). Figure 4.3 shows its tasks and queues. The two IMU tasks run at the highest priority on core 1, each holding a 5 ms period with `vTaskDelayUntil`, which schedules each wake-up from the previous target time so that variable execution time does not accumulate into drift. After a stall longer than a period it returns at once for each missed period, so the missed samples are read in a burst about 1 ms apart: none is lost, but they are unevenly spaced. The modem, BMS and dashboard-line tasks run on core 0, away from acquisition.

The 200 Hz rate was chosen from the 80 Hz upper limit of the ISO 2631-1 whole-body band [18]: sampling that band needs more than 160 Hz, its Nyquist rate, and 200 Hz adds margin without loading the bus heavily. The standard is used only for the band; its frequency weighting is not applied (Section 4.4). Each sample carries a sensor identifier, a per-sensor sequence number and a timestamp from `esp_timer_get_time()`, a monotonic microsecond clock unaffected by network time. No acquisition task prints, because serial output blocks for a variable time and would disturb the period being measured. When a queue is full the producer drops the new sample and counts it rather than blocking, so a slow

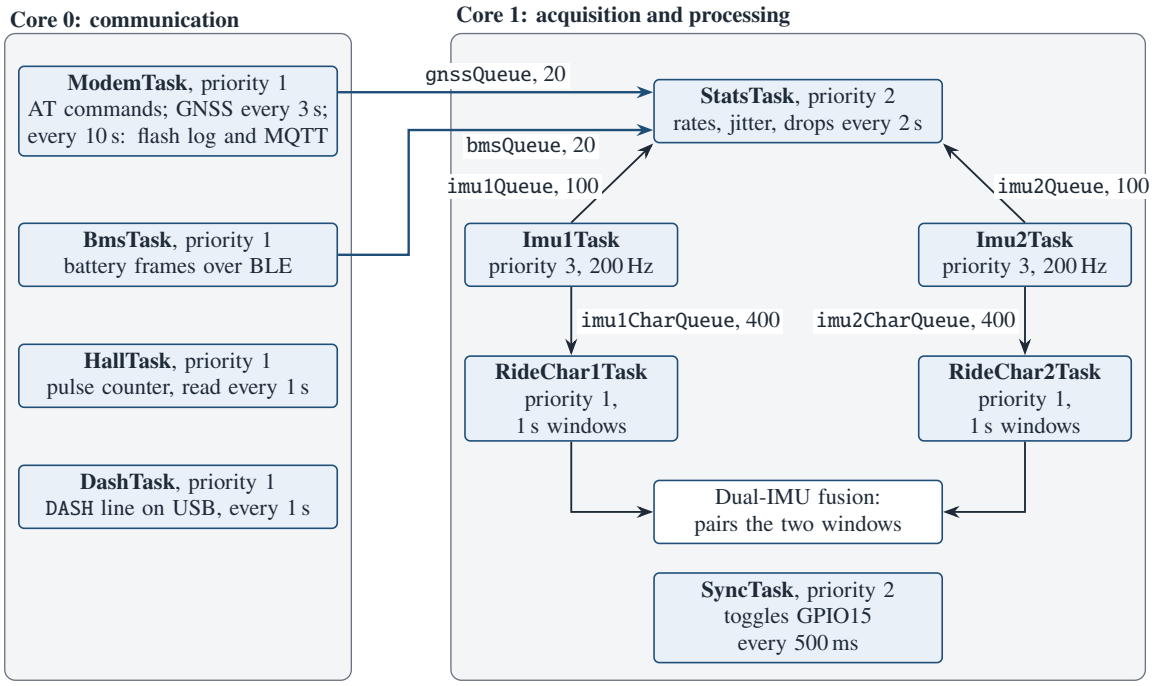


Figure 4.3: FreeRTOS tasks and queues (drawn for firmware v0.7.1; unchanged in v0.7.3 and v0.7.4). Priorities as set in the firmware (higher runs first); each queue is labelled with its name and capacity in items. Queues between the two cores are drawn in blue.

consumer can never hold up sampling. Each IMU feeds two queues, one to the statistics task and one to its ride-characterisation task, so that adding processing cannot change the acquisition accounting.

A lower-priority statistics task drains the queues and reports, every 2 s and per sensor, the achieved rate, the minimum, mean and maximum interval between samples and the drop count, with the GNSS poll and fix counts. These counters produce the acquisition results of Chapter 5 without a separate measurement pass, on the bench or on the vehicle.

Each IMU is set to $\pm 8\,g$ and $\pm 500°/s$. Up to v0.7.3 the digital low-pass filter was at 260 Hz, which lets vibration between the 100 Hz Nyquist frequency and 260 Hz alias into the band; v0.7.4, flashed on 11 October, uses 94 Hz, the widest setting below the Nyquist frequency. Bench results before 11 October used 260 Hz; test T15 and the road test use 94 Hz. The $\pm 8\,g$ range gives headroom for pothole and kerb impacts while keeping resolution; it is provisional, to be set from the peaks the vehicle produces, and samples at the limit are not flagged, so a clipped window under-reports its peaks.

4.4 Ride-characterisation signal processing

This section serves R2 and R3, and RQ1 and RQ4. A task per IMU computes time-domain features from each one-second window. Features use the acceleration magnitude rather than one axis because the mounting has not been related to a vehicle frame; the magnitude does not depend on orientation, at the cost of not saying which axis a disturbance came from. Each window holds $N = 200$ samples ($f_s = 200$ Hz, $\Delta t = 1/f_s$). The calibrated magnitude of sample k is

$$a_k = s\sqrt{a_{x,k}^2 + a_{y,k}^2 + a_{z,k}^2}, \tag{4.1}$$

where s is the sensor's calibration factor. From the window,

$$\mu = \frac{1}{N}\sum_{k=1}^{N} a_k, \qquad a_{\mathrm{RMS}} = \sqrt{\frac{1}{N}\sum_{k=1}^{N} a_k^2}, \qquad \sigma = \sqrt{\frac{1}{N}\sum_{k=1}^{N} a_k^2 - \mu^2}, \tag{4.2}$$

$$a_{\mathrm{pp}} = \max_k a_k - \min_k a_k, \qquad \mathrm{CF} = \frac{\max_k a_k}{a_{\mathrm{RMS}}}, \qquad \bar{J} = \frac{1}{N-1}\sum_{k=2}^{N} \frac{|a_k - a_{k-1}|}{\Delta t}, \tag{4.3}$$

and the threshold count is the number of samples with $a_k > \mu + 2\sigma$. Because a_k includes gravity, a_{RMS} is close to $9.8\,\mathrm{m/s^2}$ at rest and the crest factor close to 1; σ removes the constant part and is the vibration measure used. Lost samples are counted, not interpolated, and Δt is the nominal period, not the measured interval, so the jerk of a window that contains an acquisition pause is distorted. The ISO 2631-1 frequency weighting is not implemented, so the features describe vibration exposure, not comfort as the standard defines it. At every boot the feature code is checked against a synthetic 5 Hz sine of 200 samples, whose RMS, peak-to-peak and crest factor have closed-form values ($A/\sqrt{2}$, $2A$, $\sqrt{2}$), with a tolerance of 5%.

4.4.1 Per-sensor calibration

Two nominally identical sensors at rest on the same platform read 10.35 to 10.39 and 9.55 to $9.56\,\mathrm{m/s^2}$, about 5.7% above and 2.6% below standard gravity, a disagreement of about 8%. Since only the magnitude is used, one scale factor per sensor corrects it: RMS, σ, peak-to-peak and jerk scale linearly with it, and crest factor and the threshold count are unaffected. The first time a sensor is found with no stored factor, the firmware averages 200 stationary readings and stores

$$s = \frac{g_0}{\frac{1}{200}\sum_{k=1}^{200}\sqrt{a_{x,k}^2 + a_{y,k}^2 + a_{z,k}^2}}, \qquad g_0 = 9.80665\,\mathrm{m/s^2}, \tag{4.4}$$

in non-volatile storage, rejecting values outside 0.5 to 2.0 as evidence that the sensor was moving. The factor is reused at every boot rather than recomputed, since the vehicle may be powered while moving, and a console command erases it so the sensors can be recalibrated in their mounted position (not yet used). A single factor corrects the reading only in the orientation in which it was measured, because per-axis offsets remain; this is why the factors changed between boards (Chapter 5).

4.4.2 Speed-normalised index

The same road driven faster gives a larger response. Following Li et al., who classify roughness from acceleration mean square divided by speed [17], each window also computes

$$I_v = \frac{\sigma^2}{v}, \qquad v \geq v_{\min}, \tag{4.5}$$

where v is the latest GNSS speed and $v_{\min} = 1$ in the receiver's units, so that I_v is in $(\mathrm{m/s^2})^2$ per unit of that speed; without a fix or below $v_{\min}$, I_v is not computed. Firmware before v0.7.2 used a_{RMS}^2, which is dominated by gravity; v0.7.2 uses the variance. Two limits apply: the manual gives the speed in knots, but the unit's readings have not yet been checked against a known speed (Section 4.6), so I_v supports relative comparison within one dataset only; and the sensors are on the body, not the unsprung mass, so

I_v is a speed-normalised response measure, not a validated roughness class.

4.4.3 Dual-IMU fusion

The two sensors are the same part, configuration and calibration, so they are combined with equal weights: for RMS, σ, peak-to-peak, jerk and I_v, $x_f = (x_1 + x_2)/2$. Crest factor and the threshold count stay per sensor. Because the sensors are at different points on the vehicle, averaging would hide real differences, so each fused window also reports

$$D = \frac{|\sigma_1 - \sigma_2|}{\max(\sigma_f,\ \sigma_{\text{floor}})}, \qquad \sigma_{\text{floor}} = 0.05\,\text{m/s}^2, \tag{4.6}$$

which approaches 2 when one sensor sees much more than the other. A pair is fused only if the two windows ended within half a window of each other; if one sensor produces nothing for 3 s, the output falls back to the other and says so. A boot self-test checks the averaging, D, the rule that I_v needs both sensors, rejection of a misaligned pair and the fallback. The equal-noise assumption has not been tested on the vehicle.

4.5 Battery-management interface

This section serves R5 and RQ3. The pack reports voltage, current, state of charge, capacity, temperatures and protection faults over Bluetooth Low Energy in the JBD protocol. An inherited implementation leaked a client object on every reconnection, reconnected on a fixed timer and framed messages by a terminator byte that can occur inside the data. The interface was rewritten with one reused client, reconnection only on a disconnect callback, framing from the declared length field, a checksum, and a plausibility check against a 72 V pack. It searches every discovered service for the required characteristics instead of assuming the service, and logs every advertiser seen so that a wrong address is visible. The field offsets were confirmed byte for byte against an independently written parser in a teammate's code.

Current status (11 October 2026). Valid frames have been received on the vehicle (74.78 V, 0.00 A, 60%), but the link is intermittent (Section 4.10.3). The reading has not been compared with a meter or the pack's app. The pack advertises as `DB24SA01L24S150ABU`, a 24-cell, 150 A designation, and inherited documentation describes the traction pack as lead-acid without a BMS, which suggested a separate auxiliary pack. On the road session of 19 October, however, this pack supplied up to 46.9 A while the vehicle was driven and its voltage fell from 74.6 to 72.8 V, so it is the traction pack and the documentation is out of date.

4.6 Positioning

This section serves R4, and RQ1 and RQ3. The modem's GNSS is polled every 3 s with `AT+CGNSSINFO`. The parser was written defensively, checking the field count and logging the raw reply, because the manual available [47] was not known to match the fitted modem's firmware. The real reply did differ from the one in the manual (Section 4.10.2). Since v0.5.3 the parser anchors on the latitude hemisphere letter and reads every field relative to it, which fits all observed layouts, and accepts a fix only if the layout

and the coordinate, date and time formats are valid and the implied jump from the previous fix is under 100 m/s; this rejected replies corrupted when a second query competed with the firmware's polling. A console command prints every raw reply so that the parser can be checked without extra commands. Section 4.10.2 covers the modem's start-up and GNSS warm-up. The manual gives speed over ground in knots [47]; the unit's readings have still to be checked against a known speed. The reply layouts are in Appendix E.

Firmware v0.7.3 adds assisted GNSS, which downloads orbit data over LTE every two hours (`AT+CAGPS` returned success on the bench), and, while there is no fix, the modem's network position (`AT+CLBS`) once a minute. Indoors this answered in about 0.7 s with a reported accuracy of 550 m, about 30 m from the first outdoor fix at the same site; it is kept in separate fields, drawn on the map as an approximate circle, and never used for speed, heading or the ride index.

4.7 Other vehicle inputs

This section serves RQ1 and RQ4; no requirement of Table 1.1 depends on it yet. The Sensirion SEN55 reports particulate matter, humidity, temperature and VOC and NOx indices over I^2C; the gas indices are relative to recent background and take up to 1 and 6 h respectively to meet specification after switch-on [48], so gas indices from a short drive are indicative only. It was chosen so that conditions along a route can be recorded on the same timeline as the vehicle's measured response. It has been received but is not connected, for want of a six-way JST GH cable; the planned wiring is in Appendix C.

Motor speed would help separate powertrain vibration (RQ4). The Hall signal is referenced to the drivetrain, so it is to be tapped through a PC817 optocoupler module. Firmware v0.6.0 counts both edges in the hardware pulse counter with a 12.5 μs glitch filter, so counting costs the IMU tasks nothing, and computes revolutions per minute as the edge rate divided by twice the pole-pair count, which is not yet known. Until the tap is wired the counter reads the sync pulse as a loopback, against which the count can be checked exactly: on 1 October it counted 666 edges in 333.3 s, 2.00 edges per second, matching the 500 ms toggle. Which Hall wire to tap, its voltage (5 or 12 V) and the optocoupler module's input rating are still to be measured on the vehicle. The same firmware publishes the modem's signal quality every 10 s so that failed publishes can be related to coverage.

4.8 Camera subsystem

This section serves R6, R7, R8, R10 and R11, and RQ1, RQ2 and RQ3. It covers capture and synchronisation, detection, range sensing and fusion, the dashboard integration, the time-to-contact warning and the two-sided layout.

4.8.1 Capture and synchronisation

The camera's first purpose is a visual record against which inertial events can be labelled, for example to tell a pothole from a deliberate manoeuvre. The camera and the inertial data are on different boards, so their clocks are related by a shared signal: a FreeRTOS task on the ESP32 toggles GPIO15 every 500 ms

with `vTaskDelayUntil` and logs each edge's microsecond time, and the Raspberry Pi 4 logs the arrival of each edge on GPIO17. On the ESP32 side successive logged edges were 500,000 to 500,001 μs apart over an extended capture. The first Pi-side logger timestamped edges in a Python callback (`gpiozero`); its intervals clustered at 500.1 to 500.3 ms with one unexplained 413.6 ms gap. The edge logger was therefore rewritten (`CameraDetection/sync_logger.py`) to take each edge's time from the kernel through the GPIO character device, on the same monotonic clock as the camera's frame timestamps, which narrowed the spread to under 0.7 ms (Chapter 5); the camera library's `SensorTimestamp` gives the time at which the first pixel of each frame was read out, on that clock [49]. Frames and edges therefore share one time base. Two findings shaped this part. The pin first chosen for the pulse is not brought out on the board, which was caught against the manufacturer's schematic before any wiring. And twice the jumper on the pulse wire was disturbed by handling, once giving no edges and once a 16 s burst of spurious edges: edges that do not fit the known 500 ms pattern can be discarded in analysis, but a secured connector is needed on the vehicle. An earlier lens-cover test bounded the offset to under about 1 s (Appendix E). Sub-second alignment is enough for labelling events after the fact.

4.8.2 Detection

The detector (`CameraDetection/hazard_detector.py`) runs a pretrained SSD-MobileNet-v1 [34, 35], trained on COCO [37] and quantised to 8-bit integers [36] for LiteRT [50], on 640×480 frames, which needs no training data and no accelerator (Section 2.8). Only motor vehicles, motorcycles, bicycles, pedestrians and animals are reported, in three distance bands (immediate, warning, monitoring) estimated from the box width, an assumed width per class (for example 1.8 m for a car) and the pinhole relation. The focal length is not yet calibrated, so the bands are indicative.

An alert is raised when a class appears in a band for three consecutive frames, re-raised if it holds a nearer band for three frames, and cleared only after five frames without it; within a frame the nearest detection of a class is used. The clearing rule came from the second live run, in which an alert that cleared on a single missed frame produced 16 activations in 47 s for one person. The rules were checked offline by replaying scripted sequences through the unchanged loop, and later seen raising and clearing person alerts on live frames.

Data handling is minimal by design, and was fixed in code before the detector first processed live frames on 28 September 2026. Each alert records five fields: class, band, confidence, timestamp and alert status. Frames exist only in memory and are never written to disk; there is no face or number-plate recognition and no identification of anyone; inference is local; and the output is advisory, with no control of any vehicle function. The ride characterisation of Section 4.4 uses no learned model.

Every frame's latency is measured from its `SensorTimestamp`: the time from readout to capture by the detector, preprocessing, inference, alert logic and the total, reported as median, 95th percentile and maximum every 5 s and optionally written per frame. The measurement excludes the exposure time itself. Inference uses all four cores. The optional live view draws the detections on the image and serves it as MJPEG from its own thread, which always takes the newest frame, so a slow viewer cannot slow detection. Two input faults were found on the first live run: a placeholder entry in the label file shifted every class by one, and the camera's buffers are in blue, green, red order while the model expects red, green, blue [49]. The module's lack of an infrared-cut filter gave a pink cast, corrected by its own tuning file and a saturation of 1.8 (Section 4.10.5).

Because the COCO classes include no road-surface hazards, a YOLO11n pothole model [51] was fine-

tuned in Colab, first on a public dataset [52] and then on tiles cut from the road band of a South African dashcam dataset collected by Nienaber et al. at Stellenbosch University [53]; the data preparation is in Appendix E. On the Raspberry Pi 4 it runs with the NCNN library [54] as an experimental path, off by default, because only at one run every 4 s did the detector stay within 10% of its rate without it (13.9 against 14.6 frames/s; test C10). *[Outstanding: held-out test score of the pothole model, and a run on a road.]*

4.8.3 Range sensing and fusion

A VL53L5CX was chosen for proximity sensing because its 8×8 zones in a 63° field separate left, centre and right from one device, where single-point sensors would have to be duplicated across directions. It involves no image and no learned model, so it works whatever the detector's state. It had not arrived by 30 September, so the three VL53L0X sensors of a UCT micromouse sensor board stand in, facing left, ahead and right, with a range of about 2 m (1.2 m used). They share one address, so the reader holds all three in shutdown and wakes each in turn to give it its own address (Appendix C). Each camera unit has its own board of three beside its camera on a free bus, on the Raspberry Pi 4 at the rear and, since 15 October, on the Raspberry Pi 5 at the front (Section 4.8.6), which leaves the ESP32's acquisition undisturbed.

A sensor that does not answer within 0.5 s is skipped, logged and looked for again every 60 s (Section 4.10.6).

The camera says what an object is and in which direction, but its distance is uncalibrated; each time-of-flight sensor measures distance well, but only close by, in one direction and without a class. A fusion step on each camera unit (`CameraDetection/sensor_fusion.py`) combines them. The detector serves each detection with a bearing computed from the box centre and the 53.5° horizontal field, and the bridge polls it five times a second. A detection is matched to a sensor when its bearing lies in that sensor's cone widened by a 5° gate and the sensor's nearest sample is within 200 ms, valid and in range; the nearest reading wins and each sensor serves one detection. A matched detection becomes a fused hazard with the camera's class and the sensor's distance; an unmatched one keeps the camera estimate; a sensor with three valid in-range readings and no match reports an unclassified obstacle; and camera data older than 1 s are ignored, so with the detector stopped only obstacles are reported. Hazards are sorted nearest first. The sensor directions were first set to −35, 0 and +35° (12.5° half-angle) as defaults, not measurements; since 15 October the side sensors are set to ±90°, the way they point on both boards (Section 4.8.6). With the first defaults the side cones (−47.5 to −22.5° and 22.5 to 47.5°) overlap the camera's ±26.75° view only at its outer edges (Figure 4.4), so most objects to the side can be reported only by a time-of-flight sensor, without a class. A sensor reports the nearest surface in its cone, which need not be the object the camera saw, and there is no tracking. A continuous record of distances and bearings would collect more than the system needs, so the fusion log is off by default and is switched on only for a validation session, and then holds numbers only, no images. The bench validation (test C9) first calibrates the camera axis, field of view and each sensor's cone with a narrow target on a marked floor, then checks a person crossing at 1 m, a person standing at 0.5, 1.0 and 1.5 m against a tape, a box outside the detector's classes at 0.8 m (to be reported as a time-of-flight obstacle) and a person at 2.5 m, beyond the sensors' range (camera only).

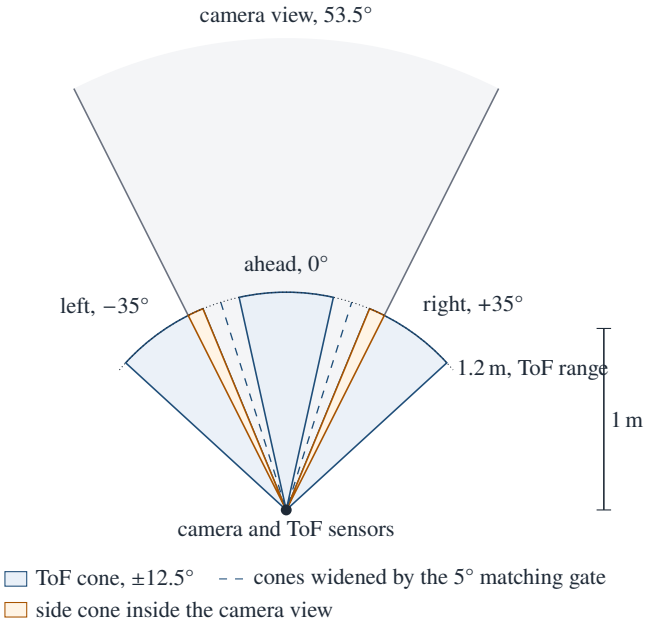


Figure 4.4: Camera and time-of-flight geometry used by the fusion on each camera unit, top view to scale, the unit's viewing direction up. The values are the first configuration defaults (±35° side sensors), not calibrated; the side sensors were later set to ±90° (Section 4.8.6). A detection is matched to a sensor when its bearing lies within that sensor's cone widened by the 5° gate.

4.8.4 Dashboard integration

An inspection of the inherited PySide6 dashboard on 2 October found no live input: every page ran on simulated data, with one unused entry point for a live vehicle state and a fallback to simulation after 2.5 s without one. A driver's display should not depend on coverage or a remote broker, so a local path was built. Firmware v0.7.0 prints one compact JSON line a second on USB; a bridge on the Raspberry Pi 4 (`telemetry_bridge.py`) reads it with the time-of-flight distances and the fused hazards and serves them over HTTP; and a live-data provider on the Raspberry Pi 5 polls the bridge every 0.5 s. The dashboard therefore needed one network link for camera, alerts and telemetry. Until 15 October that link was a laptop or phone hotspot (Appendix C), so the display depended on it. Since then the front camera and its alerts run on the Raspberry Pi 5 itself, and the rear unit and the telemetry are reached over a dedicated Ethernet cable (Section 4.8.6), so the display no longer depends on Wi-Fi. It finds the Raspberry Pi 4 on any network from a beacon that the bridge broadcasts every 2 s, with localhost, an override, a wired address, mDNS and the last good address as alternatives (Figure E.2, Appendix E). The design addresses the dashboard issues of Section 2.6 as follows; the later redesign of the pages themselves is in Appendix G.

Data validity and update age. A live-only mode (`TUKZIE_LIVE_ONLY=1`) stops the simulation entirely. When the bridge cannot be reached, or the telemetry unit's record is more than 3 s old, the provider stops sending, and within 2.5 s the dashboard shows an explicit no-data state: every sensor field empty, every validity flag false, "--" or "Unavailable" on the pages, and a message that live data was lost. A field with no live source (speed, until the GNSS units are confirmed; pedals; temperatures) stays empty rather than taking a default, and the state of charge stays empty until the BMS reports. This mode was added after two findings: when the Raspberry Pi 4 changed network, the dashboard fell back to simulated values with no sign, and a battery level of 82% was shown that was the simulator's last value, not a measurement.

Layout. The changes add few elements: a camera page with the live view and the active alerts; a Sensors page with the three distances as gauges, the fused hazards nearest first, and the battery, position, motor and ride readings; a ride-quality card on the diagnostics page; and the driving page's front-obstacle distance, taken from the forward sensor only (since 15 October the ahead sensor of the Raspberry Pi 5's board) so that a car parked alongside does not raise a front warning.

Contrast and colour. Alerts and gauges use the dashboard's own status colours (amber under 1.0 m, red under 0.5 m), so the team's visual language is kept. Contrast in daylight has not been measured.

Alert priority. An immediate hazard from the fusion, or loss of the Raspberry Pi 4 link for 5 s after it was working, raises a banner over every page; touching it opens the Sensors page, and a button hides it for 30 s. Lower bands appear only on the camera and Sensors pages.

Interaction. Finger-drag scrolling was added and developer options were moved behind one switch. Touch response was reported as slow, so the touch controller was read directly during 25 taps (Chapter 5).

The integration is applied as a patch set of 63 edits to a separate copy of the team's dashboard by a deploy script that refuses to touch the team's folder and writes nothing unless every edit's anchor text occurs exactly once, so the team's code is never changed. Since 6 October a system service starts the integrated copy at boot on the Raspberry Pi 5 in place of the team's service, which one command restores. The camera page uses only Qt's networking and widget modules, because the dashboard's web-engine map crashed the program on the Raspberry Pi 5; a street-tile map drawn directly with Qt from OpenStreetMap tiles [55], with routes from OSRM [56], replaces it. The map and navigation work is summarised in Appendix E.

4.8.5 Approaching objects: time to contact

A fixed distance band says how near an object is, not whether it is coming closer. On 15 October the detector gained a warning for objects closing on the vehicle, front or rear (`CameraDetection/hazard_detector.py`, tests in `tests/test_approach.py`).

- *Tracking.* A small tracker matches detections from frame to frame by box overlap, so each object keeps an identifier through up to two missed frames. Car, truck and bus may continue each other's tracks, because the model swaps them between frames. The identifiers start from 1 at each start-up and identify no one.
- *Time to contact from scale.* For an object closing at constant speed, its image width w grows so that $1/w$ falls linearly in time, reaching zero at contact. The time to contact is $\tau = w/\dot{w}$, the expansion measure of Section 2.8 [58, 59], and needs no focal length or real object size. It is fitted robustly (a Theil-Sen line [60] through $1/w$ against the frames' sensor timestamps) over the last 0.6 s. It is used only with at least 5 frames over 0.25 s, 75 % of the pairwise slopes agreeing, at least 5 % growth, no box touching the image's side edges, and the height giving a time within a factor of 2.
- *Alert.* An object is "approaching" when τ is under 4 s (warning) or under 2 s (immediate), after the same three-frame onset as the other alerts, and is released above 5 s (2.5 s for immediate) or after five frames without a valid estimate. A closing speed from the distance estimate is also reported, but it depends on the uncalibrated focal length.

- *Close range.* The bridge computes each time-of-flight sensor's closing speed and time to contact over 0.3 s of readings. Where camera and time-of-flight agree on bearing, the fused hazard takes its time to contact from the sensor inside the sensor's range and from the camera beyond it, and says which.

The vehicle's own motion makes a stationary object ahead "approach", which is intended: it is then a forward-collision warning. Each warning is logged with its time to contact, track and side. The added work costs about 0.3 ms per frame on a laptop. The accuracy test (C11: a person walking towards each camera at a measured pace between tape marks, and a car approaching the parked trike in a closed area) has not been run.

4.8.6 Two-sided layout

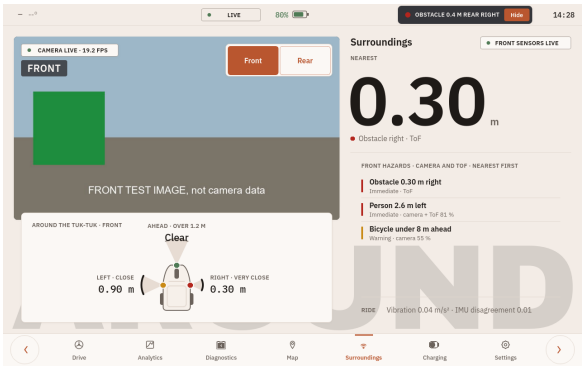
On 15 October the hardware was rearranged after two findings: the battery-management unit's Bluetooth link did not reach the telemetry unit at the front of the vehicle, and the Raspberry Pi 5 ran the detector more than twice as fast as the Raspberry Pi 4 (Results, Table 5.2). Figure 4.5 shows the result.

- *Front, Raspberry Pi 5* (behind the dashboard): its own OV5647 camera, the detector, a second time-of-flight board and its fusion, beside the dashboard. A front alert no longer crosses the network before it reaches the screen.
- *Rear, Raspberry Pi 4* (beside the battery): its camera and time-of-flight board now look backwards, and it keeps the telemetry unit on USB. Near the pack the battery-management link connected for the first time (Results, R5).
- *Link.* A dedicated Ethernet cable joins the two (fixed addresses 10.20.0.1 and 10.20.0.2, no gateway), so Wi-Fi keeps the route to the internet. It ran at 1000 Mb/s with a 0.22 ms round trip, and the dashboard found the rear unit over it unaided.
- *Dashboard.* The Surroundings page gained a Front — Rear switch. A side moves the live picture, the alert list, the three distances and the fused hazards together, each from that side's own unit. The rear picture is mirrored, like a car's reversing camera, with left and right swapped to match. The banner shows warnings from both sides, labelled "Front" or "Rear", and opens the side it names; selecting reverse on the dashboard's own gear control selects the rear (the vehicle provides no gear signal). Appendix G shows the pages.

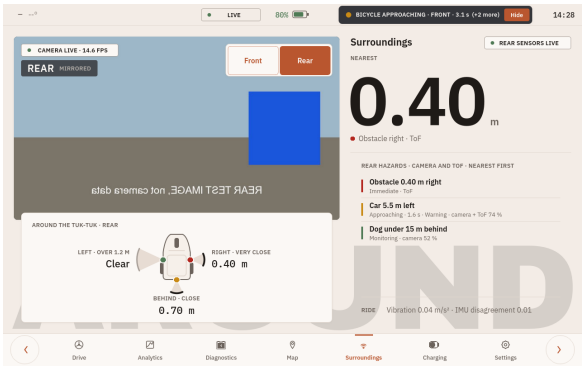
The side time-of-flight sensors on both boards point straight out sideways, so the fusion's sensor bearings were set to ±90°; at the earlier ±35°, a camera detection near the image edge could be paired with a side sensor pointing elsewhere.

4.9 Communications and local logging

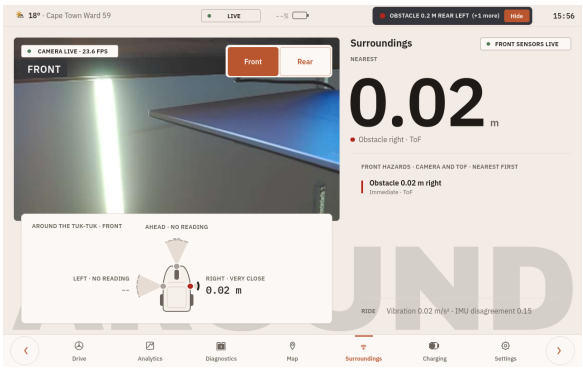
This section serves R9 and R10, and RQ3. Every 10 s the modem task publishes a compact JSON summary of the ride, BMS and GNSS state over MQTT using the modem's own `AT+CMQTT` commands.



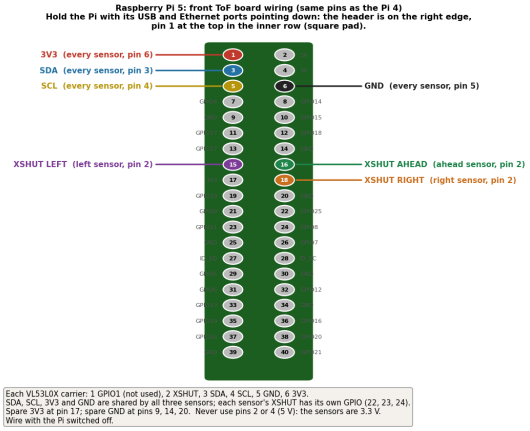
(a) Surroundings, Front: the Raspberry Pi 5's camera and time-of-flight board (test image).



(b) Surroundings, Rear: mirrored, with a car approaching at 1.6 s (test data).



(c) On the Raspberry Pi 5 on 15 October: live front camera at 23.6 frames/s, a rear warning in the banner.



(d) Front time-of-flight board on the Raspberry Pi 5: the same pins as on the Raspberry Pi 4.

Figure 4.5: Two-sided layout: front sensing on the Raspberry Pi 5, rear sensing and the telemetry unit on the Raspberry Pi 4.

Publishing is kept inside the modem task because the modem UART has no lock, and a second task issuing commands would corrupt both streams. The broker is a public, unauthenticated test broker (`test.mosquitto.org`), used only to prove the path and not as a destination for real vehicle data; the credentials of the vacation-work team's own broker were deliberately not copied into this project's code. For the start, connect and publish commands the modem returns `OK` and then a separate result line whose code gives success or failure [47]; since v0.5.0 the firmware waits for that line rather than trusting the `OK`. A session is treated as lost after three failed publishes or when the modem reports a dropped client or lost network; the old session is then shut down in the order the manual requires, and retries start at 30 s and double to at most 5 minutes. Since v0.5.3 the firmware also reconnects as soon as the modem reports that packet data is available again.

Each record is also written to a LittleFS file system in the 3.5 MB data partition of the internal flash (Table E.1), whether or not MQTT is connected, so that a session leaves a complete record without coverage. Two findings shaped this part.

Finding: flash writes pause acquisition. With a write every 10 s, both IMUs showed a 33 to 40 ms gap at each write, although they are on separate buses; switching logging off for 120 s, with publishing continuing, removed the gaps. Writing to flash suspends code executing from flash on both cores, which explains why both stalled together. Since v0.6.0 records are buffered in memory (8 KB) and written in one append about once a minute. This reduces how often the pause occurs, not its length, and up to a minute of records is lost if power is cut before a flush.

Finding: missing values made invalid JSON, which only reading the whole log back revealed; since v0.6.3 they are written as `null` (Section 4.10.4).

The log is capped at 3 MB and then truncated rather than rotated. The file survives reprogramming and grew by about 160 KB an hour on the bench, so it must be read back and cleared before a long session such as the road test; after the read-back of 1 October the copy was saved in the repository and the board's file cleared.

Store-and-forward to a central database (v0.7.4). Every record carries a boot counter, a sequence number that never repeats across reboots and, once the GNSS has a fix, the device's UTC time, and is published at QoS 1. A ledger in the modem task tracks the highest sequence number up to which every record has been acknowledged or counted as lost; after a reconnection the task republishes the unacknowledged records from the log, marked as replays, at up to about three a second. On the Raspberry Pi 5 an ingest program (`Server/sw7_ingest.py`) stores each record in SQLite once per device, boot and sequence number, and can backfill from a copy of the log; `Server/sync_report.py` reports missing and duplicate records, replays, drain time and, with device UTC, latency. The ledger passed 900 randomised outage and reboot simulations on the laptop before test T15.

4.10 Fault investigations

This section serves R1, R4, R5, R8, R9 and R10, whose tests exposed the faults below, and RQ3. Most faults in Table E.2 were fixed in one step; the seven below needed a diagnosis, because more than one cause could explain what was seen. Each is told as symptom, candidate causes, deciding test, cause and fix, and consequence, and Table 4.2 summarises them.

4.10.1 Boot loop after enabling the full flash

Symptom. The build tool treated the board as an 8 MB part without PSRAM, although the module has 16 MB of flash and 8 MB of PSRAM. Setting the flash size to 16 MB with a matching partition table made the processor reset about once a second, always at the same program counter.

Candidate causes and tests. The PSRAM setting, the flash override, the code or the hardware. One setting was changed at a time. Adding a PSRAM type (`qio_opi`) left the same loop at the same program counter, so PSRAM was not the cause on its own. Removing every override restored normal running, which cleared the code and the hardware. Two independent readings then pointed to the flash access mode: the upload tool's chip detection confirmed 8 MB of PSRAM, and the ROM boot banner reported the flash in dual-I/O (DIO) mode, while the setting tried asked for quad-I/O.

Fix and consequence. The firmware used about 9% of the default partition, so the stock configuration was kept until local logging needed the 16 MB table. Then 16 MB with DIO mode, set together with the table, booted and mounted the logging partition, confirming the diagnosis. Size, mode and table must be set together.

4.10.2 Modem power key, GNSS warm-up and reply layout

Symptoms. Three problems overlapped. Indoors every GNSS poll had the wrong number of fields; for 60 to 90 s after the GNSS was switched on, polls got no reply; on a rooftop the first fix was reported near latitude and longitude zero (0.25 and 0.10 degrees); and a later five-minute capture got no reply to any of its 69 polls.

Candidate causes and tests. A reply layout other than the 13 fields assumed, a normal receiver start-up delay, or a modem that was not running. Because the parser logged the raw reply, the real layout was visible: nine empty fields indoors and, with a fix, 17 fields (18 while settling) in decimal degrees, so the old mapping had read satellite counts as the position. A modem that had run for several minutes answered every poll, and during warm-up it answered other commands at once, so only the GNSS was waiting. The silent capture did not fit warm-up, as MQTT commands also went unanswered: the power key toggles the modem, which keeps its supply when only the processor restarts, so the pulse sent at every boot switched a running modem off after each reprogramming.

Fixes and consequence. Since v0.5.3 the parser anchors on the latitude hemisphere letter (Section 4.6); since v0.5.2 the key is pulsed only if the modem does not answer `AT`. Tests must allow the warm-up, during which a zero poll count does not mean that polling has stopped.

4.10.3 Battery-management connection and checksum

Symptoms. In turn: no BLE advertisements from any device; once the pack was powered, a link that dropped before any data; mostly failed connections; and on the vehicle three frames, all rejected on the checksum.

Tests and causes. Every advertiser was logged, so seeing no device at all in a room full of them showed

that the scanner was idle: scanning depended on an address set only after the pack had been found. A teammate's independently written parser finds the characteristics directly, which showed that in the ESP32 library they are not under the assumed service; every service is now searched. A power cycle of the board did not change the failed connections, which ruled out a stuck BLE stack. The checksum formula came from community reverse engineering. The query frame the firmware itself sends, `DD A5 03 00 FF FD 77`, carries $\texttt{0x10000} - (\texttt{0x03} + \texttt{0x00}) = \texttt{0xFFFD}$, which leaves out the byte in the register position; the firmware included it, so every genuine reply failed by exactly three.

Fix and consequence. Firmware v0.5.4 corrects the sum and prints the bytes of any frame that still fails; 14 consecutive valid records followed on the vehicle. The protocol is confirmed, but the signal on the vehicle, −88 to −93 dBm, is near the limit for a link, so the unit may have to be mounted near the pack.

4.10.4 Invalid JSON found only by reading the log back

Symptom. At the first read-back of the stored log, on 1 October, 5,545 of 6,020 records failed to parse as JSON. Earlier transfer attempts had lost characters.

Candidate causes and test. Damage in transfer, over a serial link that also carries other output, or records wrong as stored. The dump command prints each record with its index and length, and the laptop script (`BenchTest/log_dump.py`) discards any record whose length does not match and repeats until every index has arrived intact. After four passes all 6,020 records were intact, so transfer was ruled out. Every failing record contained `nan`: a field with no value, such as the position before a fix, was printed as `nan`, which JSON does not allow.

Fix and consequence. The same text had been published over MQTT, and watching the live output had not shown it. Since v0.6.3 such fields are written as `null`, and all 6,020 records parse. A strict receiver would have rejected 92% of the records; later link tests count the lines that fail to parse at the receiving end (Appendix D).

4.10.5 Camera colour cast

Symptom and candidate causes. The live view looked reddish. A test was written for each of three causes: the detector's channel order, the camera itself, and the module's lack of an infrared-cut filter, which has its own tuning file.

Tests and cause. A still with the standard tuning showed a white wall strongly pink, as did the detector's frames; with the tuning for a module without the filter the wall was neutral. The cause was the module, not the channel order.

Fix and consequence. That tuning alone washed the colours out (a blue card at 120, 124 and 125 on red, green and blue); a saturation of 1.8 restored them (Table E.3). Black objects then looked purple, since many black materials reflect infrared; greying near-black pixels took 84 to 171 ms a frame, too slow for the detection loop, so it is done in the live view only. The colours must be checked again in daylight, where there is more infrared. Whether the cast lowered detection confidence was not measured.

4.10.6 Time-of-flight start-up hang

Symptoms. On 6 October every distance from the three sensors was null; after a first fix, on 10 October the reader froze.

Tests and causes. A scan of the I²C bus showed only the left sensor, at its new address, and nothing when the ahead sensor was woken. One sensor was not answering, but the start-up, which gives each sensor its address in turn, stopped at the first failure and lost all three. The reader was changed to skip and log such a sensor, and the bridge to look for it again every 60 s. That relies on an error being raised, and on 10 October none was: the driver's I/O timeout defaults to zero, which means wait for ever, so one stuck sensor froze all three.

Fix and result. Each sensor now has a 0.5 s I/O timeout. With it, the reader no longer froze but logged all three sensors as timing out, while a device stayed at the shared start-up address 0x29 throughout, even with every shutdown line held low: one sensor was not under the control of its shutdown line. After the sensor board's wiring was re-seated, all three opened, and in a three-minute hand test the ahead and right sensors read 80 to 264 and 157 to 199 mm with a hand about 20 cm away (`BenchTest/logs/2026-10-10_tof_hand_test.txt`).

Consequence. The loose wire was inferred from these symptoms, not seen. If a shutdown (XSHUT) wire comes off, its pull-up keeps that sensor on at the shared start-up address, where it collides with each sensor being woken, and none of the three starts (Appendix C). The timeout turns this into a logged failure, visible before a test, but cannot keep the other two running.

4.10.7 Raspberry Pi 4 under-voltage and reboots

Symptoms. Through much of the bench work the Raspberry Pi 4 reported under-voltage and throttled to 600 MHz. On 4 October the detector ran at 6.1 frames/s, against 14.6 later on a healthy Pi with the same services (`BenchTest/logs/2026-10-10_bench_observations.md`). On 6 October the Pi stopped booting until everything was disconnected.

Candidate causes. The Pi's own supply under the detector's load; the time-of-flight board on its I²C bus; the telemetry unit on its USB port; or heat, since it had no cooling.

Test. A logger recorded every 2 s the power and temperature flags, the I²C addresses and the USB serial devices, so that each change in the flags could be matched to a device (`BenchTest/logs/2026-10-06_pi4_power_watch.log`). With nothing attached and the detector running, the flags were clear, which cleared the supply. The time-of-flight board stayed disconnected. From 17:45 the soft temperature limit came and went (65 to 69 °C) with no under-voltage. The telemetry unit was plugged in at 17:58:05; 14 s later, when its modem came up, under-voltage and throttling began and recurred every few seconds, and the detector slowed from 14.3 to 12 frames/s.

Cause. The modem's transmit current, drawn through the Pi's USB port, pulls the Pi's supply down; the current was not measured. On 10 October, with the unit on the Pi's USB port, the Pi restarted three times in about 20 minutes; after the unit was moved to the laptop (through a small hub without its own supply), there were none in 4 hours and the flags stayed clear. The boot failure of 6 October was probably the same fault but was not reproduced. Heat was separate: 81 °C without cooling, 48 to 66 °C with a fan.

Fix and consequence. The fault lay in how the parts were powered together, not in any one part. The telemetry unit needs its own supply, such as a powered hub, on the bench and on the vehicle. On a 3 A supply the Raspberry Pi 5 limits its USB ports to 600 mA in total (its supply on the vehicle is still to be confirmed), so the same applies there, and a measured power budget against the 72 V source is still needed.

Table 4.2: Fault investigations: candidate causes, the test that separated them and the result.

Fault	**Candidate cause**	**Test**	**Result**
Boot loop (4.10.1)	PSRAM setting	Add `qio_opi` on top of the flash change	Same loop, same program counter: not the cause alone
	Application code or hardware	Remove every override	Runs, modem works: cleared
	Flash access mode	ROM banner; chip detection; 16 MB with DIO	DIO reported; boots, LittleFS mounts: cause
GNSS (4.10.2)	Reply layout not as assumed	Raw replies indoors and with a rooftop fix	9 fields empty, 17 or 18 with a fix, decimal degrees: cause
	GNSS warm-up	Poll a modem already running for minutes	Polls answered: normal 60 to 90 s delay
	Modem switched off	Five-minute capture after reprogramming	No reply to GNSS or MQTT commands: boot pulse toggled the modem off
BMS (4.10.3)	Pack absent or scanner idle	Log every advertiser	Nothing at all logged: scanner idle
	Characteristics under another service	Compare with a teammate's parser	Found directly there: cause of the drop
	Stuck BLE stack on the ESP32	Full power cycle	Same pattern: ruled out
	Checksum formula	Firmware's own query frame	Register byte excluded; replies failed by 3: cause
Invalid JSON (4.10.4)	Damage in transfer	Indexed, length-checked, repeated read-back	All 6,020 records intact: ruled out
	Records wrong as stored	Parse each record	5,545 contain `nan`: cause
Colour cast (4.10.5)	Detector channel order	Still with the standard tuning	Also pink: ruled out
	No infrared-cut filter	Still with the NoIR tuning	Wall neutral: cause
ToF null, then hang (4.10.6)	One sensor not answering	I^2C bus scan	Only the left sensor present; start-up stopped at the next
	Driver waiting for ever	Driver's default I/O timeout	Zero, wait for ever: cause of the freeze
Pi 4 under-voltage (4.10.7)	Pi supply under detector load	2 s flag log, nothing attached	Flags clear: ruled out
	ToF board on I^2C	Board disconnected throughout	Under-voltage still occurred: ruled out
	Telemetry unit on Pi USB	Plug in during the log	Under-voltage 14 s later, recurring: cause
		Unit on Pi USB, then moved off (10 Oct)	3 restarts in 20 min, then none in 4 h
	Heat	Same log	Soft limit from 17:45, before the unit: separate; fan fitted

Recurrence in the closed housing (16 October). With the Raspberry Pi 4 in the printed box, still powering the telemetry unit from its USB port, its temperature rose from 68.6 to 73.5 and then 76.9 °C over about 70 minutes with the detector running. The flags showed under-voltage, throttling and the soft temperature limit since boot (0xD0000). Later the same day it restarted while under-voltage was present at that moment (0x50005): a brown-out. In the same period the share of unreadable dashboard lines rose from about 2 to 10 % once the battery-management link became active. Two changes followed. The box had vents only in its floor and low in one wall, so both pods' lids gained an exhaust grille over the computer under a push-on rain hood (Appendix F); and the telemetry unit must have its own supply, as found above. The temperature in the vented box and the line-loss figure with a separate supply have not yet been measured. The Raspberry Pi 5 also restarted four times during the mounting work on 19 October. A power logger recording its supply, flags and temperature every 5 s showed normal values in the last reading before each restart (4.95 to 5.08 V, flags clean, 57 to 59 °C). The supply's lowest reading in the two minutes before each was 4.89 to 4.97 V. Those were abrupt losses of power, from a loose connector, the source or the supply's overload protection, not a brown-out or overheating.

Separate supply and a wired serial link (21 October). During the road session the telemetry unit was still powered from the Raspberry Pi 4's USB port, which also carried its data. It was then given its own battery, and the Raspberry Pi 4 started with no under-voltage messages, against 11 to 16 per start before. USB could no longer be used for data, because the board charges its battery from that port and would load the Pi again. The data moved to three wires instead: the unit's spare pins IO41 and IO42 to the Pi's UART on header pins 10 and 8, and a common ground, with no supply wire between them. Firmware v0.7.4.1 sends every dashboard line on this link at 460 800 baud, drops and counts a line rather than wait if the link's buffer is full, and accepts the local commands from the Pi. On the Pi, Bluetooth, which nothing uses, was disabled so that the full UART rather than the clock-dependent mini UART is on those pins, and the serial console was removed from them. In the first check the bridge read 216 lines without a bad one, and 2,489 by midday, against 2 to 11 % lost on USB and a third of the per-second records missing on the road session (Section 5.1); 27 records from that session also had damaged fields. Running on its own battery, however, the unit restarted by itself, after which its modem did not answer for the rest of the morning, which fits the battery sagging under the modem's transmit bursts. A battery alone is therefore not the answer: the unit needs a regulated supply from the vehicle.

4.11 Testing infrastructure

This section serves the tests of every requirement, R1 to R13, and so all four research questions: it describes the tools that produce the records of Section 3.5. Tools were written so that each test leaves a record that can be checked later. A browser dashboard reads the telemetry unit's serial output over USB through the Web Serial interface, with no installation, and shows the fused and per-sensor ride features, each IMU's rate and drop count, GNSS and BMS status, free memory and the boot self-tests, with rolling plots of vibration and disagreement (Figure B.3, Appendix B). It can replay a saved log through the same parsing, so a recorded session is reviewed exactly as it appeared live. The subsystem pages were later combined into one bench console that also shows the camera's live view and alerts, sends the firmware's console commands, filters the log by subsystem and saves timestamped sessions; it only reads data, and was checked by replaying the recorded session (Figure D.1, Appendix D). Scripted tests drive the firmware and record the result: `BenchTest/v050_bench_test.py` (commands, logging and reconnection), `BenchTest/signal_loss_test.py` (radio off for 60 s), `BenchTest/sync_outage_-`

`test.py` (radio off for 10 minutes, then backlog drain), `BenchTest/log_dump.py` (full log read-back) and `BenchTest/sw7_check.py` (pre-test check of every link). Offline tests replace the camera, model or bridge with stand-ins so that the detector's alert rules, the fusion and the dashboard pages can be checked without hardware.

4.12 Maintenance design

This section serves R13 (a removable installation) and the maintainability expected of a prototype that others in the programme will keep running; it supports RQ3, since recovery depends on how faults are found and fixed. The provisions below were checked in the repository; what is not yet maintainable is listed at the end.

- *Firmware updates.* The telemetry unit's firmware is a PlatformIO project (`TelemetryUnit/platformio.ini`, `TelemetryUnit/src/main.cpp`). It is built and flashed over the board's native USB port with `pio run -t upload`, and the board settings that caused the boot loop (16 MB flash, DIO mode, the 16 MB partition table) are fixed in that file with the reasons beside them (Section 4.10.1). The libraries are declared there with minimum versions, not pinned, so the installed versions are recorded separately (Table E.1).
- *Version strings.* One constant in `main.cpp` holds the firmware version (v0.7.4.1 at the time of writing). It is printed in the boot banner and by the `!status` command, and carried in the `fw` field of every published record and dashboard line, so each stored record says which firmware produced it.
- *Calibration and logs from the console.* The calibration factors are kept in non-volatile storage across reprogramming, and the console has commands to recalibrate, to switch local logging off and on, to flush it, and to dump and clear the stored log (Sections 4.4.1 and 4.9).
- *Deployment from the repository.* The Raspberry Pis are set up from files in the repository, not by hand. `CameraDetection/deploy/` holds the services for the rear unit's detector and telemetry bridge and an installation guide (`INSTALL.md`). `DashboardIntegration/deploy/` holds the dashboard's deploy script, the patch tool, the start script, the link set-up script for the Ethernet cable and the services for the dashboard, the front detector, the front time-of-flight bridge and the road recorder. The deploy script copies the integration to the Raspberry Pi 5, keeps the previous copy as `~/Dashboard_sw7.prev`, rebuilds the copy from the team's dashboard, applies the patches and writes the deployed git commit to a `DEPLOYED_COMMIT` file, so the version running on the vehicle can be traced to the repository (Section 4.8.4). The team's own dashboard is never changed, and one command restores its service.
- *Services that restart.* The detectors, the bridges, the dashboard and the road recorder run as systemd services that start at boot. The detectors, the bridges and the dashboard restart on failure after 3 to 5 s, and the road recorder restarts always. A crash therefore costs seconds of data, not the session, and the service logs keep the cause.
- *Tests that travel with the code.* The camera programs have tests in `CameraDetection/tests/` (alert logic, sensor fusion, time to contact, telemetry bridge, latency and others), the dashboard integration in `DashboardIntegration/tests/`, and the store-and-forward ledger a host test (`TelemetryUnit/test_host/`); a change can be checked against them before it is deployed. The pre-test check (`BenchTest/sw7_check.py`) reports every link before a session.

- *Hardware access.* The display and the sensor pod come off the vehicle without tools, on a bar clamp with a dovetail rail and a locking thumb screw, for charging, security and bench work (Table F.2). Repeated joints use heat-set inserts so that they survive being opened many times. The rear pod has a side hatch for the Raspberry Pi 4's flash drive. The revision-3 IMU pods are screwless, with push-fit lids, and the vent hoods push into the lids without screws (Appendix F). Every wire is listed by connector and pin in Appendix C, so a unit can be rewired from the report alone.
- *Records for diagnosis.* The evidence index (Appendix D), the lab notebook, the change log and the development record (Appendix E) give each fault, its cause and its fix, so a recurring fault can be recognised.

Not yet maintainable. The firmware has no over-the-air update: `main.cpp` contains no update code, so a new version needs a USB cable connected to the unit itself, which sits in the rear pod on the vehicle. The upload port is fixed in `platformio.ini` (COM10 on the development laptop), so another computer needs that line changed. The stored log must be read back and cleared by hand before it reaches its 3 MB cap (Section 4.9). The vehicle installation has not been timed for removal and return to standard (R13, Chapter 5), and the enclosures have not yet been tested for how often they can be opened.

The design described in this chapter is what the experiments of Chapter 3 were run on. Chapter 5 gives their results, requirement by requirement.

Chapter 5

Results

This chapter gives the measured results against the requirements of Table 1.1. Bench results were measured on the target board and the two Raspberry Pis between 25 September and 15 October 2026; results from the vehicle (19 to 21 October) are marked as such in the text and in Table 5.6. The raw evidence is indexed in Appendix D, and results still to be obtained are marked as outstanding where they will go.

The results come from the experiments of Section 3.4, collected and analysed as described in Sections 3.5 and 3.6, on the design of Chapter 4. The chapter is structured by requirement: acquisition, calibration and ride features (R1 to R3, Sections 4.3 and 4.4), position (R4), battery telemetry (R5), hazard sensing (R6 to R8, Section 4.8), telemetry and recovery (R9, R10, Section 4.9) and the driver display (R11, R12, Section 4.8.4). Within each section the earlier results come first and those of the first road session, where there are any, last. Integration and power follow, and Section 5.10 closes with a summary table against every requirement.

5.1 Inertial acquisition (R1)

Table 5.1 summarises the acquisition results. Both IMUs held 200 Hz with no dropped samples in every bench session, including with all subsystems running together on the target board. The only timing fault was the pause at each flash write. With a write every 10 s (v0.5.0) both IMUs showed a 33 to 40 ms gap at every write, twelve in two minutes; with logging switched off for the next two minutes the worst gap was 5.9 ms. With buffered logging (v0.6.0), in a 330 s run with five flushes, all ten gaps over 10 ms (37 to 40 ms, both IMUs at each flush) fell in the first statistics window reported after a flush, 0.2 to 1.05 s after it (Figure 5.1), and the worst gap elsewhere was again 5.9 ms. Each flush occupied the modem task for 71 to 202 ms, which does not affect acquisition. With v0.7.4 and its 94 Hz filter the rate and drop count were unchanged (T15). “No dropped samples” does not mean even spacing: in the 25 September session the shortest interval fell to 0.8 to 1.0 ms in each window with a write gap, as `vTaskDelayUntil` caught up, and jerk, computed with a fixed Δt, is distorted there. R1 is met on the bench as defined. The drop counter counts samples lost because the queue was full; a read that fails on the bus is not detected and would pass as a sample, so the loss figures cover the processing path, not the bus.

Figure 5.1 shows the longest sample interval in each 2 s window. With v0.5.0 (29 September), 30 of the

Table 5.1: Inertial acquisition on the target board (bench).

Measure	**Result**
Sample rate, each IMU	200.0 to 200.5 Hz in every bench session
Dropped samples (queue full)	0 in every bench session
Longest gap, logging off	5.9 ms; none over 10 ms
Gap at each flash write, 25 Sep session	24.8 to 29.9 ms every 12 s; shortest interval 0.8 to 1.0 ms in the same windows (catch-up)
Gap at each flash write, v0.5.0	33 to 40 ms, every 10 s
Gap at each flash write, v0.6.0 (buffered)	37 to 40 ms, about once a minute, each in the first window after a flush
v0.7.4, 94 Hz filter, 12-minute T15 run	200.0 to 200.5 Hz, 0 dropped in 736 windows; 36 to 44 ms at each flush, none added by the replay
All subsystems together (IMUs, BLE, modem)	400 to 401 samples per 2 s window on each IMU, 0 dropped
Dashboard line added (v0.7.0)	200 Hz on both IMUs, 0 dropped

151 logging-on windows per IMU had a maximum interval of 33.0 to 40.0 ms, one about every 10 s, and in the 59 windows with flash logging off the worst interval was 5.9 ms. With buffered writes in v0.6.0 (1 October) only 5 of 173 windows per IMU exceeded 10 ms (37.3 to 40.1 ms), each in the first window reported after one of the five flushes, and the worst interval elsewhere was 5.9 ms; no sample was dropped in either run.

On the first road session (19 October; Section 3.4.3), which included about 16 minutes of driving, the drop count stayed at zero on both IMUs throughout. The unit sends one record a second, but the recorder polled twice a second and the bridge repeated its last record until a new line arrived, so the 10,287 rear replies hold 3,464 distinct records. Over the 5,557 s from the first to the last (17:16:37 to 18:49:13), 2,093 seconds (38 %) have no record, and in the driving period (18:15 to 18:49) 67 % of seconds have one. The missing seconds are lines lost between the unit and the Raspberry Pi 4 over USB, the loss already seen on the bench (R10 below), and 27 of the records that arrived had damaged fields. Of the 3,437 readable records, 3,431 show both IMUs at 200.0 to 200.5 Hz; five, at 18:03 to 18:05, show 197 to 204.5 Hz, an uneven split of samples between two-second windows; and one, at 18:29:16 while driving, shows IMU1 with 4 samples in a 2 s window instead of 400, a stall of nearly 2 s that the drop count does not show and whose cause is not known. On the vehicle R1 is therefore partly met: no samples were dropped and the rate held except for that stall, but a third of the per-second records did not reach the recorder.

On 21 October, with the unit on its own battery and its record sent over the new wired link (Section 4.10.7), two faults appeared on the bench. After about 23 minutes both IMUs' drop counts were rising, by about 167 and 46 a second, until the unit was reset; after another reset IMU2 was not detected until the next. Neither cause has been found, and both have to be cleared before the second road session.

[Outstanding: the gap distribution on the moving vehicle, which needs the raw sample stream (firmware v0.7.5).]

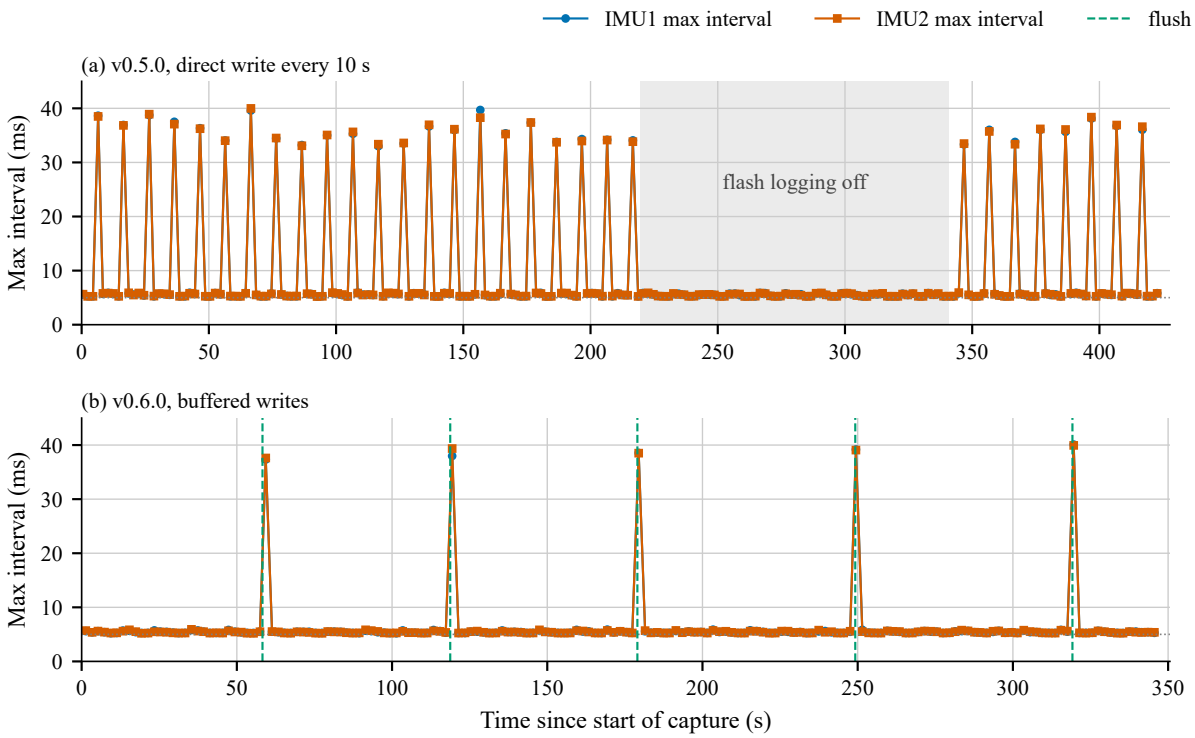


Figure 5.1: Longest interval between consecutive IMU samples in each 2 s statistics window, on the bench. (a) Firmware v0.5.0, 29 September, writing to flash every 10 s; the grey band is the period with flash logging switched off (219.7 to 340.8 s). (b) Firmware v0.6.0, 1 October, buffered logging; dashed lines mark the five flushes as logged by the firmware (71 to 202 ms each). The firmware reports only the minimum, mean and maximum interval per window, so each point is one window maximum (212 windows per IMU in (a), 173 in (b)); times are when the host received the line. The dotted line is the nominal 5 ms interval at 200 Hz. No samples were dropped in either run.

5.2 Calibration (R2)

Before calibration, the two IMUs on the development board read about 5.7% above and 2.6% below standard gravity at rest, about 8% apart. The factors stored at their first calibration were 0.9773 and 0.9901, after which both read 9.80 to 9.81 m/s^2, agreeing within measurement noise. The factors imply mean magnitudes at the moment of calibration of 10.03 and 9.90 m/s^2, not the readings seen earlier that day, so the uncalibrated readings had shifted between sessions; the likely cause, not confirmed, is that the sensors were moved in between. On the target board the first boot measured 9.44 and 9.57 m/s^2 (factors 1.0390 and 1.0245), and with those factors a later session read median resting magnitudes of 9.93 and 10.10 m/s^2, 1.3% and 2.9% above gravity. R2 was met immediately after calibration but not in the later session, which confirms that a single factor holds only for the orientation in which it was measured.

[Outstanding: recalibration in the mounted position on the vehicle and the resulting resting magnitudes.]

5.3 Ride features and fusion (R3)

The feature self-test passed at every boot: RMS, peak-to-peak and crest factor of the synthetic sine were within 5% of their analytical values. All five fusion self-tests passed. In a 150 s recorded session (Figure 5.2) all 149 fused windows combined both sensors with a skew below 1 ms (the log holds 222 fused lines, because a window can be printed by either sensor's task or both; each window is counted once). The planned disturbance, between 50 and 70 s on audible cues, did not happen because the cues were not heard; the board was disturbed from about 75 to 128 s instead, and the log is annotated to say so. While the board was still (the first 70 s), the fused vibration level σ had a median of 0.041 m/s^2 and a maximum of 0.067 m/s^2; during the hand disturbance it rose to 9.35 m/s^2, and the largest peak-to-peak amplitude in a window was 77.6 m/s^2, within 1 m/s^2 of the ±8 *g* range, so that peak was probably clipped. The disagreement figure D ranged from 0.00 to 1.68. In an earlier disturbance test both sensors rose together (peak-to-peak from about 0.2 to 18.4 and 10.2 m/s^2, crest factor from about 1.01 to 2.17 and 1.43, jerk from about 8 to 160 and 162 m/s^3) and decayed together, so the calibrated pair also tracked each other under motion, not only at rest. R3 is met. These results show that the features are computed and combined as designed; they do not show how the features respond to the road or the motor.

On the road session the per-second vibration value separated the states clearly. Counting each record once, and labelling a second as driving when the battery current exceeded 1 A or the value exceeded 0.3, it had a median of 0.03 in the 2,832 parked seconds and, in the 605 driving seconds, a median of 1.39, a 95th percentile of 3.2 and a maximum of 5.09, with battery current of up to 46.9 A. The full 1 s features were stored only in the unit's 10 s log, no raw samples were kept, and the runs were not marked, so the per-run analysis below is outstanding.

Figure 5.3 shows the 19 October session from the rear unit's telemetry. After removing repeats there were 3464 records between sequence numbers 4549 and 10105, so 2093 of 5557 (37.7 %) were not received: 407 in one gap from 18:06:34 to 18:13:23, the rest in short gaps of at most 8 records. Vibration stayed near 0.03 m/s^2 (median 0.030) while parked and had a median of 1.39 m/s^2 (maximum 5.09) in the 607 records flagged as driving (605 of them with every field readable), and the battery current reached −46.9 A with a median of −6.3 A over the 307 records above 1 A.

[Outstanding: ride features, speed-normalised index and disagreement on the vehicle: stationary

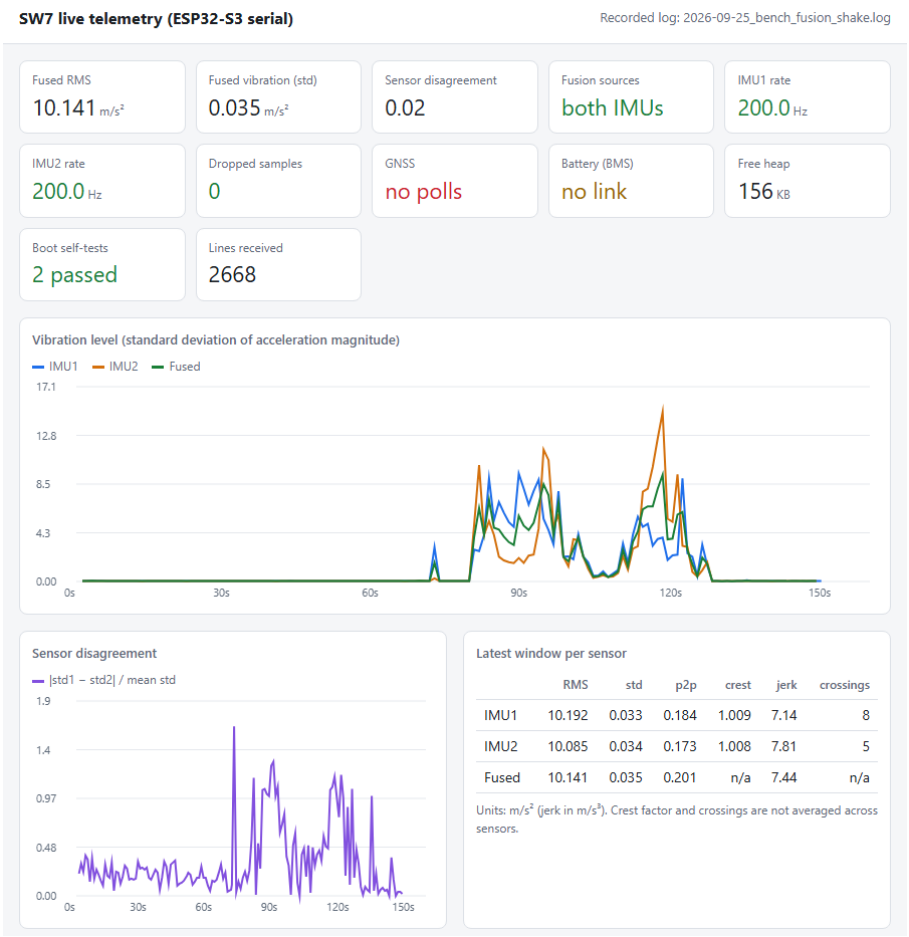


Figure 5.2: A 150-second bench session recorded from power-up and replayed through the bench dashboard: still until about 75 s, disturbed by hand until about 128 s, then still again.

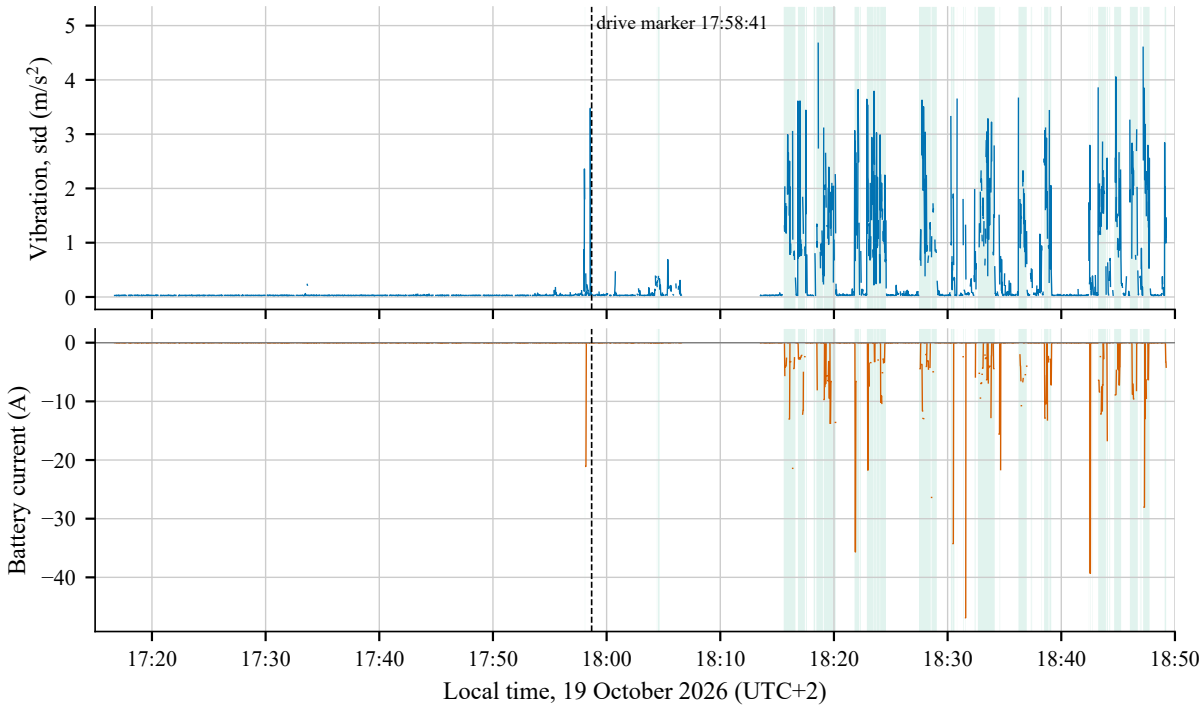


Figure 5.3: Road session on 19 October 2026, 17:15 to 18:50 local time (UTC+2), as recorded by the Raspberry Pi 5 from the rear unit (file `2026-10-19_171534_roadtest.jsonl`). Top: vibration value (standard deviation of the fused acceleration magnitude per 1 s window). Bottom: battery current from the BMS, negative for discharge. The recorder polled twice a second and the bridge repeats the latest record, so the 10 287 rear rows carrying a telemetry record were reduced to 3464 by keeping the first row for each sequence number. Lines break wherever a sequence number is missing (2093 of 5557, 37.7 %, including one gap of 407 records from 18:06:34 to 18:13:23). Green shading marks records with $|i| > 1$ A or vibration above 0.3 m/s^2 (607 records); the dashed line is the drive marker added from the laptop at 17:58:41.

powertrain baseline at each motor setting, repeated traversals of the same road sections, and speed dependence (RQ1, RQ4).]

5.4 Position (R4)

With the corrected parser, the receiver reached a three-dimensional fix 105 s after a cold start (51 s in an earlier session), with 23 to 25 satellites and a horizontal dilution of precision of about 0.9 to 1.5. Over 104 fixes the position was a median of 2.7 m from a phone's position, 95% within 11.0 m and at most 12.3 m away. The phone has an error of a few metres itself, so this is a consistency check, not an absolute accuracy; against the criterion of 10 m, the median and most fixes pass but the 95th percentile does not. The speed field read 0 to 5.8 while stationary; the manual gives knots, but the readings and their low-speed noise have not been checked against a known speed. The network position (v0.7.3) answered in about 0.7 s with a reported accuracy of 550 m.

On the road session of 19 October the receiver reported no fix in any of the 3,464 records, so no position or speed was recorded during the drive. The cause was that no GNSS antenna had been connected to the board, not a fault in the receiver; only the network-based position (about 550 m accuracy) was available. R4 is therefore untested on the vehicle rather than failed.

[Outstanding: time to first fix with assisted GNSS, and the speed field against a known speed on the vehicle.]

5.5 Battery telemetry (R5)

After the checksum correction the unit stored 14 consecutive valid records on the vehicle over about 140 s: 74.78 V, 0.00 A and 60% state of charge (3.12 V per cell for 24 cells), at −88 to −93 dBm, after which the connection was lost. On 16 October, with the telemetry unit moved to within about 2 m of the pack (Section 4.8.6), the link held: 60 % state of charge, 74.67 to 74.68 V and 0.0 A with the vehicle off, each record 0.2 to 1.3 s old. Before that the laptop had found the unit advertising as "DB24SA01L24S150ABU" at −100 dBm, at the edge of reception, which explains the earlier drop-outs. The name gives a 24-cell pack and a 150 A BMS; 74.7 V at 60 % (3.11 V per cell) fits lithium iron phosphate, about 60 to 88 V over its range. With the system mounted on the vehicle on 19 October the link held throughout the session, at 60 % and 74.57 to 74.58 V, with each record under 3 s old. Until the road session the current had not been seen under load.

On the road session the link held throughout. The current reached 46.9 A under load, the pack voltage fell from 74.6 to 72.8 V while driving, and the state of charge from 60 to 58 %. R5 is partly met: the link held on the vehicle under load, but the voltage has not been compared with a meter.

[Outstanding: the voltage checked against a meter.]

5.6 Hazard sensing (R6 to R8)

Table 5.2 summarises the hazard-sensing results and Table 5.3 gives the latency breakdown. With four inference threads and nothing else running (no bridge, no fusion polling, live view not watched; 30 September) the detector ran at 19.5 frames/s with a median of 102 ms from sensor readout to result; about 52 ms of that was readout and the camera's image processing, and 36 ms inference. With the whole system running on a healthy Raspberry Pi 4 (11 October, from the service's own timing lines) it ran at 14.6 to 14.7 frames/s, median 118 to 122 ms, 95th percentile 146 to 148 ms; the cause of the difference has not been isolated, and these are the figures that represent the rear unit on the vehicle; the front unit on the Raspberry Pi 5 is in Table 5.2 and Section 4.8.6. The three-frame onset adds about two frame intervals, so at the detector an alert is raised about 0.26 s, or 2.2 m at 30 km/h, after an object first appears; the bridge's and dashboard's polling and rendering add an unmeasured delay before the driver sees it. A person was detected with confidence 0.50 to 0.73; only the person class has been confirmed on live frames. These are bench figures with the camera indoors and no objects to track; the number of detections does not change the inference time, but the vehicle's heat and supply may, and detection has not been tested outdoors or in daylight, rain or glare. R7 is met at the detector on the bench, for the person class. R6 is met for the pulse itself, which the camera unit timestamps with a spread of under 0.7 ms. The offset between the camera's and the telemetry unit's clocks, which aligning the two streams needs, has not been measured; the median interval of 500.06 ms also means the two clocks run about 120 parts per million apart, about 0.4 s an hour, so the alignment must fit a drift as well as an offset. The front camera on the Raspberry Pi 5 has no pulse wire.

Figure 5.4 compares the per-frame time from sensor readout to detection result. On the Pi 4 with four inference threads the median was 102 ms (95th percentile 120 ms, 1161 frames) and 105 ms with the live view open (127 ms, 1031 frames); with one thread it rose to 186 ms (207 ms, 419 frames) as inference went from 36 to 126 ms. The Pi 5 on 15 October had a median of 53 ms and a 95th percentile of 62 ms over 1151 frames, with inference at 28 ms and queue wait at 18 ms against 52 ms on the Pi 4. The 11 October whole-system log on the Pi 4 (bridge, fusion and preview running) recorded only per-interval summaries, median 118 to 122 ms and 95th percentile 146 to 148 ms, so it is not plotted.

The time-of-flight stand-ins answered and each responded to a fingertip in the right order (centre 267 to 300 mm and right 24 to 38 mm on the bench; left 8190, the code for no target), so the first part of R8 is met; their accuracy against a tape and the fusion validation (test C9) have not been run, so the rest of R8 is outstanding. The first live fused hazard shows the matching working and how far the uncalibrated camera distance can be wrong at close range. The fusion's sensor directions and the camera's focal length are still defaults.

On the road session (17:15 to 19:10) the front detector raised 1,342 alerts: people 815, of which 366 were marked approaching, and cars 513, of which 261 were approaching. The rear detector raised 741, from its own log: people 514 (183 approaching), cars 201 (36 approaching) and 26 other vehicles. Whether each alert was correct has not been checked against video.

[Outstanding: fusion validation C9 (distance error against a tape at 0.5, 1.0 and 1.5 m), focal-length calibration, the pothole model's held-out test score and a run on a road, and detection on the vehicle outdoors.]

Table 5.2: Hazard sensing on the bench: the Raspberry Pi 4 (the only camera unit until 15 October, the rear unit in the final layout) unless a row names the Raspberry Pi 5.

Measure	**Result**
Camera to telemetry-unit alignment (C6)	Sync pulse median 500.06 ms, range 499.72 to 500.41 ms (121 edges, 60 s)
Detector rate (SSD-MobileNet, 4 threads)	19.5 frames/s alone; 17.3 with the live-view server on; 14.6 to 14.7 with the whole system
Sensor readout to result	Alone: 102 ms median, 120 ms 95th percentile, 136 ms maximum. Whole system: 118 to 122 ms median, 146 to 148 ms 95th percentile
Alert onset at the detector (3 frames)	about 0.26 s, 2.2 m of travel at 30 km/h (0.2 s, 1.7 m alone)
Alert clearing	before the fix, 16 activations in 47 s for one person; after it, cleared only after 5 empty frames
Time-of-flight sensors (3 × VL53L0X)	about 31 readings per second each; steady; each confirmed by covering it; not yet checked against a tape. On 12 to 13 October the right sensor started and answered at its address but returned no range (status 4, signal 0 to 13 in 357 readings over 90 s) while the left and ahead sensors ranged; wiring ruled out, the sensor's aim suspected; after it was adjusted on 14 October it ranged normally (0.63 to 0.67 m, signal about 70)
Camera and time-of-flight fusion	29 offline tests pass; first live fused hazard: camera estimate 0.874 m, time-of-flight 0.096 m, reported 0.096 m (before calibration)
YOLO11n, COCO, NCNN, 320 input	7.9 frames/s, 175 ms median readout to result
Approaching-object warning (time to contact from box growth, tracker, ToF closing rate; 15 October)	Implemented; 40 synthetic tests pass (time to contact within 2 % at constant speed, no alerts in 5,000 frames of receding, still or jittering boxes); 0.27 ms per frame on the laptop; not yet run on either Pi or against a real approach (test C11)
Same detector on the Raspberry Pi 5 with the dashboard running (15 October, 60 s, second OV5647 camera)	19.2 frames/s; readout to result 51 to 54 ms median, 61 to 63 ms 95th percentile (inference 28 ms median); CPU 64 to 73 %, 53 to 54 °C, no throttling; one person alert raised and cleared
Pothole model, run 1	mAP@50 0.597 on its own test set; 0.000 on South African frames
Pothole model, run 2 (road-band tiles)	mAP@50 0.714 at 640 and 0.566 at 320, on validation tiles
Pothole path inside the detector (C10)	Found the pothole in a test picture (confidence 0.63); detector 13.9 frames/s at one run per 4 s, 10.9 to 13.3 at higher rates or with more threads; off by default

Table 5.3: Camera detector latency on the Raspberry Pi 4, the rear unit in the final layout (ms, median / 95th percentile; 60 s per run, except the whole-system row, read from the service's timing lines).

Run	frames/s	Queue	Pre	Inference	Alert logic	Total
4 threads	19.5	52 / 71	12 / 14	36 / 39	0.1 / 0.2	102 / 120
4 threads, live view	17.3	57 / 78	12 / 12	36 / 39	0.1 / 0.2	105 / 127
1 thread	7.0	47 / 68	12 / 14	126 / 129	0.1 / 0.2	186 / 207
Whole system	14.6–14.7	55–58 / -	-	51–52 / -	-	118–122 / 146–148

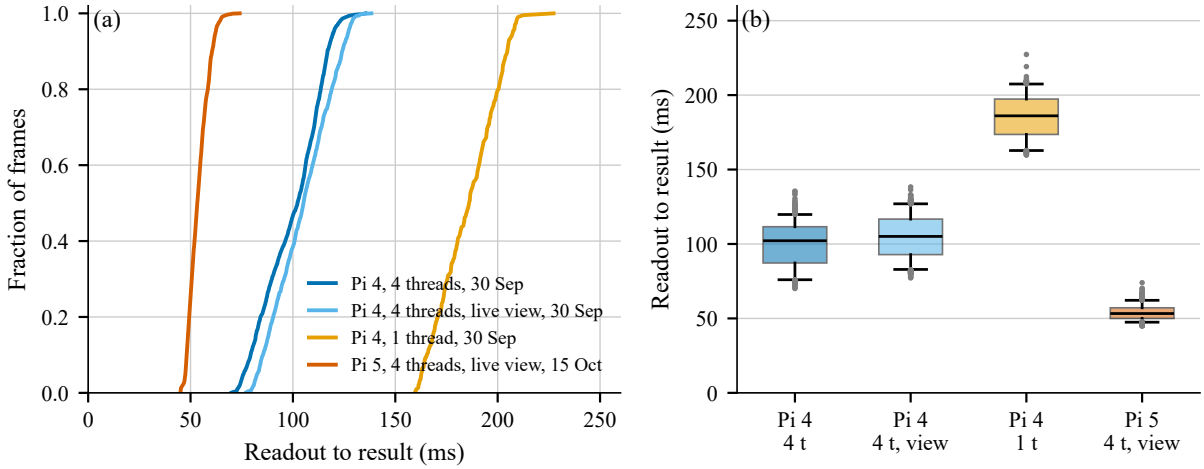


Figure 5.4: Detector latency from camera readout to detection result, per frame, for 60 s runs of the TensorFlow Lite detector (`detect.tflite`, 300×300 input, 640×480 camera stream). (a) Cumulative distribution; (b) box plots with the box at the quartiles, whiskers at the 5th and 95th percentiles and dots beyond them. Pi 4 runs on 30 September: four threads (1161 frames, 19.5 frames/s), four threads with the live view open in the Pi's browser (1031 frames, 17.3 frames/s), one thread (419 frames, 7.0 frames/s). Pi 5 run on 15 October: four threads with the live-view server running (1151 frames, 19.3 frames/s). Data from the detector's per-frame timing CSV files.

5.7 Telemetry, logging and recovery (R9, R10)

The unit connected to the broker at the first attempt and published 47 times with no failures; a requested reconnection completed in 0.4 s. With the modem's radio switched off for 60 s, the firmware detected the loss from the modem's report 0.8 s after the radio went off, or after three failed publishes at 20.7 s, and retried after 30 and then 60 s. In v0.5.0 it reconnected 36 s after the radio returned, waiting for its next retry although data were back within 2.4 s; v0.5.3 reconnected about 5 s after. Logging continued throughout. A full read-back recovered all 6,020 stored records (2.2 MB, about 16.7 hours at one record per 10 s) after four passes; 5,545 were not valid JSON because of `nan`, and after the v0.6.3 substitution all 6,020 parse.

Synchronisation with the database was tested with v0.7.4 (test T15, 11 October, indoors, public test broker; Table 5.4). Every record was stored once, but 24 only after a backfill from the unit's log: two published while the ingest was restarted, the rest before an ingest fault found during the test was fixed (its persistent session on the broker stopped delivering while still connected; such sessions are now opt-in). The public broker acknowledges a record with no subscriber present and does not queue it, so on this broker reading the log back is the safety net. R9 is met on the bench, including the database.

Table 5.4: Store-and-forward to the database after a 10-minute outage (test T15, firmware v0.7.4, bench).

Measure	**Result**
Outage	Radio off (`AT+CFUN=4`) for 600 s; 60 records built up, 0 lost
Reconnection after the radio returned	5.2 s
Backlog cleared	37.3 s for the 60 replays at the database (1.58 per second); unit's backlog zero 42.8 s after reconnection
Stored in the database	112 of 112 records of the boot; 0 missing sequence numbers; 0 duplicates
Against the unit's log	111 of 111 logged records stored (24 backfilled from the log)
IMUs during the test	200.0 to 200.5 Hz, 0 dropped
End-to-end latency	Not measured (no GNSS time indoors)

Figure 5.5 shows the backlog during T15. With the radio off for 600 s the backlog grew by one record every 10 s to 60 records, MQTT reconnected 5.2 s after the radio was switched back on, and the backlog was cleared 37.9 s later. On the server side the 60 replayed records arrived over 37.3 s (1.6 records/s), and the database held all 112 records expected for the boot (sequence 65 to 176) with no duplicates and none missing.

The dashboard line (v0.7.0) delivered 40 lines in 40 s with no gap in the sequence numbers, every line valid JSON and at most 272 bytes, with both IMUs at 200 Hz and no drops. Through the bridge on the Raspberry Pi 4 the record reached a laptop, not yet the Raspberry Pi 5, 0.37 s old with no malformed lines, and opening the port did not reset the telemetry unit; the pre-test check on 4 October found the record 0.5 s old and the telemetry answering in 18 ms. In a longer bench run (12 to 13 October, 4.7 hours, v0.7.4 with the modem and the database sync active) the bridge counted 858 unusable dashboard lines among 9,229 lines read (9.3 %); the cause has not been found. On 16 October it fell to about 2 % (1.9 to 2.8 %), then rose to 10 to 11 % once the battery-management link was active, with the Raspberry Pi 4 under-voltage at the same time (Section 4.10.7). A skipped line delays the dashboard by a second; the unit's own log and the uplink are not affected. On the road session a third of the per-second records

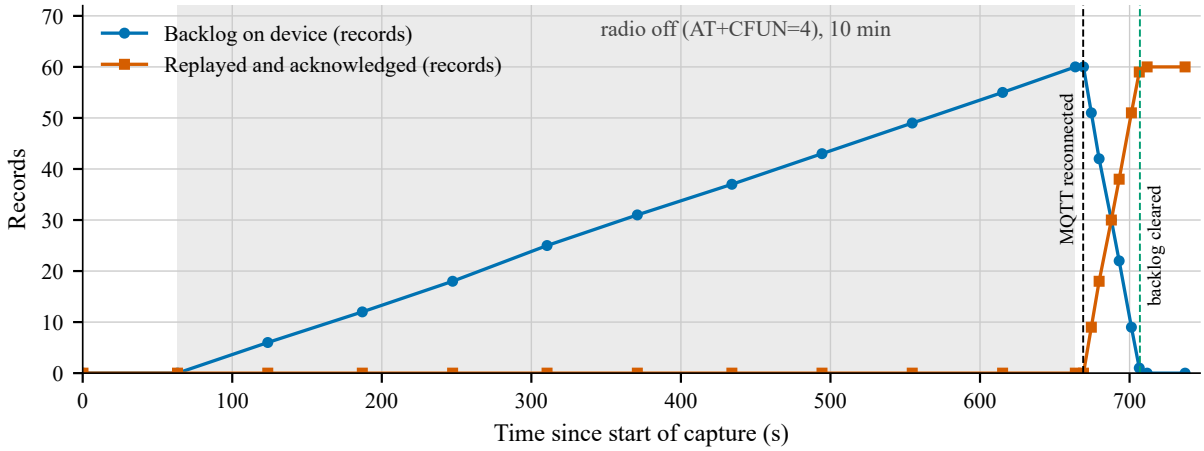


Figure 5.5: Store-and-forward outage test T15 on the bench, 11 October, firmware v0.7.4. The radio was turned off with `AT+CFUN=4` at 63.4 s and back on with `AT+CFUN=1` at 663.7 s (grey band); the board was not reset. Points are the device's own counters from the 21 `!sync` status lines in the capture: records stored on the device and not yet acknowledged (backlog), and records replayed and acknowledged since the outage began. Dashed lines mark the MQTT reconnection (669.0 s) and the firmware's "backlog cleared" message (706.9 s). The server's per-record receive times were not exported for this run, so the figure uses the device counters; the server-side totals are in the text.

did not reach the recorder for the same reason (Section 5.1). Since 21 October the record travels over a wired UART instead of USB (Section 4.10.7); none of the first 2,489 lines was bad, but a long run has not been done. R10 is met only in the 40-line test and is partly met overall.

[Outstanding: end-to-end synchronisation latency with GNSS time, and the same measures on the moving vehicle.]

5.8 Driver display (R11, R12)

Run on a laptop inside a copy of the dashboard's code, against the live detector on the Raspberry Pi 4 (1 October), the camera page showed video 0.6 s after it was opened and decoded 14.2 frames/s against the detector's 14.8, with no gap longer than 0.22 s; with a person in front of the camera it listed a red "Person, Immediate" alert 0.5 s after the page was opened with the person already in view (not an onset latency), and it switched to its offline state within a second of the detector stopping; the person stayed in view, so clearing on the page was seen only in the offline test and when the camera stopped, although the detector's log shows person alerts both raised and cleared. In live-only mode, run offscreen on a laptop against a simulated bridge (3 October), every page showed "--" or "Unavailable" with the bridge down and never a simulated value, and the dashboard returned to the no-data state, not to simulation, when the bridge stopped. In an offscreen run of the whole integrated dashboard on the Raspberry Pi 5 against the live bridge (6 October), the Sensors page showed the ahead sensor at 0.43 m in red, the right at 0.69 m in amber and the left with no reading, and the banner appeared over the driving page; the page's use by touch had not been checked at the time. Reading the touch controller during 25 taps, the panel reported contact together with the first position in every tap, so it adds no measurable delay; the taps lasted 200 to 640 ms and the buttons act on release, and the processor was idle (load average 0.04). R11 is met on the bench, from laptop and offscreen runs. R12, the glance demand, has not been measured, and the layout has not been checked against the legibility figures of Section 2.6.

[Outstanding: alert onset to screen on the Raspberry Pi 5, legibility in daylight, and glance observation on the vehicle.]

5.9 Integration and power

All telemetry-unit subsystems ran together on the target board, and the chain from the telemetry unit through the Raspberry Pi 4 to the dashboard on the Raspberry Pi 5 ran on the bench. The under-voltage that throttled the Raspberry Pi 4 through the bench work, and held the detector at 6.1 frames/s on 4 October, was traced to the telemetry unit's modem on its USB port (Section 4.10.7). The Pi reached 81 °C without a fan and 48 to 66 °C with one.

On 19 October the whole system was mounted on the vehicle and checked in the lab. Both cameras and detectors ran (front 17.5 to 22.0 frames/s, 53 to 102 ms; rear 14.4 to 14.5 frames/s, 126 to 151 ms, slower while the Raspberry Pi 4 was throttled). Both time-of-flight boards answered, both IMUs held 200 Hz with no drops, and the battery link reported on the vehicle. The Ethernet link ran at 1000 Mb/s with a 0.25 ms round trip, and the recorder on the Raspberry Pi 5 received all four sources at 2 Hz. The Raspberry Pi 4 logged 11 to 16 under-voltage warnings per boot with the telemetry unit still on its USB port, and both computers restarted during the mounting work (Section 4.10.7). Table 5.5 gives the power budget against the 72 V pack (GA 5). Only the Raspberry Pi 5 figure is measured, from its own power-management readings on 14 October with the dashboard running (2.2 to 3.0 W on its internal rails, before it took on the front camera, detector and time-of-flight board on 15 October, so its share in the final layout is higher than shown); the rest are estimates from datasheets and published bench figures, to be replaced by inline USB meter readings. On the same day the Raspberry Pi 4, with the detector using three of its four cores, reported that its supply had fallen below the under-voltage threshold since boot (`get_throttled` 0x50000), so its present supply is not adequate.

Table 5.5: Power budget of the added electronics at 5 V, and the draw on the 72 V pack through a DC-DC converter of about 88 % efficiency. Typical is the steady draw while driving; peak is the worst short burst, not all at once.

Load	**Typical (W)**	**Peak (W)**	**Source**
Raspberry Pi 5, dashboard (measured before the front camera unit was added)	4.0	8	Measured 2.2 to 3.0 W on the rails, plus input conversion and USB
10.1 in display, full backlight	5.0	6	Estimate; to measure
Raspberry Pi 4 with camera, fan and three ToF sensors, detector running	6.8	8	Estimate from published full-load figures (about 6.4 W) plus the camera, fan and sensors
ESP32-S3, A7670 modem (with GNSS) and two IMUs	1.0	9	Estimate; the modem's transmit bursts of up to about 2 A are the peak, and the cause of the brown-out in Section 4.10.7
SEN55 (planned)	0.4	0.5	Datasheet
Total at 5 V	**17**	**31**	
From the 72 V pack	**20 (0.27 A)**	**35 (0.5 A)**	At 88 % converter efficiency

At about 20 W the added electronics use 20 Wh per hour, 160 Wh over an eight-hour shift. Against a 7.2 kWh pack, the figure the team's dashboard uses (not confirmed; the BMS's name gives 24 cells but not the capacity), that is 0.28 % of the pack per hour and 2.2 % per shift, and about 0.8 % of the motor's rated input of 2.5 kW. The draw is small while driving but not when parked: left on, the full system

would empty the same pack in about 15 days, so it must switch off with the vehicle, leaving at most the telemetry unit (about 1 W) on. The converter must accept the pack's whole voltage range, about 60 to 88 V for its 24 lithium iron phosphate cells (R5 above), and be rated to at least 100 V for transients, and deliver at least 40 W at 5 V for the peaks, with a fuse of about 2 A on its 72 V input.

On the road session the recorder on the Raspberry Pi 5 ran for 108.6 minutes without the laptop and did not restart while driving, and the front sources were recorded throughout. The rear unit answered it until 18:49:13, just after the last driving segment, and not for the last 15 minutes (1,785 failed requests), although the rear detector kept logging its alerts until 19:02; the cause was not recorded. Before that the rear data had one gap, of 409 s, before the drive, and a third of its seconds had no new record (Section 5.1).

[Outstanding: full-system test on the vehicle over a defined route, with the telemetry unit on its own supply; inline current measurements of the display, the Raspberry Pi 4 and the telemetry unit to replace the estimates in Table 5.5; the pack's capacity and chemistry from the vehicle team.]

5.10 Summary against the requirements

Table 5.6: Status of each requirement at the time of writing (bench unless stated).

ID	Result	Status
R1	Bench: 200.0 to 200.5 Hz, 0 dropped (260 and 94 Hz filters); gaps over 10 ms only at a flush, with catch-up samples. Vehicle: 0 dropped, one 2 s stall of IMU1; 38 % of per-second records not received	Met on the bench; partly met on the vehicle; gap distribution outstanding
R2	9.80 to 9.81 m/s^2 after calibration; drifts with orientation	Met at calibration; mounted recalibration outstanding
R3	Self-tests pass; 149 of 149 windows fused	Met
R4	Median 2.7 m, 95% within 11.0 m of the phone; no fix on the road session	Partly met on the bench; untested on the vehicle (no antenna)
R5	Sustained on the vehicle: up to 46.9 A, 74.6 to 72.8 V under load; no meter check	Partly met: sustained under load on the vehicle; meter check outstanding
R6	Median interval 500.06 ms, range 499.72 to 500.41 ms; clock offset not measured	Partly met: pulse timing met, offset outstanding
R7	Rear (Raspberry Pi 4): 14.6 frames/s with the whole system (19.5 alone), onset at the detector about 0.26 s. Front (Raspberry Pi 5): 19.2 frames/s on the bench beside the dashboard, 17.5 to 22.0 on the vehicle	Met at the detector (person class)
R8	Sensors answer; no tape check; fusion not validated	Partly met
R9	47 of 47 publishes; reconnection about 5 s after coverage; full read-back; after a 10-minute outage 112 of 112 records stored, backlog cleared in 43 s	Met on bench via a test broker; latency and vehicle outstanding
R10	40 of 40 lines, record 0.37 s old at a laptop; 2 to 11 % of lines lost over hours on USB; wired UART since 21 October, 0 bad in 2,489 lines	Partly met; long run on the wired link outstanding
R11	Live-only presentation; alert listed 0.5 s after opening the page with a person in view; offline state	Met (laptop and offscreen)
R12	Not measured	Outstanding
R13	Mounted on the vehicle on 19 October; removal and return to standard not timed	Partly met

Chapter 6

Discussion

This chapter interprets the results against the literature and the research questions. Its main limitation is stated throughout: the results are from the bench.

6.1 Acquisition integrity and what it costs

The literature review found that edge-telemetry studies are usually judged by accuracy or mean latency, without jitter, queue behaviour or outage recovery. The results here report those quantities directly: zero dropped samples, the worst gap with and without logging, and the cause of the only recurring gap. The flash-write pause shows why they matter. Buffering cut its rate sixfold but not its length, because the pause comes from the flash itself. A 40 ms gap in a 200 Hz stream is eight missing sample instants once a minute, small for 1 s ride windows but significant for impact detection. Logging to separate storage, or on the Raspberry Pi, would remove it rather than reduce it. The open sensor settings belong to the same requirement: the 260 Hz filter of the earlier bench sessions let content above 100 Hz alias into the band (v0.7.4, used for T15 and the road test, has 94 Hz), and the $\pm 8\,g$ range was probably reached by a hand disturbance, so clipped samples should be flagged before vehicle data are interpreted.

6.2 Ride characterisation (RQ1, RQ4)

The calibration result supports the review's warning that low-cost inertial measurements depend on the individual sensor and its mounting: two nominally identical units disagreed by about 8% in one session and read 10.03 and 9.90 m/s^2 at calibration in the next, and their factors changed between boards because one magnitude factor holds only for one orientation. A ride index compared across sessions therefore has to be calibrated in its final mounting, which the firmware supports with one command. The features are computed correctly, but on the magnitude rather than a vehicle-fixed vertical axis and without ISO 2631 weighting, so they describe overall vibration exposure, not comfort. The equal-weight fusion and the disagreement figure are designed to show when the two mounting points see different motion; a D of 1.68 in the bench session shows that this happens even on a hand-held board, and it will be the first thing to examine on the vehicle.

RQ4 cannot be answered from the bench. The method is in place: with the motor's Hall pulses giving its speed, vibration that follows motor speed with the vehicle on stands can be separated from vibration that appears only on the road, as the review recommends [17]. It depends on the Hall tap and on the first vehicle sessions.

The first road session (19 October) supports two findings. On the vehicle no inertial samples were dropped and the rate held except for one stall of IMU1 of about 2 s, although a third of the per-second records were lost on the USB link to the recorder; and driving and parked are separated clearly by the per-second vibration value together with the battery current. It cannot yet answer the questions that matter most. Without GNSS speed, raw samples or marked runs, neither the repeatability over repeated traversals (RQ1) nor the separation of powertrain from road vibration (RQ4) can be assessed. A second, shorter session is planned with the raw sample stream, the GNSS antenna connected (it was not fitted on the first), timed runs and the stationary powertrain baseline.

[Outstanding: interpretation of the road-test data: repeatability of σ and I_v over repeated traversals, speed dependence, and whether powertrain components are separable (to be written after the second road session).]

6.3 Hazard sensing

At the detector the camera pipeline is fast enough for its purpose: an alert about 0.26 s after an object appears is about 2.2 m of travel at 30 km/h, although the delay to the screen is unmeasured. The weak point is distance, not speed. The first fused hazard put a person at 0.874 m by the camera's width heuristic and 0.096 m by the time-of-flight sensor, so a camera-only band can be a whole band wrong at close range; this is the case for fusion, in which the camera says what and where and the sensor says how far. The stand-in sensors' 1.2 m range is only about 0.14 s of travel at 30 km/h, so they serve low-speed manoeuvring, and the camera remains the only source at road speed until longer-range sensing is fitted. The pothole models show the same dependence on local data that the review noted for roughness classifiers: a model scoring 0.597 on its own source found almost none of the potholes in South African footage, and training on the road band of that footage raised the validation score to 0.714. The held-out score and a run on the vehicle's own camera, which has no infrared-cut filter, are needed before that improvement can be claimed.

Two later changes address the weaknesses above. First, the time to contact from scale change (Section 4.8.5) turns the camera's least reliable output, distance, into one that needs no calibration. That fits the review's finding that monocular distance is the weak link and that production warnings rely on scale change instead [59]. The time-of-flight sensors supply the same measure inside their short range, so at road speed the warning still rests on the camera. Its accuracy is untested (C11), and a stationary object ahead counts as closing while the vehicle moves, which is the intended forward-collision behaviour, not a fault. Second, moving detection to the Raspberry Pi 5 beside the display (Section 4.8.6) cut the median from readout to result from 118 to 122 ms (Raspberry Pi 4 with the whole system) to 51 to 54 ms (Raspberry Pi 5 beside the dashboard). The two bench runs did not carry identical loads, but on the vehicle the front unit ran at 53 to 102 ms against 126 to 151 ms at the rear, so the gain held. A front alert no longer crosses the network to reach the screen. At 30 km/h that is about 0.6 m less travel before the alert, the same order as the stand-in sensors' whole range. The price is a second processor's load beside the dashboard (64 to 73 % of the Raspberry Pi 5's CPU in total) and a rear unit whose own detector, on the Raspberry Pi 4, remains the slower path.

6.4 Data integrity and connectivity (RQ3)

Logging independently of the connection, and reconnecting when the modem reports that data are available, gave the store-and-forward behaviour the review identified as necessary for intermittent coverage. The read-back is the important result: it recovered every record and found a fault, invalid JSON for missing values, that watching the live stream had not shown. With v0.7.4 the path reaches a database: after a 10-minute outage every logged record was stored once and the backlog cleared within a minute, which gives the completeness, duplicate and recovery measures of Chapter 2. On the public broker, however, an acknowledgement does not mean that a subscriber received the record, so completeness rests on comparing the database with the unit's log, as it did for 24 records; a broker that queues for a persistent subscriber would remove that dependence. Delay needs GNSS time and is left to the road test.

[Outstanding: synchronisation delay and completeness on the road test.]

6.5 Driver display (RQ2)

The choice of the existing dashboard over a windshield HUD is supported by the review: a head-up position does not make a display safe by itself, its optics are hard to make legible on a removable unit in daylight, and a second display adds a competing channel. Of the five dashboard issues, data validity and update age were the ones that integration showed to matter most: the inherited dashboard showed simulated speed and battery values without any sign whenever its data link was down, which is worse than showing nothing, and live-only mode with a 3 s age limit removes that. Layout and alert priority follow the review's guidance on few elements and a stable hierarchy, with only immediate hazards interrupting the driving page. Contrast is the weakest of the five: the interface was restyled with dark ink on a light ground, a single tap colour and status words beside every colour mark (Appendix G), but it has not been measured in daylight. With two cameras the same rules decide what interrupts the driver. The banner names the side of each warning and puts an approaching object before a merely close one, and the Surroundings page shows one side at a time, so a second camera does not become a second competing display. The touch measurement shows that the panel is not a source of delay, so the slowness noticed comes from the buttons acting on release, which can be changed. Whether the display can be read within the 2 s guidance has not been measured; the character-size and touch-target figures of Section 2.6 give a check that can be applied before the on-vehicle observation.

6.6 Integration and power

The most consequential finding of the integration was not in any one subsystem. The telemetry unit's LTE modem, powered from the Raspberry Pi's USB port, pulled the supply into under-voltage whenever it transmitted, throttled the processor and slowed the detector, and on its USB port the Raspberry Pi 4 restarted three times in 20 minutes, against none in 4 hours after the unit was moved (Section 4.10.7). The modem worked correctly throughout; the fault was in how the parts were powered together. A separate supply for the telemetry unit is therefore a requirement for the vehicle installation. The same fault returned once the Raspberry Pi 4 was closed in its printed box with the unit still on its USB port: it brought heat to 76.9 °C, throttling, a brown-out restart and a rise in lost dashboard lines. So the enclosure, the supply and the cooling have to be designed together, which led to the vented lids.

Placement shaped the architecture as much as processing did. The battery-management link only held once the telemetry unit sat beside the pack, and that, with the Raspberry Pi 5's faster detection, produced the two-sided layout: front sensing beside the display, rear sensing and the telemetry unit beside the battery, joined by a cable. The power budget (Table 5.5) puts the added electronics at about 20 W, under 1 % of the motor's rated input, so the draw while driving is small. The real requirements are switching off with the vehicle and a converter for the pack's 60 to 88 V. Only the Raspberry Pi 5's share is measured; the rest are estimates until each unit's current is read.

6.7 Validity and limitations

Almost all results come from one set of hardware on the bench, with one road session, without the vehicle's vibration, heat, supply and lighting. Several parameters are still defaults or uncalibrated: the camera's focal length, the time-of-flight directions, the accelerometer range and filter, and the GNSS speed units. The pothole score is on validation tiles. These limits do not invalidate the bench results, which measure what they claim to measure, but they bound what can be concluded from them.

Chapter 7

Conclusions

This project set out to build a low-cost edge-processing telemetry unit for the TUKZIE Rev 0 electric cargo trike and to present its results safely to the driver on the vehicle's existing dashboard. Each research question is answered here from the evidence in Chapter 5, separating what has been shown from what depends on the vehicle.

RQ1: accuracy and repeatability of the dynamic-response measurement. The measurement chain is sound on the bench: two IMUs are sampled at a fixed 200 Hz with no lost samples, including with every other subsystem running; each is calibrated against gravity, which brought both to 9.80 to 9.81 m/s^2 immediately after calibration, in the orientation measured; and the features and the dual-sensor fusion are computed correctly. On the first road session (19 October) no samples were dropped and the rate held except for one stall of about 2 s, but a third of the per-second records were lost on the USB link to the recorder; the per-second vibration value separated driving from parked clearly. How repeatably it characterises the trike's response over repeated traversals has not been measured, because that session recorded neither GNSS speed (no antenna was fitted) nor raw samples and its runs were not marked. RQ1 is answered for the instrument and for acquisition on the vehicle, but not yet for repeatability.

RQ2: presenting safety-critical information on the existing dashboard. A sparse presentation was implemented on the vehicle's own dashboard: a camera page with the live view and active alerts, a Sensors page with distances and fused hazards, and a banner over every page for an immediate hazard, all showing only live data and marking missing or out-of-date data as absent. With the second camera, one page switches between front and rear, and the banner labels each warning by side and puts approaching objects first. The touch panel adds no measurable delay, but the time from an object's appearance to the screen has not been measured. Whether a driver can read this within the 2 s glance guidance has not been measured, so RQ2 is answered for the information hierarchy and update strategy but not for glance time.

RQ3: latency, data loss and recovery. On the bench, acquisition loses no samples; the only recurring pause, from writing the log to flash, is bounded to one 33 to 44 ms gap a minute; the camera produces a result a median of about 120 ms after the sensor is read with the whole system running on the Raspberry Pi 4, and 51 to 54 ms with detection moved beside the display on the Raspberry Pi 5 (53 to 102 ms front and 126 to 151 ms rear on the vehicle); every publish succeeded; the unit reconnects about 5 s after

coverage returns; the local log is recovered completely; and after a 10-minute outage the backlog was replayed and every logged record stored once in the database. RQ3 is answered on the bench except for synchronisation latency, which needs GNSS time. On the vehicle the USB link to the recorder lost a third of the per-second records, which led to the wired serial link; the same measures on the moving vehicle remain.

RQ4: features that separate road from powertrain vibration. The time-domain features are implemented and verified, and the method for separating the two sources, using the motor's Hall pulses as a speed reference, is designed. The road session gave vehicle data, but without a speed reference (no GNSS antenna was fitted and there is no Hall tap) or raw samples it cannot separate the two sources, so RQ4 is not answered.

[Outstanding: revise each answer after the second road session (GNSS antenna, raw samples, run markers and a speed reference).]

Overall. The project delivers an integrated, low-cost architecture for a three-wheeled electric vehicle: fixed-rate dual-sensor acquisition with measured timing and loss, on-board ride features, camera detection fused with time-of-flight ranging, cellular telemetry with local logging, automatic reconnection and replay into a database, and presentation on the vehicle's existing dashboard, with front sensing beside the display and rear sensing beside the battery, and a warning for objects closing on the vehicle from either end. Each part has been measured on the bench and the whole system has run on the vehicle for one session, and integrating them exposed faults that no subsystem test showed, the most important being the under-voltage caused by powering the LTE modem from the Raspberry Pi, which recurred, with overheating, once the computer was enclosed. The work has reached a prototype that has run once on the vehicle, short of the demonstration in a relevant environment set as the target in Chapter 1; the on-vehicle sessions and the calibrations listed in Chapter 6 are what remain, and Chapter 8 sets out the work in order of priority.

Chapter 8

Recommendations

The recommendations come in four parts: the next steps for this system, in order of priority (Section 8.1); a staged path from this warning system towards autonomous operation, which is the programme's stated aim (Section 8.2); the wider value of the work beyond this vehicle (Section 8.3); and what the next person needs to carry it on (Section 8.4).

8.1 Next steps for this system

These are in order of priority, the first being those on which the validity of the vehicle data depends.

1. Acquisition integrity. Remove the flash-write pause rather than reduce it: move logging off the acquisition processor (to the Raspberry Pi or separate storage), or place the acquisition code in instruction RAM, or read the IMUs through their hardware FIFOs so that a pause is absorbed. The 94 Hz filter of v0.7.4 kept 200 Hz with no drops in T15; check it against the vehicle's measured vibration, or sample faster and decimate in software, compute jerk from the measured sample times, and flag samples at the $\pm 8\ g$ limit so that clipped windows are marked. Set the range from the peaks recorded on the vehicle.

2. Calibration. Recalibrate both IMUs in their mounted position before every measurement campaign, and replace the single magnitude factor with a multi-position per-axis calibration so that the result no longer depends on orientation. Calibrate the camera's focal length, the fusion's sensor directions and each time-of-flight sensor against a tape (test C9), confirm the GNSS speed units against a known speed, and check the BMS voltage against a meter.

3. Power. Give the telemetry unit its own regulated supply from the vehicle, never a Raspberry Pi's USB port; give the Raspberry Pi 4 a 5.1 V, 3 A supply and cooling; and measure the current of each unit to produce the power budget against the 72 V pack that the brief asks for.

4. Synchronisation. Replace the jumper on the sync line with a locking connector; drive an LED from the sync pin within the camera's view to measure the frame-to-edge offset optically; and record the mapping of frames to sync edges in every session so that video and inertial events can be aligned after the fact.

5. Synchronisation to the database. Sequence numbers, replay and the database are in place (T15). Measure end-to-end latency with GNSS time on the road test; rotate rather than truncate the log; and move from the public test broker to an authenticated, encrypted one that queues records for a persistent subscriber, so that completeness no longer depends on reading the log back.

6. Display failure behaviour. Extend the live-only rule to every field with its own age (for example, mark the battery state as stale when the BMS stops reporting); make the navigation buttons act on press; measure alert onset to screen on the Raspberry Pi 5 and the legibility of each page in daylight against the ISO 15008 character-size figure; and carry out a glance observation in a controlled low-risk trial.

7. Ride analysis. Wire the Hall tap, set the pole-pair count, record the stationary powertrain baseline, add a spectral feature (band energy or power spectral density) and orientation compensation, and then test which features separate road from powertrain vibration (RQ4).

8. Approaching objects. Run the time-to-contact accuracy test (C11) before relying on the warning: a person walking towards each camera at a measured pace between tape marks, and a car approaching the parked trike in a closed area. Then set the 2 and 4 s thresholds from the measured error and the vehicle's stopping distance.

9. Accelerator. The Raspberry Pi 5 carries an AI HAT+ that the detector does not yet use. Moving inference to it would free the processor beside the dashboard and allow a larger model; measure the gain before relying on it.

10. Enclosures. Measure the temperature of both computers in the vented boxes over a full session, and the share of lost dashboard lines with the telemetry unit on its own supply.

11. Obstacle detection. The detector recognises only its trained classes and cannot report unexpected debris; a method from the road-obstacle and anomaly literature (Section 2.8) would have to be measured on the Raspberry Pi before use. Until then the time-of-flight sensors' unclassified obstacles cover close range, and the pothole model should be validated on its held-out test set and on the vehicle's camera before it raises any alert.

8.2 Towards autonomous operation

The TUKZIE Flagship Programme aims at an autonomy-ready platform (Chapter 1). By the SAE taxonomy [61], the system built here is at level 0: it warns the driver and controls nothing. That is the right starting point, because the same parts an automated vehicle needs are already present in a first form: perception from two cameras with ranging at both ends, a time-to-contact estimate, inertial and battery state, time-stamped logging, on-board processors and a wired network between them. The steps below would take it further, each one building the evidence the next needs. None should be skipped, and none involves controlling the vehicle until the last.

1. Validated driver assistance (still level 0). Measure what the warnings are worth before building on them: label a sample of alerts against video of a road session and report their precision and the rate of false alerts per kilometre, run the time-to-contact test (C11), and add a test that an approaching object is actually on a collision course, so that parked cars ahead do not raise warnings. Replace the 1.2 m stand-in sensors with longer-range ranging (the planned multi-zone sensor, then radar or lidar) so that warnings at road speed do not rest on the camera alone.

2. Knowing where the vehicle is and how it moves. Automated driving needs position, heading and speed it can trust. Add the wheel-speed sensor (Section 4.7), fuse it with the IMUs and GNSS into one estimate of position and motion in the vehicle's own axes, and synchronise the clocks of all units over the Ethernet link so that every sensor shares one time base.

3. A dataset from the vehicle. The detector was trained elsewhere and missed South African road features (Section 2.8). Recording synchronised camera, inertial, position and battery data on the TUKZIE's routes would give the programme training and test data for its own conditions. This project stores no frames by design, so it needs a revised ethics application covering images of public roads, with faces and number plates removed before storage.

4. Control in a closed area. Only then should any control be attempted, and first at low speed in a closed area such as a depot yard, which defines a narrow operating domain in the sense of the taxonomy [61]. The vacation-work teams' CAN bus design already carries throttle and brake messages for sensing; actuating them would need a drive-by-wire interface, an emergency stop independent of the computers, and a safety case built to the functional-safety standard for road vehicles [62] and to the standard on the safety of the intended function, which covers hazards from limited perception rather than from faults [63]. The split already used here, deterministic tasks on a microcontroller and perception on separate computers, is the pattern such a design would keep, with the control path on the deterministic side.

8.3 The bigger picture

The work matters beyond one vehicle in four ways.

- Fleet operation. The same record, from many vehicles, would let an operator follow battery health and use, spot faults from abnormal current or vibration, and plan maintenance from data instead of breakdowns. The store-and-forward link with its database (Section 4.9) is the part that scales to a fleet, once it moves to an authenticated broker.
- Roads. Vibration and pothole detections tagged with position, from vehicles that drive the same streets every day, would map road condition across a city at little extra cost. Other projects in the programme already publish to a shared broker, so the TUKZIE's data can join theirs.
- Vehicle design. Measured vibration, loads and battery current on real routes are the inputs the next revision of the vehicle needs for its chassis, suspension and pack sizing.
- Cost and reuse. The system is built from low-cost parts that can be bought locally and runs on open software. The architecture is not specific to this trike and could serve other light electric vehicles in the same cities.

Two cautions go with this. Data from vehicles on public roads includes their position and what their cameras see, so privacy has to be designed in from the start (Section 3.7), and none of the claims above can be made before the repeatability that RQ1 asks about has been shown on the road.

8.4 Handing the project on

To let the next student or the programme's team continue without starting again, keep with the repository: a short guide to its folders and services; the wiring tables (Appendix C); the deploy and set-up scripts and the order in which to run them (Section 4.12); the road-test checklist and run sheet; the list of known faults and their status; and where every recording is kept. Each recommendation above should be linked to the result in Chapter 5 that motivates it.

Bibliography

[1] United Nations, Department of Economic and Social Affairs, Population Division, *World Urbanization Prospects: The 2018 Revision*. New York, NY, USA: United Nations, 2019.

[2] Jiangsu Jinpeng Group Co., Ltd, *Electric Tricycle User Manual*. Vehicle documentation, TUKZIE Rev 0 platform.

[3] N. Adams and S. Rasara, "Jinpeng Electric Tricycle Drivetrain: Characterization & Modelling," TUKZIE vacation work report, University of Cape Town, Department of Electrical Engineering, Jul. 2026.

[4] World Health Organization, *Global Status Report on Road Safety 2023: Summary*. Geneva, Switzerland: World Health Organization, 2023.

[5] National Highway Traffic Safety Administration, *Visual-Manual NHTSA Driver Distraction Guidelines for In-Vehicle Electronic Devices*, Federal Register, vol. 78, no. 81, pp. 24818–24890, Apr. 2013.

[6] S. Winberg and S. Jayalath, "Development of an Embedded HPEC Telemetry Unit and Windshield HUD for the TUKZIE Rev 0 Platform," EEE4022S project brief, University of Cape Town, Department of Electrical Engineering, 2026.

[7] Y. Zhu, Y. Jing, M. Jiang, Z. Zhang, D. Wang, and W. Liu, "An experimental study of the cognitive load of in-vehicle multiscreen connected HUD," in *Design, User Experience, and Usability: Design for Contemporary Technological Environments*, Lecture Notes in Computer Science, vol. 12781. Cham, Switzerland: Springer, 2021, pp. 268–285, doi: 10.1007/978-3-030-78227-6_20.

[8] W. M. Kimmel, C. D. Weiss, and A. M. Mazzuchi, *Technology Readiness Assessment Best Practices Guide*, NASA/SP-20205003605, 2020.

[9] O. S. Sebwa, P. V. Chombo, G. B. Gerutu, R. O. Kivugo, J. H. Hosea, and K. A. Greyson, "Energy, economy and environmental assessment of e-tricycle for medium-size cargo deliveries: A case of Dar es Salaam City," *The Journal of Engineering*, vol. 2025, no. 1, Art. no. e70073, 2025, doi: 10.1049/tje2.70073.

[10] J. Saenz, M. Figliozzi, and J. Faulin, "Assessment of the carbon footprint reductions of tricycle logistics services," *Transportation Research Record*, vol. 2570, no. 1, pp. 48–56, 2016, doi: 10.3141/2570-06.

[11] Federal Highway Administration, *Measuring and Specifying Pavement Smoothness*, Tech. Brief FHWA-HIF-16-032, Washington, DC, USA, 2016.

[12] A. González, E. J. O'Brien, Y.-Y. Li, and K. Cashell, "The use of vehicle acceleration measurements to estimate road roughness," *Vehicle System Dynamics*, vol. 46, no. 6, pp. 483–499, 2008, doi: 10.1080/00423110701485050.

[13] S. Sattar, S. Li, and M. Chapman, "Road surface monitoring using smartphone sensors: A review," *Sensors*, vol. 18, no. 11, Art. no. 3845, 2018, doi: 10.3390/s18113845.

[14] Y. I. Alatoom and O. Smadi, "Embedded framework for low-cost pavement condition evaluation using microcontroller and single-board computer platforms," *Automation in Construction*, vol. 178, Art. no. 106442, 2025, doi: 10.1016/j.autcon.2025.106442.

[15] V.-H. Bui, D.-N. Tran, P. Q. Huy, N. V. Cong, and T.-H. Dao, "Signal processing-driven real-time rough road detection with GPS accuracy improvement on edge microcontrollers," in *Proc. IEEE 15th Int. Conf. Communication Systems and Network Technologies*, 2026, doi: 10.1109/CSNT69054.2026.11502079.

[16] B. Yiliguoqi, B. Siriguleng, Arong, G. Guo, and J. Wang, "Research on the evaluation and analysis of road surface roughness based on smartphone sensors and SVM," *Scientific Reports*, vol. 16, Art. no. 4409, 2026, doi: 10.1038/s41598-025-34396-3.

[17] Z. Li, W. Yu, and X. Cui, "Online Classification of Road Roughness Conditions with Vehicle Unsprung Mass Acceleration by Sliding Time Window," *Shock and Vibration*, vol. 2018, Art. no. 5131434, 2018, doi: 10.1155/2018/5131434.

[18] International Organization for Standardization, *Mechanical Vibration and Shock: Evaluation of Human Exposure to Whole-Body Vibration, Part 1: General Requirements*, ISO 2631-1:1997, including Amendment 1:2010.

[19] W. Shi, J. Cao, Q. Zhang, Y. Li, and L. Xu, "Edge computing: Vision and challenges," *IEEE Internet of Things Journal*, vol. 3, no. 5, pp. 637–646, 2016, doi: 10.1109/JIOT.2016.2579198.

[20] M. Barnell, C. Raymond, C. Capraro, D. Isereau, C. Cicotta, and N. Stokes, "High-performance computing (HPC) and machine learning demonstrated in flight using Agile Condor," in *Proc. IEEE High Performance Extreme Computing Conf.*, 2018, pp. 1–4, doi: 10.1109/HPEC.2018.8547797.

[21] A. Pérez-González, A. F. Villa-Salazar, I. N. Gomez-Miranda, J. D. Velásquez-Gómez, A. F. Romero-Maya, and A. Jaramillo-Duque, "An IoV-based real-time telemetry and monitoring system for electric racing vehicles: Design, implementation, and field validation," *Vehicles*, vol. 7, no. 4, Art. no. 128, 2025, doi: 10.3390/vehicles7040128.

[22] Y.-C. Liu, "Effects of using head-up display in automobile context on attention demand and driving performance," *Displays*, vol. 24, nos. 4–5, pp. 157–165, 2003, doi: 10.1016/j.displa.2004.01.001.

[23] M. Smith, J. L. Gabbard, and C. Conley, "Head-up vs. head-down displays: Examining traditional methods of display assessment while driving," in *Proc. 8th Int. Conf. Automotive User Interfaces and Interactive Vehicular Applications*, 2016, pp. 185–192, doi: 10.1145/3003715.3005419.

[24] A. Mareška, T. Kordová, and M. Havlík Míka, "Production of Head-Up Display windshield and its relation with the image quality," *Transactions on Transport Sciences*, vol. 13, no. 2, pp. 63–70, 2022, doi: 10.5507/tots.2022.011.

[25] Z. Qin, F.-C. Lin, Y.-P. Huang, and H.-P. D. Shieh, "Maximal acceptable ghost images for designing a legible windshield-type vehicle Head-Up Display," *IEEE Photonics Journal*, vol. 9, no. 6, pp. 1–12, 2017, doi: 10.1109/JPHOT.2017.2758820.

[26] R. Häuslschmid, B. Pfleging, F. Alt, and A. Butz, "A design space to support the development of windshield applications for the car," in *Proc. 2016 CHI Conf. Human Factors in Computing Systems*, 2016, pp. 5076–5091, doi: 10.1145/2858036.2858336.

[27] R. Häuslschmid, S. Osterwald, M. Lang, and A. Butz, "Augmenting the driver's view with peripheral information on a windshield display," in *Proc. 20th Int. Conf. Intelligent User Interfaces*, 2015, doi: 10.1145/2678025.2701393.

[28] K. Park and Y. Im, "Ergonomic guidelines of Head-Up Display user interface during semi-automated driving," *Electronics*, vol. 9, no. 4, Art. no. 611, 2020, doi: 10.3390/electronics9040611.

[29] B. Reimer, B. Mehler, J. Dobres, J. F. Coughlin, S. Matteson, D. Gould, N. Chahine, and V. Levantovsky, "Assessing the impact of typeface design in a text-rich automotive user interface," *Ergonomics*, vol. 57, no. 11, pp. 1643–1658, 2014, doi: 10.1080/00140139.2014.940000.

[30] F. You, Y.-F. Yang, M.-T. Fu, J. Zhang, and J.-M. Wang, "Design guidelines for the size and length of Chinese characters displayed in the intelligent vehicle's central console interface," *Information*, vol. 12, no. 5, p. 213, 2021, doi: 10.3390/info12050213.

[31] H. Kim, S. Kwon, J. Heo, H. Lee, and M. K. Chung, "The effect of touch-key size on the usability of in-vehicle information systems and driving safety during simulated driving," *Applied Ergonomics*, vol. 45, no. 3, pp. 379–388, 2014, doi: 10.1016/j.apergo.2013.05.006.

[32] Euro NCAP, *Safe Driving: Driver Engagement Protocol*, version 1.2. Leuven, Belgium: Euro NCAP, 2026.

[33] D. L. Strayer, J. M. Cooper, R. M. Goethe, M. M. McCarty, D. J. Getty, and F. Biondi, *Visual and Cognitive Demands of Using In-Vehicle Infotainment Systems*. Washington, DC, USA: AAA Foundation for Traffic Safety, 2017.

[34] W. Liu, D. Anguelov, D. Erhan, C. Szegedy, S. Reed, C.-Y. Fu, and A. C. Berg, "SSD: Single shot multibox detector," in *Computer Vision – ECCV 2016*, Lecture Notes in Computer Science, vol. 9905. Cham, Switzerland: Springer, 2016, pp. 21–37, doi: 10.1007/978-3-319-46448-0_2.

[35] A. G. Howard, M. Zhu, B. Chen, D. Kalenichenko, W. Wang, T. Weyand, M. Andreetto, and H. Adam, "MobileNets: Efficient convolutional neural networks for mobile vision applications," arXiv:1704.04861, 2017.

[36] B. Jacob, S. Kligys, B. Chen, M. Zhu, M. Tang, A. Howard, H. Adam, and D. Kalenichenko, "Quantization and training of neural networks for efficient integer-arithmetic-only inference," in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition*, 2018, pp. 2704–2713, doi: 10.1109/CVPR.2018.00286.

[37] T.-Y. Lin, M. Maire, S. Belongie, J. Hays, P. Perona, D. Ramanan, P. Dollár, and C. L. Zitnick, "Microsoft COCO: Common objects in context," in *Computer Vision – ECCV 2014*, Lecture Notes in Computer Science, vol. 8693. Cham, Switzerland: Springer, 2014, pp. 740–755, doi: 10.1007/978-3-319-10602-1_48.

[38] D. K. Alqahtani, M. A. Cheema, M. A. Rodriguez, and A. N. Toosi, "A comprehensive evaluation of deep learning object detection models on heterogeneous edge devices," arXiv:2409.16808v3, Jul. 2026.

[39] J. Zhu and Y. Fang, "Learning object-specific distance from a monocular image," in *Proc. IEEE/CVF Int. Conf. Computer Vision*, 2019, pp. 3838–3847, doi: 10.1109/ICCV.2019.00394.

[40] T. E. Choe, J. Wu, X. Lin, K. Kwon, and M. Park, "HazardNet: Road Debris Detection by Augmentation of Synthetic Models," arXiv:2303.07547, 2023.

[41] R. Chan, K. Lis, S. Uhlemeyer, H. Blum, S. Honari, R. Siegwart, P. Fua, M. Salzmann, and M. Rottmann, "SegmentMeIfYouCan: A Benchmark for Anomaly Segmentation," in *Proc. Neural Information Processing Systems (NeurIPS) Track on Datasets and Benchmarks*, 2021.

[42] A. Bint-e Tanveer, M. A. Kamal, M. M. Alam, and M. Mohd Su'ud, "Real-time detection of rare roadside obstacles using YOLOv8-n in autonomous vehicles," *PLOS ONE*, 2026, doi: 10.1371/journal.pone.0350732.

[43] M. De Simone, Z. B. Rivera Chavez, and D. Guida, "Obstacle Avoidance System for Unmanned Ground Vehicles by Using Ultrasonic Sensors," *Machines*, vol. 6, no. 2, Art. no. 18, 2018, doi: 10.3390/machines6020018.

[44] N. Noomwongs, K. Kiataramgul, S. Chantranuwathana, and G. Phanomchoeng, "GNSS-RTK-Based Navigation with Real-Time Obstacle Avoidance for Low-Speed Micro Electric Vehicles," *Machines*, vol. 13, no. 6, Art. no. 471, 2025, doi: 10.3390/machines13060471.

[45] G. Caroleo, A. Albini, and P. Maiolino, "On the Characterisation of the Time-of-Flight VL53L5CX Sensor by STMicroelectronics for Indoor Robotics Applications," *Sensors*, vol. 26, no. 5, Art. no. 1639, 2026, doi: 10.3390/s26051639.

[46] Makerfabs, "ESP32S3 4G LTE CAT1 A7670X," product page and hardware repository. [Online]. Available: `https://www.makerfabs.com/esp32s3-4g-lte.html` and `https://github.com/Makerfabs/ESP32-S3-4G-LTE-CAT1-A7670X` (accessed 24 Sep. 2026).

[47] SIMCom Wireless Solutions, *A76XX Series AT Command Manual*, V1.09, Apr. 2023. [Online]. Available: `https://files.waveshare.com/wiki/A7670E-Cat-1-GNSS-HAT/A76XX_Series_AT_Command_Manual_V1.09.pdf` (accessed 24 Sep. 2026).

[48] Sensirion AG, *Datasheet SEN5x: Environmental Sensor Node for HVAC and Air Quality Applications*, version 2, Stäfa, Switzerland, Mar. 2022.

[49] Raspberry Pi Ltd, *The Picamera2 Library*, Sep. 2026. [Online]. Available: `https://datasheets.raspberrypi.com/camera/picamera2-manual.pdf` (accessed 28 Sep. 2026).

[50] Google, "LiteRT," Google AI Edge documentation. [Online]. Available: `https://ai.google.dev/edge/litert`

[51] G. Jocher and J. Qiu, *Ultralytics YOLO11*, version 11.0.0, software, 2024. [Online]. Available: `https://github.com/ultralytics/ultralytics`

[52] gerapothole, "pothole-detection-yolov8," dataset, version 1, Roboflow Universe. [Online]. Available: `https://universe.roboflow.com/gerapothole/pothole-detection-yolov8` (accessed 2 Oct. 2026).

[53] S. Nienaber, M. J. Booysen, and R. S. Kroon, "Detecting potholes using simple image processing techniques and real-world footage," in *Proc. 34th Southern African Transport Conf.*, Pretoria, South Africa, 2015. Image dataset redistributed as "Road pothole images for pothole detection," Kaggle (sovitrath). [Online]. Available: `https://www.kaggle.com/datasets/sovitrath/road-pothole-images-for-pothole-detection`

[54] Tencent, *ncnn: A High-Performance Neural Network Inference Framework Optimized for the Mobile Platform*, software. [Online]. Available: `https://github.com/Tencent/ncnn`

[55] OpenStreetMap contributors, *OpenStreetMap*, map data available under the Open Database License. [Online]. Available: `https://www.openstreetmap.org`

[56] D. Luxen and C. Vetter, "Real-time routing with OpenStreetMap data," in *Proc. 19th ACM SIGSPATIAL Int. Conf. Advances in Geographic Information Systems*, 2011, pp. 513–516, doi: 10.1145/2093973.2094062.

[57] Raspberry Pi Ltd, "Raspberry Pi hardware: GPIO and the 40-pin header," Raspberry Pi documentation. [Online]. Available: `https://www.raspberrypi.com/documentation/computers/raspberry-pi.html`

[58] D. N. Lee, "A theory of visual control of braking based on information about time-to-collision," *Perception*, vol. 5, no. 4, pp. 437–459, 1976, doi: 10.1068/p050437.

[59] E. Dagan, O. Mano, G. P. Stein, and A. Shashua, "Forward collision warning with a single camera," in *Proc. IEEE Intelligent Vehicles Symposium*, 2004, pp. 37–42, doi: 10.1109/IVS.2004.1336352.

[60] P. K. Sen, "Estimates of the regression coefficient based on Kendall's tau," *Journal of the American Statistical Association*, vol. 63, no. 324, pp. 1379–1389, 1968, doi: 10.1080/01621459.1968.10480934.

[61] SAE International, *Taxonomy and Definitions for Terms Related to Driving Automation Systems for On-Road Motor Vehicles*, SAE J3016, Apr. 2021.

[62] International Organization for Standardization, *Road Vehicles: Functional Safety*, ISO 26262:2018.

[63] International Organization for Standardization, *Road Vehicles: Safety of the Intended Functionality*, ISO 21448:2022.

Appendix A

Graduate Attributes, AI Use and Project Repository

A.1 Graduate attribute evidence

This section summarises how the work reported here meets the six graduate attributes assessed in this course, with the sections that hold the evidence.

GA1: Problem solving

The research questions were framed around measurable vehicle dynamic response rather than subjective ride comfort, and each was tied to a requirement and a verification method (Section 1.4). The design choices were reasoned from first principles: a 200 Hz sampling rate from the ISO 2631-1 band and the sampling theorem, drift-free task timing and non-blocking bounded queues so that acquisition is never held up (Section 4.3), the acceleration magnitude as an orientation-tolerant signal (Section 4.4), and speed normalisation after the literature showed that raw RMS mixes road roughness with speed (Section 4.4.2). Faults were root-caused from evidence rather than worked around, and the causes changed the design (Section 4.10, Table 4.2): a flash-configuration boot loop isolated one setting at a time (Section 4.10.1); the Raspberry Pi 4 under-voltage traced to the modem's supply path, which gave the telemetry unit its own supply (Section 4.10.7); and a time-of-flight start-up in which one failed sensor stopped all three, redesigned to skip, time out and re-probe (Section 4.10.6). Others were a sync-pin choice checked against the board schematic (Section 4.8.1), a serial-routing fault (Section E.3) and the flash-write acquisition stall (Section 4.9).

GA4: Investigations, experiments and data analysis

Assumptions were treated as hypotheses and tested on the real hardware, with a test chosen to tell the candidate causes apart (Section 4.10): the GNSS reply format and the modem's start-up, separating warm-up from a modem switched off at boot (Section 4.10.2); the BMS service layout and checksum

(Section 4.10.3); invalid records, separated from transfer damage by an indexed read-back (Section 4.10.4); the camera's colour cast, one test per hypothesis (Section 4.10.5); the Pi 4 under-voltage, matched to the device that caused it by a 2 s power-flag log (Section 4.10.7); and the periodic acquisition stall, confirmed by a controlled comparison with local logging on and off (Section 4.9). The feature computation was validated against a synthetic signal with known answers, the two IMUs were calibrated against gravity and checked under disturbance (Section 4.4.1), the dual-IMU fusion was verified in a recorded session (Section 4.4.3), and the camera-to-IMU timing offset was bounded (Section 4.8.1). The raw evidence is indexed in Appendix D.

GA5: Use of engineering tools

PlatformIO, the Arduino framework and FreeRTOS were used for the firmware, with esptool and serial diagnostics for hardware verification (Appendix E). Python tools were written to support testing: scripted bench tests, a browser bench console that parses and replays the telemetry, and schematic checks (Section 4.11). On the camera side, LiteRT and Picamera2 run the detector, and per-stage latency is measured from the sensor's own frame timestamps (Section 4.8.2). A parametric bracket was modelled in OpenSCAD and a camera casing was designed and 3D printed (Section 4.2). The limits of each tool were checked rather than assumed, for example a BMS checksum formula that no vendor document confirmed and that proved wrong on the vehicle (Section 4.5).

GA6: Professional and technical communication

This report documents the engineering work, including the faults found and how they were resolved. The written work was developed with the supervisor through regular meetings and a draft Introduction and Methodology, and inconsistencies between chapters were corrected as the work changed. The raw evidence behind each result is kept and indexed (Appendix D), and the firmware, tools and planning documents are in the project repository.

GA8: Individual work

The hardware bring-up, firmware, bench tools and report were done individually, with the supervisor consulted on specific questions. The components were selected, compared against the requirements and ordered by the author, for example the 8×8-zone time-of-flight sensor chosen over a single-point one to separate left, centre and right (Section 4.8.3); each component was tested on the bench before being relied on; and the camera casing was designed and 3D printed by the author (Section 4.2). Inherited work is identified where it is used: the vehicle dashboard and its team, and a teammate's BMS parser used only as an independent cross-check (Section 4.5). The use of AI tools is declared in Section A.3.

GA9: Independent learning ability

The work required learning FreeRTOS timing, BLE GATT, the A76XX AT command set and on-device inference on the Raspberry Pi, largely from primary sources: manufacturer schematics and manuals, and

live tests on the hardware. Assumptions were repeatedly corrected against those sources, for example GPIO assignments against the schematic and the MQTT result handling against the modem manual (Section 4.9). A continuously running literature search was kept up so that the research-gap claims were tested rather than asserted once, and it narrowed two of them (Chapter 2).

A.2 Project repository

Firmware, camera software, planning documents and supporting tools for this project are available at `https://github.com/0geder/Tukzie-HPEC-HUD`.

A.3 Declaration of Artificial Intelligence Use

Artificial intelligence tools were used to assist with clarifying the organisation of this report, improving the alignment and consistency of its presentation, refining wording, and checking LaTeX formatting. All AI-assisted suggestions were reviewed by the author. The author remains responsible for the report's technical content, analysis, interpretations, and final editorial decisions.

Appendix B

Supporting Figures

The figures below support the text of Chapters 1 and 3, where each is referenced. The parametric bracket of Figure B.4 was modelled first but not printed; the printed casing of Figure B.5 was used instead. The casing has a bottom shell (29.2 by 28.2 by 7.6 mm) and a top cover (4.8 mm) that enclose the camera module and its ribbon connection, and a separate 30-degree corner mount (32.3 mm tall) by which it is to be attached to the vehicle. The bracket's lens cavity was first cut through the whole mounting face, leaving no lip to retain the module; this was visible only in a full render, not in the preview, and was corrected before the design was set aside.



Figure B.1: Manufacturer identification plate on the TUKZIE Rev 0 platform, photographed directly during vehicle inspection.

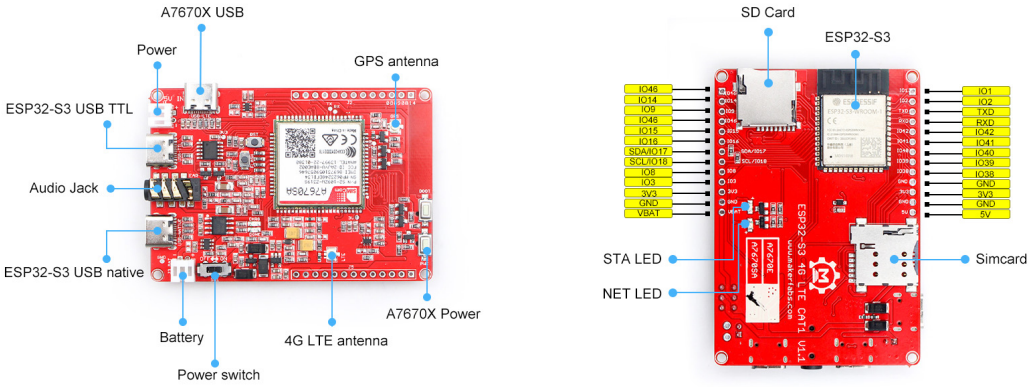


Figure B.2: Makerfabs ESP32-S3 4G LTE CAT1 A7670X board: manufacturer-labelled ports (left) and pinout (right) [46].

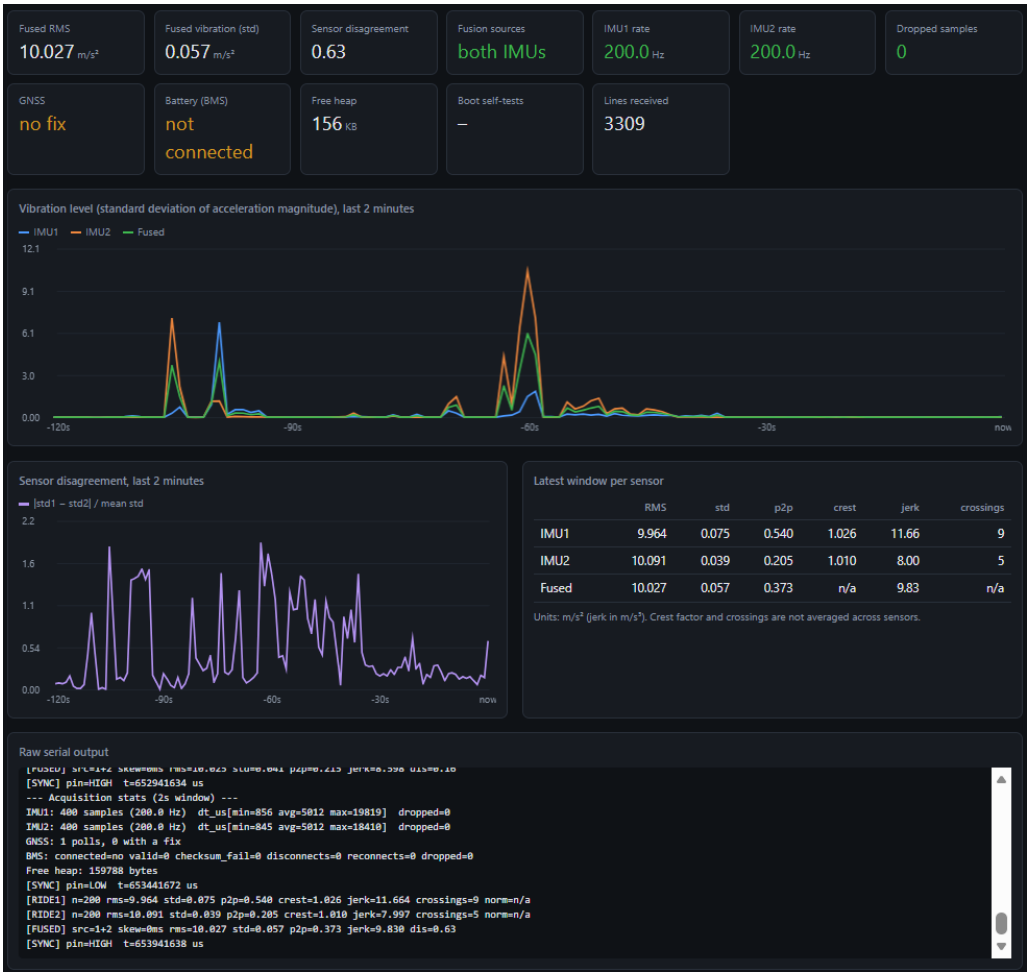


Figure B.3: The live telemetry dashboard connected to the telemetry unit over USB during bench testing, showing both IMUs fused at 200 Hz with no dropped samples and two hand disturbances in the vibration plot.

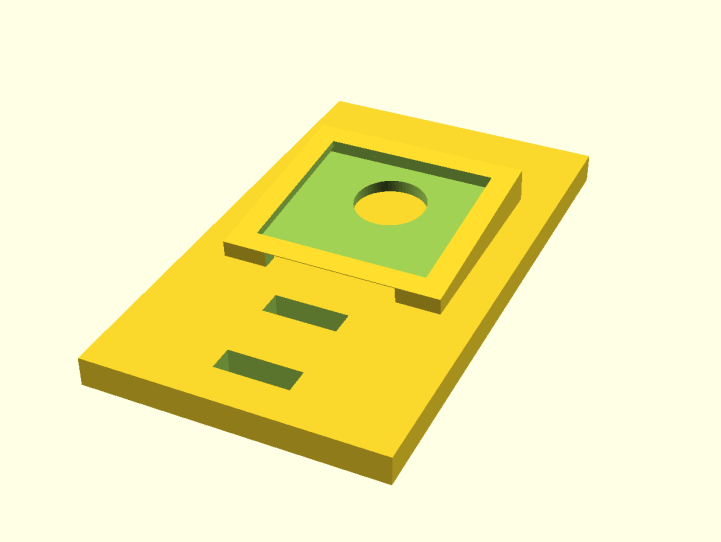


Figure B.4: Initial parametric camera bracket (OpenSCAD, full render), showing the corrected lens-retention lip and cable-routing slots. This design was not printed.

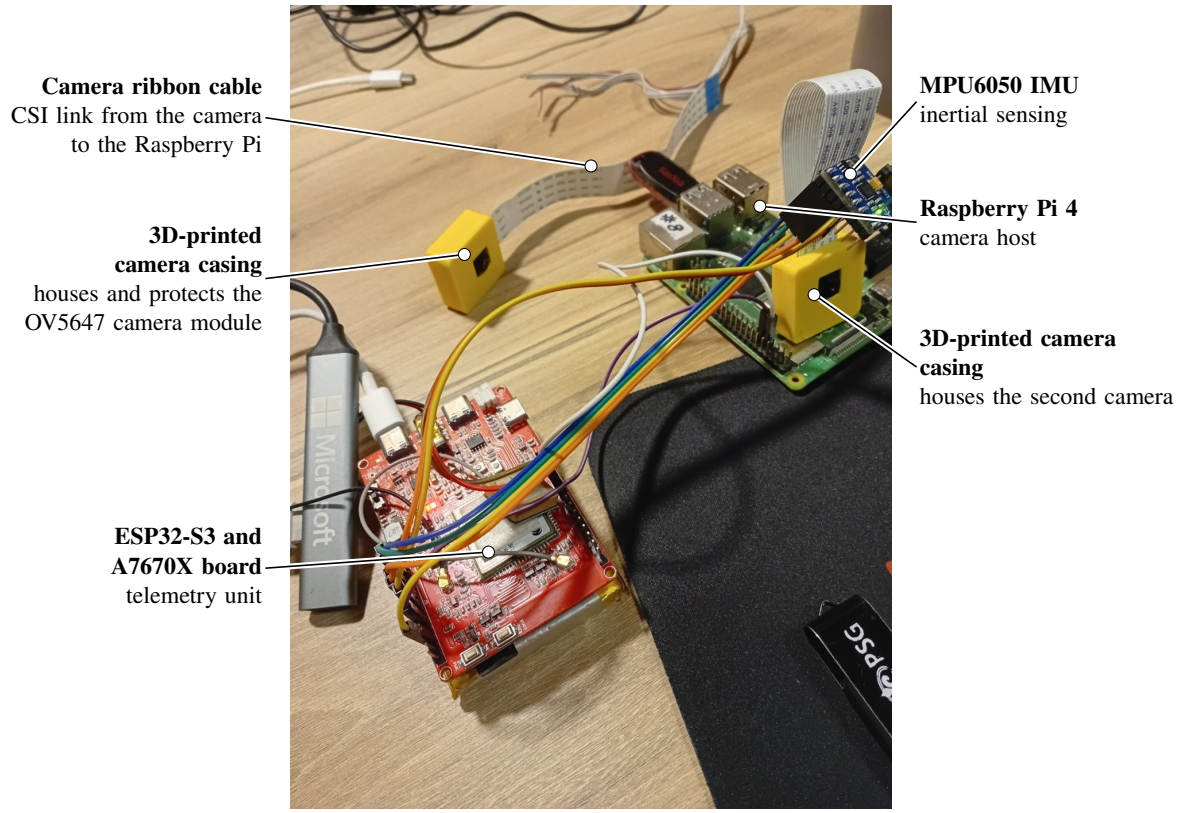


Figure B.5: The two cameras in their 3D-printed casings on the bench, before the two-sided layout, when the Raspberry Pi 4 was the camera host. In the final layout one camera is on the Raspberry Pi 4 at the rear and the other on the Raspberry Pi 5 at the front (Section 4.8.6).

Appendix C

Wiring and Connections

This appendix records how every part of the system is connected, so that it can be rebuilt or checked from this document alone. Each table gives both ends of each wire by connector, pin number and signal. The pin assignments come from the firmware (`TelemetryUnit/src/main.cpp`), the time-of-flight reader (`CameraDetection/tof_reader.py`) and the bench procedures (`BenchTest/TEST_PROCEDURES.md`), and the pin positions from the manufacturers' documents reproduced in the figures. Connections not yet made on the hardware are marked as planned. Tables C.1 to C.6 give, in turn, the Raspberry Pi 4 to the time-of-flight board (Table C.1), the telemetry unit to the Raspberry Pi 4 (Table C.2), the IMUs (Table C.3), the telemetry unit's other connections (Table C.4), the planned SEN55 (Table C.5) and the camera, displays and power (Table C.6).

C.1 Pin references

The Makerfabs ESP32-S3 A7670X board's headers are shown in Figure B.2 (Appendix B). In this appendix its left header is called J5 and its right header J2, as on the board's schematic, with pins numbered from the top of the pinout figure: J5 runs IO46, IO14, IO9, IO46, IO15, IO16, IO17, IO18, IO8, IO3, 3V3, GND, VBAT (pins 1 to 13). The firmware comments confirm pins 5, 7 and 8 against the schematic; the others are read from the figure.

C.2 Raspberry Pi 4 to the time-of-flight sensor board

The three VL53L0X sensors share one I^2C bus and all start at address 0x29. Each sensor's shutdown line (XSHUT) goes to its own Pi GPIO, so that the reader can hold all three off, wake one at a time and move each to its own address (0x30, 0x31, 0x32). The sensors have 10 kΩ pull-ups on XSHUT and no I^2C pull-ups; the Pi's own 1.8 kΩ pull-ups on pins 3 and 5 are used.

If a cooling fan uses Pi pins 4 and 6, the sensor ground goes to pin 14. The sensors are 3.3 V parts and are never connected to the 5 V pins (2 and 4). If one XSHUT wire is disconnected, that sensor's pull-up keeps it on at 0x29, where it collides with each sensor being woken, and none of the three starts; the

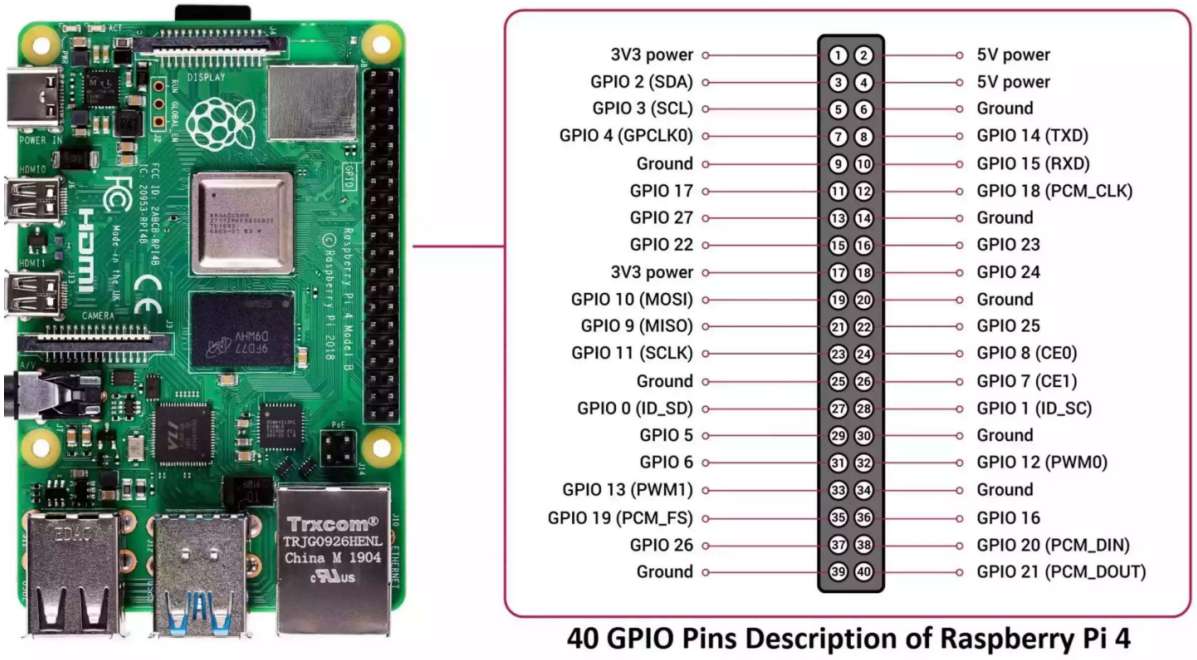


Figure C.1: Raspberry Pi 4 40-pin header, physical pin numbers and functions, from the project's reference sheet (`References/Hardware/Rasp Pi 4 pins.pdf`) [57]. Odd-numbered pins are on the inner row and even-numbered pins on the board edge.

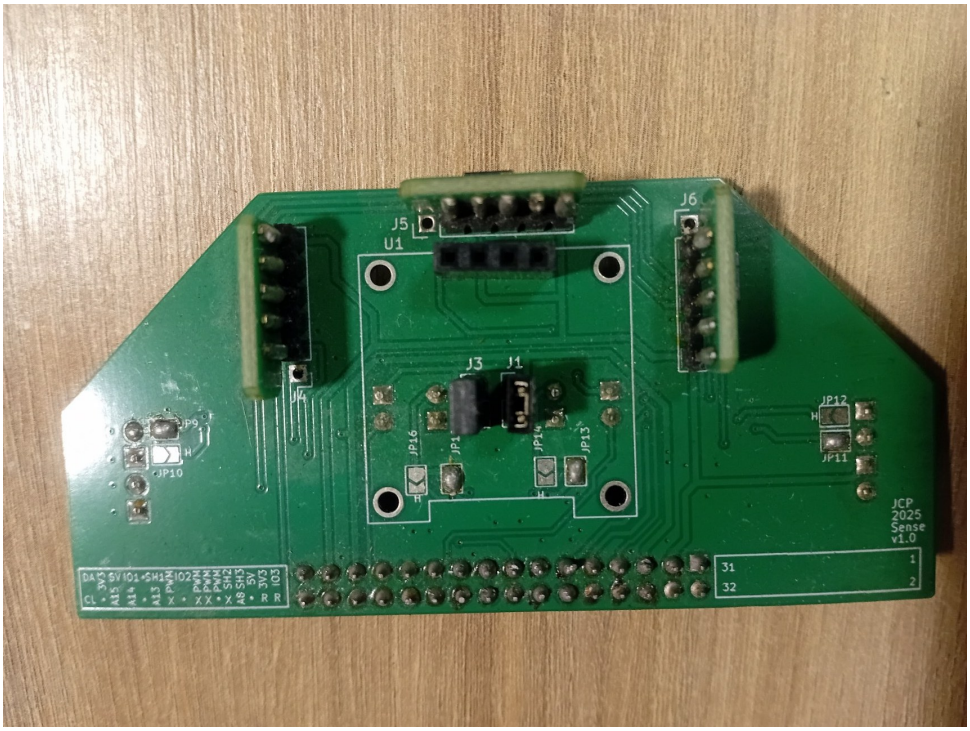


Figure C.2: Sensor board (UCT micromouse "JCP 2025 Sense v1.0") used as the time-of-flight stand-in. The 2×16 header J2 runs along the top edge; the silkscreen marks pins 1 and 2 at one end and 31 and 32 at the other, and the legend at the right names the signals (3V3, DA, CL, SH1 to SH3, 5V). The three VL53L0X sensor carriers sit on J4, J5 and J6, facing left, ahead and right.

Part	Connector
Sensor side	ACES 51451-0060N-001
Cable side	Compatible with ACES 51452-006H0H0-001 (e.g., JST GHR-06V-S)

Table 10: SEN5x connector

Pin	Name	Description	Comments
1	VDD	Supply voltage	5V ± 10%
2	GND	Ground	
3	SDA	Serial data input / output	LVTTL 3.3V compatible
4	SCL	Serial clock input	LVTTL 3.3V compatible
5	SEL	Interface select	Connect to GND
6	NC	Do not connect	

Table 11: SEN5x pin assignment.

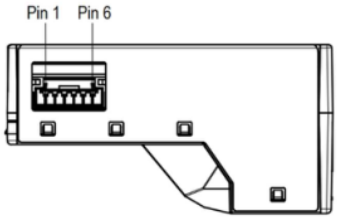


Figure 4 Pin layout. The communication interface connector (ACES 51451-0060N-001) is located at the side of the sensor adjacent to the air outlet.

Figure C.3: Sensirion SEN5x interface connector and pin assignment, reproduced from the datasheet [48]. The mating plug is a JST GHR-06V-S (1.25 mm pitch, six way).

Table C.1: Raspberry Pi 4 to sensor board J2 (Figures C.1 and C.2).

Signal	**Sensor board J2**	**Pi 4 pin**	**Pi 4 function**
3.3 V supply	pin 3 (3V3)	1	3V3 power
Ground	pin 23 (GND)	6 (or 14)	Ground
I^2C data	pin 31 (DA)	3	GPIO2, SDA
I^2C clock	pin 32 (CL)	5	GPIO3, SCL
XSHUT, left sensor	pin 21 (SH1)	15	GPIO22
XSHUT, ahead sensor	pin 9 (SH2)	16	GPIO23
XSHUT, right sensor	pin 7 (SH3)	18	GPIO24
LED supply (VDD)	not connected		

reader then logs each sensor as not answering after a 0.5 s timeout.

C.3 Telemetry unit (ESP32-S3) to the Raspberry Pi 4

Table C.2: ESP32-S3 board to Raspberry Pi 4.

Link	**ESP32-S3 board**	**Raspberry Pi 4**
Data and 5 V (telemetry lines, `/dev/ttyACM0`)	"ESP32-S3 USB native" USB-C port	any USB port, through a powered USB hub
Time-sync pulse (3.3 V logic)	J5 pin 5, GPIO15	pin 11, GPIO17
Ground for the pulse	J5 pin 12, GND	pin 9, Ground

The powered hub is required: with the board powered from the Pi's own USB port, the LTE modem's transmit current caused repeated under-voltage on the Raspberry Pi 4 (Section 4.8.4 and Chapter 4). The two boards' 3.3 V logic levels match, so the pulse wire connects directly.

C.4 Telemetry unit sensors and antennas

Table C.3: ESP32-S3 board to the two MPU6050 inertial sensors. Each sensor has its own I^2C bus, so both use address 0x68 (AD0 left low).

MPU6050 pin	**IMU1 (bus 0)**	**IMU2 (bus 1)**
VCC	J5 pin 11, 3V3	J5 pin 11, 3V3
GND	J5 pin 12, GND	J5 pin 12, GND
SDA	J5 pin 9, GPIO8	J5 pin 7, GPIO17
SCL	J5 pin 3, GPIO9	J5 pin 8, GPIO18
AD0, INT, XDA, XCL	not connected	not connected

Table C.4: Other telemetry unit connections.

Part	**Connection on the ESP32-S3 board**	**Notes**
LTE antenna	"4G LTE antenna" u.FL socket	away from metal and from the GNSS antenna
GNSS antenna	"GPS antenna" u.FL socket	ceramic face up, clear view of the sky
SIM card	SIM holder on the underside	
A7670X modem	on board: UART RX GPIO47, TX GPIO48, PWRKEY GPIO4, RESET GPIO5	no external wiring
Battery management system	Bluetooth Low Energy, no wire	board within about 1 m of the pack

Table C.5: Sensirion SEN55 to the ESP32-S3 board (planned, not yet connected or tested; Figure C.3). The sensor shares IMU2's bus; its address, 0x69, does not clash with the MPU6050 at 0x68.

SEN55 pin	**Signal**	**ESP32-S3 board**
1	VDD, 5 V ±10%	J2 pin 13, 5V
2	GND	J2 pin 12, GND
3	SDA (3.3 V compatible)	J5 pin 7, GPIO17
4	SCL (3.3 V compatible)	J5 pin 8, GPIO18
5	SEL	GND (selects I^2C)
6	NC	not connected

C.5 Camera, displays and power

Table C.6: Remaining connections.

Part	Connection	Notes
Rear camera (OV5647 NoIR)	15 cm ribbon to the Pi 4 "CAMERA" connector	contacts face the HDMI ports; longer ribbons pick up noise
Raspberry Pi 4 power	USB-C, 5.1 V 3 A supply	a weaker supply or a long thin cable gives under-voltage
Raspberry Pi 4 cooling	fan on 5 V and GND	needed: 81 °C without it, 48 °C with it (10 Oct 2026)
Front camera (second OV5647)	ribbon to a camera connector on the Pi 5	since 15 October
Raspberry Pi 5 (dashboard, front unit)	the vehicle's existing dashboard computer and display; front camera, detector and time-of-flight board added on 15 October	reaches the Pi 4 over the Ethernet cable
Network	dedicated Ethernet cable between the two Pis, fixed addresses 10.20.0.1 and 10.20.0.2; Wi-Fi only for the internet route. Until 15 October both Pis were on one 2.4 GHz hotspot	

C.6 Wiring checks

Before power is applied: every board is switched off and unplugged; each wire is checked end to end with a multimeter on continuity; 3.3 V and ground are checked not to be shorted; and each bundle is tied and fixed close to the board, so that a pull on the cable is taken by the tie and not by the header pins, since loose jumpers came off twice during bench testing. After power-up, the pre-test check (`BenchTest/sw7_check.py`) reports each link: telemetry from the ESP32, the three time-of-flight readings, the camera rate and the Raspberry Pi 4's power and temperature flags.

Appendix D

Test Evidence

This appendix indexes the raw evidence behind the bench results reported in Chapter 5. Every item is the unedited output of the test it names and is kept in the project repository, under `Report/evidence/` or `BenchTest/logs/`; Table D.1 gives each file. Figure B.3 in Appendix B and Figure 5.2 in Chapter 5 belong to the same record.

Table D.1: Index of test evidence.

Date	Test	Result shown	File
25 Sep 2026	Recorded bench session, telemetry unit, 150 s from power-up	200 Hz on both IMUs with no dropped samples; 149 fused windows, all from both sensors; five of five fusion self-tests passed; acquisition gaps of 25 to 30 ms every 12 s, with intervals down to 0.8 ms after each (catch-up)	`BenchTest/logs/2026-09-25_bench_fusion_shake.log`
25 Sep 2026	The same log replayed through the bench console (Figure D.1)	Every subsystem's state at the end of the session, and the vibration and disagreement traces for the whole recording	`Report/evidence/2026-09-25_bench_console_replay.png`
28 Sep 2026	Hazard detector, first run on live camera frames	430 frames in 60.1 s (7.2 frames/s); a person in view logged as "motorcycle", which exposed the label-offset fault	`Report/evidence/2026-09-28_camera_run1_label_bug.txt`
28 Sep 2026	Hazard detector after the label and colour fixes	419 frames in 60.1 s (7.0 frames/s); person detected at confidence 0.50 to 0.73; 16 alert activations in 47 s for one person in view, which led to clear-side hysteresis	`Report/evidence/2026-09-28_camera_run2_after_fixes.txt`

Date	Test	Result shown	File
29 Sep 2026	Firmware v0.5.0 on the telemetry unit: commands, flash-write stall test, MQTT reconnect	With logging on, 33 to 40 ms gaps on both IMUs every 10 s; with logging off, worst gap 5.9 ms; no dropped samples; 47 MQTT publishes with no failures; reconnect in 0.4 s	BenchTest/logs/2026-09-29_v050_bench_test.log
29 Sep 2026	GNSS outdoor fix on a rooftop, firmware v0.5.0, raw replies requested every 15 s	3D fix after 51 s; 17-field reply with decimal degrees; old parser read satellite counts as the position; well-formed replies 2.4 and 5.5 m from the phone's position; replies garbled by the competing query	BenchTest/logs/2026-09-29_gnss_rooftop.log
30 Sep 2026	GNSS outdoor fix with the corrected parser (v0.5.2), raw replies printed by the firmware	3D fix after 105 s from a cold start; 23 to 25 satellites; over 104 fixes, median 2.7 m and 95% within 11.0 m of the phone's position	BenchTest/logs/2026-09-30_gnss_rooftop_v052.log
30 Sep 2026	MQTT recovery from a 60 s radio-off period (AT+CFUN=4, then AT+CFUN=1), v0.5.0 and v0.5.3	Loss detected in 0.8 to 20.7 s; retries at 30 and 60 s; reconnect 36 s after the radio returned in v0.5.0, about 5 s in v0.5.3; no failed publishes afterwards	BenchTest/logs/2026-09-30_signal_loss_test.log, ..._v053.log
30 Sep 2026	Sync-pulse edges logged on the Pi (GPIO17, kernel timestamps), two 60 s runs	Run 1: 16 s burst of spurious edges, consistent with a disturbed contact; run 2: 121 edges, median interval 500.06 ms, range 499.72 to 500.41 ms	BenchTest/logs/2026-09-30_sync_edges_run1_noisy.csv, ..._run2.csv
30 Sep 2026	Camera detector latency on the Pi, three 60 s runs (4 threads, 4 threads with live view, 1 thread)	Median 102, 105 and 186 ms from sensor readout to result; 19.5, 17.3 and 7.0 frames/s; no throttling	BenchTest/logs/2026-09-30_c4_latency/
1 Oct 2026	Firmware v0.6.0 bench run, 330 s, logging on throughout (buffered log, signal quality, Hall counter in loopback)	Five flushes; all ten gaps over 10 ms (37 to 40 ms) in the first window after a flush (0.2 to 1.05 s after it); worst gap otherwise 5.9 ms; no dropped samples; Hall counter 2.00 edges/s; CSQ 24 to 26	BenchTest/logs/2026-10-01_v060_bench_test.log
1 Oct 2026	Read-back of the stored telemetry log (firmware v0.6.3, BenchTest/log_dump.py, four passes)	6,020 records recovered complete; 5,545 contained nan (invalid JSON, fixed in v0.6.3); 14 records with a valid BMS reading (74.78 V, 60%) from 30 Sep; log then cleared	BenchTest/logs/2026-10-01_telemetry_log_dump.jsonl, ..clean.jsonl

Date	Test	Result shown	File
1 Oct 2026	Stand-in ToF: micromouse sensor board on the Pi (three VL53L0X, `tof_probe.py`, `tof_reader.py`)	Chip confirmed VL53L0X; three sensors addressed and read at about 31 per second each; centre 267 to 300 mm, right 24 to 38 mm, left 8190 (no target) throughout	`BenchTest/logs/2026-10-01_tof_three_sensors.csv`
1 Oct 2026	Dashboard camera page (laptop, offscreen) against the live detector on the Pi 4, 60 s	Live after 0.6 s; 638 frames decoded in 45 s (14.2 per second); offline state shown when the detector stopped; no alerts (nothing in view)	`Report/evidence/2026-10-01_dashboard_page_live.png`, `DashboardIntegration/tests/test_front_camera_page_live.py`
1 Oct 2026	Dashboard camera page against the live detector with a person in view, 40 s	Red "Person, Immediate" alert listed 0.5 s after start, matching the box on the video; 15.4 frames/s, 109 to 118 ms; clearing not seen (person stayed in view)	`Report/evidence/2026-10-01_dashboard_page_person.png`
1 Oct 2026	Camera colour check (C3): stills with the standard and the NoIR tuning, detector frames with NoIR tuning and saturation 1.8	Standard tuning: white wall pink; NoIR tuning: neutral but washed out (blue card 120, 124, 125); with saturation 1.8: blue 1, 86, 216; red 222, 8, 77; green 6, 124, 103; grey 124, 129, 138; white 186, 199, 197 (slightly cool)	`Report/evidence/c3/`
2 Oct 2026	First pothole model: YOLO11n fine-tuned in Colab (T4) on a public 608-image pothole dataset, test split at 640 and 320	mAP@50 0.597 (P 0.66, R 0.50) at 640; 0.550 at 320; 46 epochs, 15 min	`CameraDetection/training/runs/run1_gerapothole_640/`
2 Oct 2026	First pothole model on the South African test split (605 images, 1,238 potholes) at 640, 1280 and 1920 input; labels checked visually	mAP@50 0.000 at all three sizes, recall at most 1.2%; label positions confirmed correct	`CameraDetection/training/runs/colab_session_2026-10-02.ipynb`
2 Oct 2026	Firmware v0.7.0 DASH line on the bench (40 s) and through the Raspberry Pi 4 bridge to a laptop	40 lines in 40 s, none missing, all valid JSON, max 272 bytes; IMUs 200 Hz, 0 drops; bridge served live record (0.37 s old) and ToF distances	`BenchTest/TEST_PROCEDURES.md` T11, T12
2 Oct 2026	YOLO11n (COCO) via NCNN on the Raspberry Pi 4, 320 input, 4 threads, 15 s trial	7.9 frames/s; inference 111 ms median; 175 ms sensor to result (p95 258 ms)	`CameraDetection/ncnn_bench.py`

Date	Test	Result shown	File
2 Oct 2026	Dashboard map crash on the Raspberry Pi 5, diagnosed over ssh without changing the team's installation	Segmentation fault about 2 s after the dashboard is shown with the web map (online or offline data); no crash with the native map after 30 s; the web engine alone loads normally	Dashboard Integration/ PI5_MAP_CRASH.md
3 Oct 2026	Dashboard live-only mode and automatic Raspberry Pi 4 finding, offscreen against a simulated bridge (laptop)	With the bridge down every page shows -- or Unavailable and no simulated value; live when the bridge appears, back to no data when it stops; host found by localhost, override and beacon, and followed after it moved	Dashboard Integration/ tests/test_live_- data_provider.py
4 Oct 2026	Street-tile map, offscreen, fed the generated UCT replay track (not recorded data) against a stand-in tile server	Tiles requested at most two at a time and cached; heading-up follow; "Waiting for GPS fix"; REPLAY badge; vector roads with the tile server down	Dashboard Integration/ tests/tile_map_- *.png
4 Oct 2026	Camera and time-of-flight fusion, offline tests	29 of 29 pass; not yet run on the Raspberry Pi 4 (test C9 planned)	CameraDetection/ tests/test_- sensor_fusion.py
6 Oct 2026	Sensors page and hazard banner, whole dashboard offscreen against the live bridge	ToF gauges (ahead 0.43 m red, right 0.69 m amber, left no reading), both listed in the fusion's immediate band (under 3 m) while the gauges colour by their own proximity thresholds (red under 0.5 m, amber under 1.0 m); banner over the driving page, hidden on the Sensors page	DashboardIntegration/ tests/sensors_- page.png, hazard_banner.png
6 Oct 2026	First fused camera and time-of-flight hazard, live, before calibration	Person, ahead sensor: camera 0.874 m, time-of-flight 0.096 m, reported 0.096 m; not checked against a tape	BenchTest/ TEST_- PROCEDURES.md, test C9 notes
6 Oct 2026	Pothole model run 2, South African road-band tiles (Colab)	Validation mAP@50 0.714 at 640 and 0.566 at 320; held-out test tiles not yet scored	CameraDetection/ training/runs/ run2_sa_band_- tiles_640/ training_- record.json
6 Oct 2026	Simulated drive on the street map, offscreen tests	Starts without a fix, 25 km/h along the route, heading-up, arrives, a real fix stops it	DashboardIntegration/ tests/tile_map_- simulated_- drive.png
6 Oct 2026	Raspberry Pi 4 power and temperature watcher, 20 min	Clear with nothing attached; soft temperature limit from 17:45; under-voltage 14 s after the telemetry unit was plugged into its USB port	BenchTest/logs/ 2026-10-06_pi4_- power_watch.log

Date	Test	Result shown	File
10 to 11 Oct 2026	Raspberry Pi 4 reboots, temperature and detector timing with the whole system running	Three restarts in 20 min with the telemetry unit on its USB port, none in 4 h after; 14.6 to 14.7 frames/s, median 118 to 122 ms, 95th percentile 146 to 148 ms	BenchTest/logs/ 2026-10-10_bench_- observations.md
11 Oct 2026	Store-and-forward outage test T15, firmware v0.7.4, 10 min radio off, ingest on the Raspberry Pi 5	60 records built up, 0 lost; reconnect 5.2 s; backlog cleared in 37 to 43 s; 112 of 112 stored, 111 of 111 against the unit's log (24 backfilled); IMUs 200 Hz, 0 dropped	BenchTest/logs/ 2026-10-11_sync_- outage_test.log, 2026-10-11_t15/
11 Oct 2026	Pothole path in the detector (C10)	Pothole found in a test picture (confidence 0.63); 13.9 frames/s at one run per 4 s against 14.6 without the path	BenchTest/ TEST_- PROCEDURES.md, test C10
15 Oct 2026	Detector on the Raspberry Pi 5 with the dashboard running, second OV5647 camera, 60 s	19.2 frames/s; readout to result 51 to 54 ms median, 61 to 63 ms 95th percentile; CPU 64 to 73 %; no throttling	BenchTest/logs/ 2026-10-15_pi5_- detector.log
19 Oct 2026	Whole system mounted on the vehicle, lab check	Front 17.5 to 22.0 frames/s, 53 to 102 ms; rear 14.4 to 14.5 frames/s, 126 to 151 ms; both ToF boards and IMUs answered; Ethernet 1000 Mb/s, 0.25 ms	Planning/ lab-notebook.md, 19 Oct
19 Oct 2026	First road session, recorder on the Raspberry Pi 5, 17:15 to 19:04	3,464 distinct records (10,287 replies), 38 % of seconds missing; 0 dropped; one 2 s IMU1 stall; up to 46.9 A, 74.6 to 72.8 V; no GNSS fix (no antenna); 1,342 front and 741 rear alerts	BenchTest/logs/ roadtest_- 2026-10-19/
21 Oct 2026	Telemetry unit on its own battery, wired UART to the Raspberry Pi 4 (v0.7.4.1)	No under-voltage on the Pi; 2,489 lines, 0 bad; unit restarted by itself and its modem stopped answering; IMU drop counts rose once and IMU2 was lost once, each until a reset	Planning/ lab-notebook.md, 21 Oct

The two detector runs end with the following lines, copied from the Raspberry Pi terminal. Before the fixes, every alert carried the wrong class:

```
Stopped. 430 frames in 60.1s (7.2 fps average).
{"class": "motorcycle", "distance_band": "immediate", "confidence": 0.66,
 "timestamp": "2026-09-28T11:58:32", "alert_status": "active"}
```

After the fixes, the class is correct, and the log shows the alert clearing and re-activating within a few seconds while the same person stayed in view:

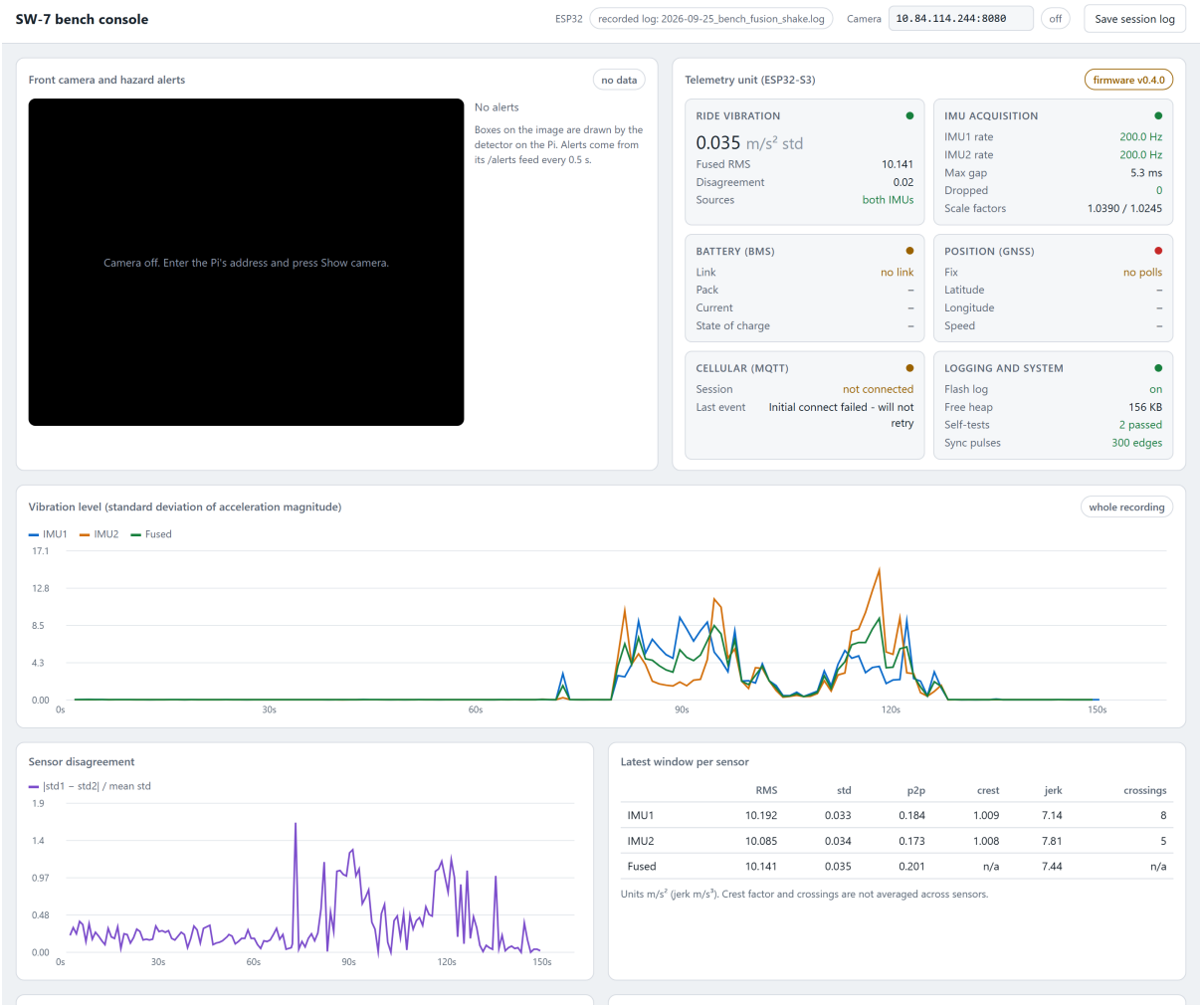


Figure D.1: The recorded bench session of 25 September 2026 replayed through the bench console, which parses the telemetry unit's serial output. Only recorded data is shown; the camera panel is off because the camera was not part of this session.

```
[14:31:28] person band=immediate conf=0.59 streak=3/3
  -> ALERT logged: person (immediate)
Stopped. 419 frames in 60.1s (7.0 fps average).
{"class": "person", "distance_band": "immediate", "confidence": 0.59,
 "timestamp": "2026-09-28T14:31:28", "alert_status": "active"}
{"class": "person", "distance_band": null, "confidence": 0.0,
 "timestamp": "2026-09-28T14:31:30", "alert_status": "cleared"}
{"class": "person", "distance_band": "immediate", "confidence": 0.59,
 "timestamp": "2026-09-28T14:31:31", "alert_status": "active"}
```

Appendix E

Configuration and Development Record

This appendix holds the firmware configuration, the record of development faults, and supporting detail moved out of Chapter 4. The full text of the earlier, chronological Methodology is kept in the project repository (`Report/archive_full_2026-10-11/`). Figure E.1 gives the project timeline as run, for the phases of Section 3.2.

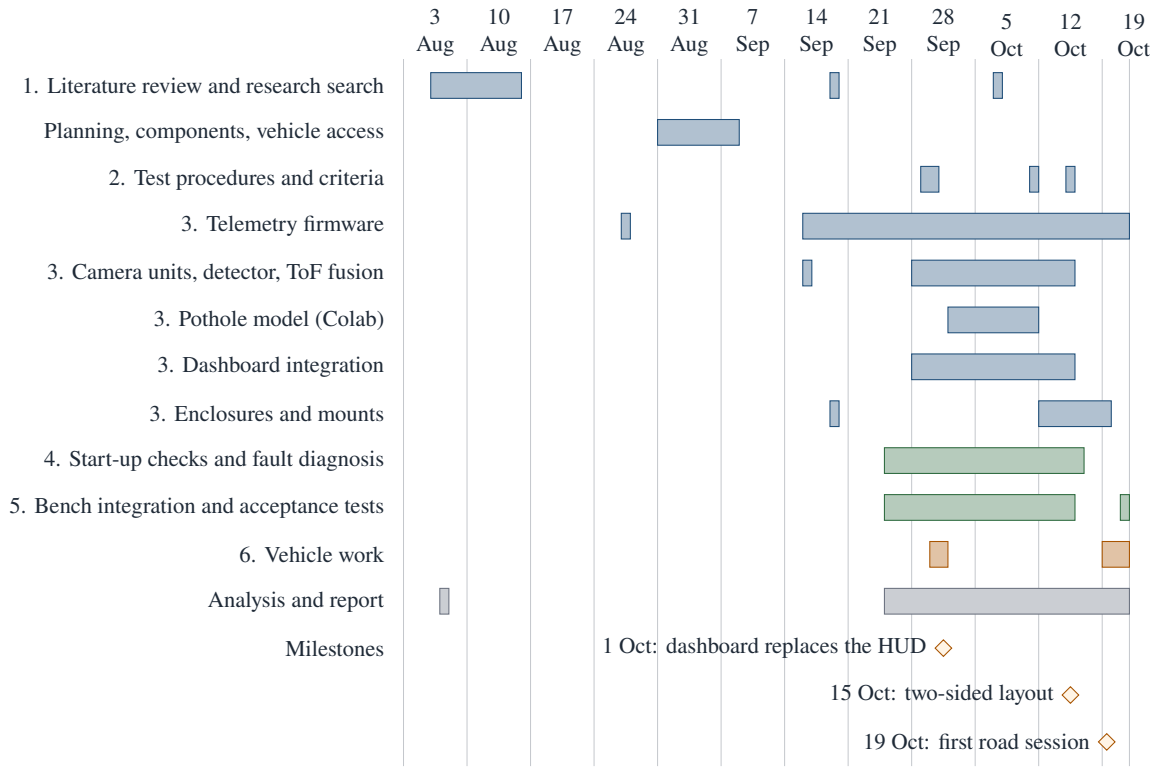


Figure E.1: Project timeline as run, 3 August to 21 October 2026, by week. Each bar spans the first and last dated record of an activity in the git history, `PROJECT_LOG.md`, `CHANGELOG.md`, `Planning/lab-notebook.md` and the evidence index (Appendix D); short bars are single dated events. The numbers are the phases of Figure 3.1. Blue: definition and building; green: bench checks and tests; amber: vehicle work; grey: analysis and writing.

E.1 Firmware configuration

Table E.1 gives the telemetry unit's configuration as committed for firmware v0.7.4, flashed on 11 October 2026 and used for test T15 and the road test. It is taken from `TelemetryUnit/platformio.ini` and `TelemetryUnit/src/main.cpp`; installed package versions are those in the PlatformIO folders on the development laptop, since `platformio.ini` does not pin them. Against v0.7.3, used for the bench results before 11 October, v0.7.4 changes the IMU low-pass filter from 260 to 94 Hz and adds the record identity and the store-and-forward ledger (rows marked v0.7.4); tasks and queues are unchanged.

Table E.1: Telemetry unit configuration (firmware v0.7.4).

Item	**Setting**
Build environment	PlatformIO, environment `esp32-s3-devkitc-1`, Arduino framework
Platform and core (installed)	`espressif32` platform 7.0.1; `framework-arduinoespressif32` 3.20017.241212 (Arduino-ESP32 core 2.0.17)
Libraries (installed)	Adafruit MPU6050 2.2.9 (declared ^2.2.4); Adafruit Unified Sensor 1.1.15 (^1.1.14); Adafruit BusIO 1.17.4 (^1.14.1)
Build flags	`ARDUINO_USB_MODE=1`, `ARDUINO_USB_CDC_ON_BOOT=1` (application serial on native USB)
Flash	16 MB, DIO mode; PSRAM memory type left at default (PSRAM not used)
Partition table	`default_16MB.csv`: nvs 0x9000 (20 KB), otadata 0xE000 (8 KB), app0 0x10000 and app1 0x650000 (6.25 MB each), data 0xC90000 (3.375 MiB, LittleFS), coredump 0xFF0000 (64 KB)
Serial	USB CDC at 115200 bit/s; bounded 3 s wait for a host at boot
IMU1	MPU6050 at 0x68, I^2C bus 0, SDA GPIO8, SCL GPIO9
IMU2	MPU6050 at 0x68, I^2C bus 1, SDA GPIO17, SCL GPIO18
I^2C clock	400 kHz on both buses
IMU settings	$\pm 8\,g$, ±500°/s, low-pass filter 94 Hz (v0.7.4; 260 Hz up to v0.7.3); 200 Hz (5 ms) sampling
IMU sample record	`sensor_id` (uint8, 0 or 1); `sequence` (uint32, per sensor); `timestamp_us` (int64, `esp_timer_get_time()`); `ax, ay, az` (float, m/s^2); `gx, gy, gz` (float, rad/s)
Calibration	200 samples at rest; factor accepted in 0.5 to 2.0; stored in NVS namespace `imucal`
Modem	A7670X on UART1, RX GPIO47, TX GPIO48, 115200 bit/s 8N1; PWRKEY GPIO4, RESET GPIO5
Sync pulse	GPIO15, toggled every 500 ms
Hall counter	Pulse counter unit 0, both edges, filter 1000 APB cycles (12.5 μs); input on GPIO15 (loopback) until the tap is wired; pole pairs not set
BMS	BLE, service 0xFF00, notify 0xFF01, write 0xFF02; query every 3 s; receive buffer 128 bytes
Periods	GNSS poll 3 s; statistics 2 s; ride window 1 s (200 samples); dashboard line 1 s (at most 480 bytes); MQTT publish 10 s; network position 60 s without a fix; assisted GNSS every 2 h
Fusion	Pair skew at most 0.5 s; fallback after 3 s; σ floor 0.05 m/s^2; speed index I_v needs $v \geq 1$

Item	Setting
MQTT	Broker `test.mosquitto.org`:1883, topic `sw7/telemetry`, QoS 1, keep-alive 60 s; retry 30 s doubling to 300 s; session lost after 3 failed publishes
Local log	LittleFS `/telemetry.log`; cap 3 MB; 8 KB memory buffer flushed every 60 s
Record identity (v0.7.4)	`dev`, `boot`, `seq` on every record, `utc_ms` from GNSS once fixed; replays marked `"replay":true`
Store-and-forward (v0.7.4)	NVS namespace `sw7sync` (boot, seqblk, acked, lost); sequence numbers reserved in blocks of 64; replay at most one record per 300 ms, 25 log lines per pass, 5 s after a failed replay; unacknowledged records removed by the log cap counted as lost
Tasks (name: core, priority, stack in bytes)	
Acquisition	Imu1Task, Imu2Task: core 1, priority 3, 4096
Statistics, sync	StatsTask: core 1, priority 2, 4096; SyncTask: core 1, priority 2, 2048
Ride features	RideChar1Task, RideChar2Task: core 1, priority 1, 4096
Modem, BMS, dashboard, Hall	ModemTask: core 0, priority 1, 4096; BmsTask: core 0, priority 1, 8192; DashTask: core 0, priority 1, 4096; HallTask: core 0, priority 1, 2048
Queues (depth in items)	
Queues	imu1Queue, imu2Queue: 100 samples each; imu1CharQueue, imu2CharQueue: 400 each; gnssQueue: 20; bmsQueue: 20

E.2 Development faults

Table E.2 lists every fault found during development, with its cause and fix. Seven of them, which needed a diagnosis, are described step by step in Section 4.10.

Table E.2: Development faults, causes and fixes.

Symptom	Cause (how found)	Fix and lesson
Boot loop at a fixed program counter after enabling 16 MB flash and PSRAM	Quad-I/O access requested on flash that runs in DIO mode (one setting changed at a time; ROM banner reports DIO; chip detection confirmed 8 MB PSRAM)	16 MB with DIO mode set together with the partition table; PSRAM setting left at default. Set flash size, mode and table together
No output past the ROM banner on the target board	Serial had been routed to UART0 for the bare development board; on the target board UART0 goes to a separate CH340K port (each hypothesis tested in turn; schematic)	Native-USB flags restored and documented in `platformio.ini`. The two boards need opposite settings
Modem silent after a reset	Power key not pulsed (v0.1); later, the pulse at every boot switched an already running modem off	v0.2.0 pulses GPIO4 for 3 s; v0.5.2 pulses only if `AT` gets no answer

Symptom	**Cause (how found)**	**Fix and lesson**
GNSS queue full after about a minute	Statistics task drained the IMU queues but not the GNSS queue	GNSS queue added to the drain cycle
Every GNSS poll rejected, then the first fix at 0.25 and 0.10 degrees	Assumed 13-field reply; real reply 9 fields indoors, 17 or 18 with a fix, decimal degrees (raw replies logged)	Parser anchored on the hemisphere letter, layout and jump checks (v0.5.1 to v0.5.3)
Zero GNSS polls reported for the first minutes	Polls time out for 60 to 90 s after GNSS power-on; statistics count only answered polls	Timeout lines distinguish the cases; allow warm-up before recording
Sync output pin not on any header	Placeholder chosen before the schematic was available (schematic and inspection)	Reassigned to GPIO15 before wiring
No sync edges, later a burst of spurious edges	Jumper disturbed by handling (ESP32 log showed the pulse still running)	Reseated; secured connector needed on the vehicle; off-pattern edges discarded
BMS scan never retried	Scan made conditional on an address set only after the device is found	Scan whenever disconnected and not scanning
BMS connection dropped before reading	Characteristics not under the assumed service in this BLE library (compared with a teammate's parser)	All discovered services searched
Every BMS frame rejected	Checksum included the register byte, so it failed by exactly 3 (the firmware's own query frame shows the rule)	v0.5.4; confirmed by 14 valid records on 30 Sep
MQTT reported connected when it was not; no retry after a failure	First `OK` taken as success; separate result line ignored (modem manual)	v0.5.0 waits for the result code; retry with backoff
Reconnection 36 s after coverage returned	Waited for the next scheduled retry although data were back in 2.4 s	v0.5.3 reconnects on the modem's data-available report (about 5 s)
33 to 40 ms acquisition gaps every 10 s	Flash writes suspend code running from flash on both cores (logging on and off test)	v0.6.0 buffers and flushes once a minute; pause length unchanged
Most stored records not valid JSON	Missing values printed as `nan` (full log read-back)	v0.6.3 writes `null`
Speed index dominated by a constant	Mean square of the magnitude includes gravity	v0.7.2 uses the variance
A person reported as "motorcycle"; poor confidence	Placeholder label shifted every class; camera buffers in BGR order	Label list and channel order corrected (fixed together)
16 alert activations in 47 s for one person	Alert cleared on a single missed frame near the threshold	Clears after 5 frames; nearer band needs 3 frames
Detector slower with the live view on	Drawing and encoding in the detection loop	Moved to a separate thread taking the newest frame

Symptom	Cause (how found)	Fix and lesson
Pink image; later black objects purple	Module has no infrared-cut filter (stills with standard and NoIR tuning)	NoIR tuning and saturation 1.8; dark-pixel correction in the live view only
Raspberry Pi 4 drive would not boot	Boot files copied instead of an image written: one partition, missing root	Rewritten with Raspberry Pi Imager
Raspberry Pi 4 did not join the hotspot; ssh closed	5 GHz hotspot without a regulatory domain; ssh not re-enabled in a second provisioning pass	Hotspot on 2.4 GHz and country code set (sufficient fix not isolated); ssh enabled
Recording aimed at the ceiling	No rigid camera mount	Live preview before recording; printed casing
Bracket model with no lens lip	Cavity cut through the face, visible only in a full render	Cut bounded; checked by full render (bracket not printed)
Simulated speed and battery on the dashboard with no sign	Dashboard fell back to simulation when the bridge address changed	Live-only mode and network discovery
Pi 4 not found on a phone hotspot; 6,828 lookup warnings	1 s probe shorter than the hotspot round trip; searches every 2 s	3 s probe; back-off to 30 s; beacon tried at once
Dashboard crash about 2 s after showing the web map	Native crash in the web engine on the Pi 5	Native map selected, web map not created; Qt tile map
Screen blank after 600 s; poller at 53% of a core	X11 screensaver and power management; game-controller poller	Both switched off by the start script
Team's dashboard started after a reboot	Desktop auto-start is not read on the bare X display	System service for the integrated copy
All three distances null	Start-up stopped at the first sensor that did not answer	Skip, log and re-probe; 0.5 s I/O timeout
Pi 4 under-voltage, throttling, failure to boot	LTE modem powered from the Pi's USB port (2 s power-flag log)	Separate supply for the telemetry unit; fan for temperature
Touch felt slow	Panel reports contact with the first position; buttons act on release (25 taps read directly)	Act on press and hide the pointer (next step)

E.3 Diagnosis records

Two further faults are recorded here in more detail than Table E.2 gives, because the method of finding them is part of the evidence.

Serial routing on the target board

After the IMUs were moved onto the target board, nothing beyond the ROM boot banner appeared on the serial port, across reflashes, resets and reseated wiring. Three hypotheses were tested and ruled out in turn: a hung I^2C bus (a `Serial.flush()` immediately after `Serial.begin()`, before any I^2C code, still printed nothing); USB enumeration time (a fixed start-up delay changed nothing); and damage from the rewiring (power LED and temperature normal). The cause was a build setting made earlier for the bare development board, which reaches the host over UART0 through a USB-to-serial chip: on the target board the schematic shows UART0 wired through a CH340K to a separate USB-C connector, so application output went to the unconnected port while the ROM banner, which always uses native USB, reached the connected one. Restoring the native-USB setting fixed it at once; the two boards need opposite settings, which are now documented in `platformio.ini`.

Raspberry Pi 4 provisioning and network

A boot drive made by copying boot files had one FAT32 partition and a `cmdline.txt` pointing to a root partition that did not exist; inspecting the partition table rather than the file listing showed it, and writing the image with Raspberry Pi Imager gave the expected boot (about 512 MB) and root (about 29 GB) partitions, checked against the disk signature. The Pi did not join a 5 GHz laptop hotspot: `iw reg get` showed the kernel's country code but a generic "world" domain on the radio itself, and `rfkill` showed no block. Moving the hotspot to 2.4 GHz and setting the country code through the driver-level configuration restored the link; both changes were applied together, so the sufficient fix was not isolated. A second provisioning pass, meant only to change the network credentials, had left ssh disabled, which shows that each field of a staged configuration has to be re-checked rather than assumed.

E.4 Supporting detail

GNSS reply layout

With a fix, the fitted modem's `+CGNSSINFO` reply has seventeen fields: fix mode; GPS, GLONASS, BeiDou and Galileo satellite counts; latitude and hemisphere; longitude and hemisphere; date; UTC time; altitude; speed; course; and PDOP, HDOP and VDOP, with latitude and longitude in decimal degrees. It matches the sixteen-field form in version 1.09 of the manual [47] with one extra satellite count; while a fix was settling an eighteen-field variant with an empty course and one undocumented trailing value also appeared, and the nine-field reply seen indoors is the same reply with the empty satellite block collapsed. The first outdoor capture, with a second query competing with the firmware's polling, contained replies with dropped characters, which the jump check now rejects. The modem's start-up command sequence takes about 27 s after boot before polling begins (Section 4.10.2).

Camera timing and modes

The lens-cover test covered the lens by hand during a recording, with the operator reporting the instant and the Raspberry Pi's wall clock read at once; that time was compared with the recording's start time plus the elapsed time to the first frame that turned uniformly dark. A first attempt in which the operator reacted to a countdown failed (the event was missing in one case and seconds late in another); with the operator triggering and reporting the event, the two agreed to within one second. The OV5647 sensor (2592×1944 pixels) supports, among other modes, 1920×1080 at about 32.8 frames/s and 1296×972 at about 46.3; the detector reads it at 640×480 for speed.

Planned VL53L5CX validation

When the VL53L5CX arrives it is to be validated on the bench before use: (1) confirm I^2C communication and the device identifier; (2) repeat the white and black target comparison of [45] at a fixed distance to measure this unit's valid-reading rate and bias; (3) sweep a target through the rated range to build a measured distance curve; (4) confirm that a target moved left, centre and right is reported in the right zones; and (5) check whether the warm-up drift affects the alert threshold. A single unit is planned because its 8×8 zones give the left, centre and right discrimination; a second would be needed only for a wider field or another direction, such as reverse.

Raspberry Pi 4 discovery

The dashboard's resolver (`DashboardIntegration/sw7_endpoints.py`) tries, in order: localhost, an explicit override, a fixed address on the Ethernet link (10.20.0.1 and 10.20.0.2, in use since 15 October), the source of a UDP beacon sent by the bridge every 2 s on port 50808 (ignored if older than 10 s), `pi4-camera.local` through mDNS, and the last address that worked (Figure E.2). A candidate is accepted when it answers with valid JSON within 3 s; three failed polls start a new search, with the wait doubling to 30 s. The offscreen tests pass; the wired link has been in use since 15 October and carried the rear data on the road session of 19 October.

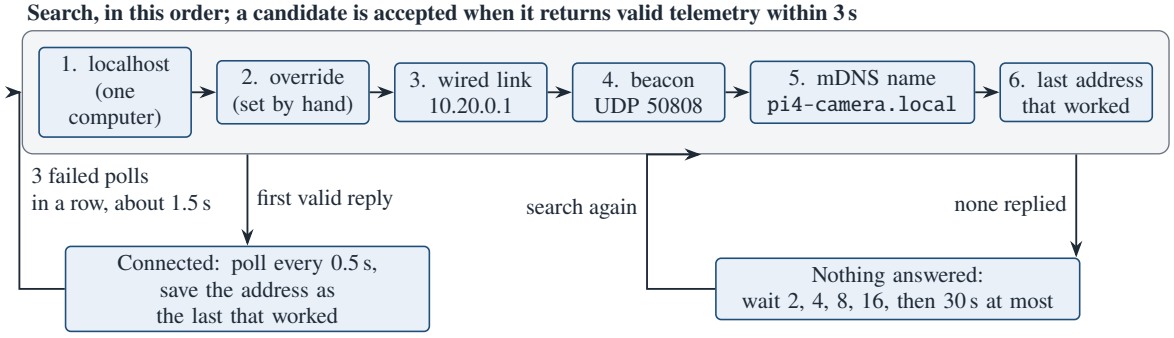


Figure E.2: How the dashboard finds the Raspberry Pi 4 (`DashboardIntegration/sw7_endpoints.py`). The same resolver serves the camera page and the live-data provider.

Camera colour

With the NoIR tuning and saturation 1.8, printed cards held about 1 m from the camera measured as in Table E.3 in the detector's own frames under indoor light. The three colours are clearly identified; grey and white show a small cool shift. Held together in one frame the cards remained distinct. The dark-pixel correction (Section 4.10.5) scales each pixel's saturation from fully grey at a brightest channel of 50 or below to unchanged at 110 and above, so the cards pass through it unchanged. The colours should be checked again in daylight.

Table E.3: Colour cards in the detector's frames with the NoIR tuning and saturation 1.8 (mean red, green and blue, 0 to 255; 1 October 2026, indoor light).

Card	R	G	B
Blue	1	86	216
Red	222	8	77
Green	6	124	103
Grey	124	129	138
White	186	199	197

Pothole model

Run 1 (2 October 2026) fine-tuned YOLO11n, pretrained on COCO, on a public 608-image pothole dataset from Roboflow Universe [52] in Colab, with no images from this vehicle's camera; its test images come from the same source, so its score describes that dataset, not South African roads. It ran 46 epochs in 15 minutes, stopped 15 epochs after the best validation result; mAP@50 0.597 at 640 pixels (precision 0.66, recall 0.50) and 0.550 at 320 on the dataset's own test images. Validation loss stopped improving after about 20 epochs, so more varied images were needed, not longer training. On the South African dataset's 605 test images (1,238 potholes) it scored 0.000 at 640, 1280 and 1920 pixels, recall at most 1.2%; drawing the converted labels on an image confirmed that the label conversion was correct. Of that dataset's 4,337 images (3680×2760 pixels), 1,987 contain 5,029 potholes, with a median size of 100 by 26 pixels, about 17 by 4.5 pixels after reduction to a 640-pixel input, and all centres lie between 48 and 62% of the image height. Run 2 (6 October) cropped each frame to the band from 42 to 68% of its height and cut it into three overlapping 1280-pixel tiles, so that a median pothole is about 50 by 13 pixels at the training input; after 60 epochs on a T4 it reached a validation mAP@50 of 0.714 at 640 pixels (precision 0.81, recall 0.62) and 0.566 at 320. These are validation scores, optimistic because validation chose the best epoch, and per tile; the notebook's record also did not state which weights training started from (both corrected in the notebook on 6 October). On the Raspberry Pi 4, installing the PyTorch library used by the Ultralytics software failed over the available network, so the exported model was run with the NCNN library alone, with letterboxing, box decoding and non-maximum suppression written for the purpose (`CameraDetection/ncnn_bench.py`), the lighter form a deployment would take. In a 15 s trial a COCO YOLO11n at a 320-pixel input with four threads ran at 7.9 frames/s, with a median of 111 ms for inference and 175 ms from readout to result (95th percentile 258 ms), against 19.5 frames/s and 102 ms for the SSD-MobileNet detector at its 300-pixel input.

Dashboard map and navigation

The street-tile map (`DashboardIntegration/sw7_tile_map.py`) draws OpenStreetMap tiles with Qt's painter under the tile usage policy (at most two requests at a time, an identifying user agent, no bulk download), caches tiles for 30 days, falls back to scaled lower-zoom tiles or the team's vector roads, and follows heading-up at zoom 17. A prefetch tool cached 656 tiles (11.9 MB) at zoom 11 to 16 around the campus, since the tile policy forbids bulk download at zoom 17 and above. Routes come from the public OSRM service [56]; map data are © OpenStreetMap contributors [55]. Firmware v0.7.1 added the GNSS course over ground (`crs`) and altitude (`alt`) to the dashboard line for the heading-up map, both null without a fix; a recorded bench check of the v0.7.1 output is not in the repository. A generated replay track (2,993 m along campus roads from an OSRM route, 650 points at one per second at 18 km/h, not recorded data) and a simulated drive at 25 km/h allow the map to be shown indoors; both are labelled on screen (REPLAY, Simulated drive) so that they cannot be taken for real positions, nothing from them is written to the vehicle state, and a real fix stops the simulation. The map has not yet followed a real fix. For bench work the start script also skips the driver and PIN page; the page is to be restored for the demonstration. The Raspberry Pi 5 also carries a Hailo-8 accelerator, which did not appear on the PCI Express bus because the external connector is not enabled in its boot configuration.

Appendix F

Enclosure and Mount Design

This appendix records the mechanical design of the printed assemblies that put the system on the vehicle: the dashboard display mount, the two sensor pods (rear for the Raspberry Pi 4 and the telemetry unit, front for the Raspberry Pi 5), and the two IMU pods. It gives the measurements each design rests on, the choices made and why, the checks run on the three-dimensional geometry, and the print plan. All were modelled in SOLIDWORKS 2025, or edited as meshes, and sliced for the Prusa MINI. On 12 October none had been printed; the pods were printed and mounted on the vehicle for the road session of 19 October, and the display mount is being redesigned. The checks below are on the CAD geometry, not on physical parts.

The parts were built by Python scripts that drive SOLIDWORKS through its API (`CAD/*/build/` in the repository). Every size comes from one parameter list, so a new measurement is one number and a rebuild. The scripts are the source of truth: most sketches are not dimensioned inside SOLIDWORKS, so editing a dimension in SOLIDWORKS does not drive the model.

F.1 Requirements and measurements

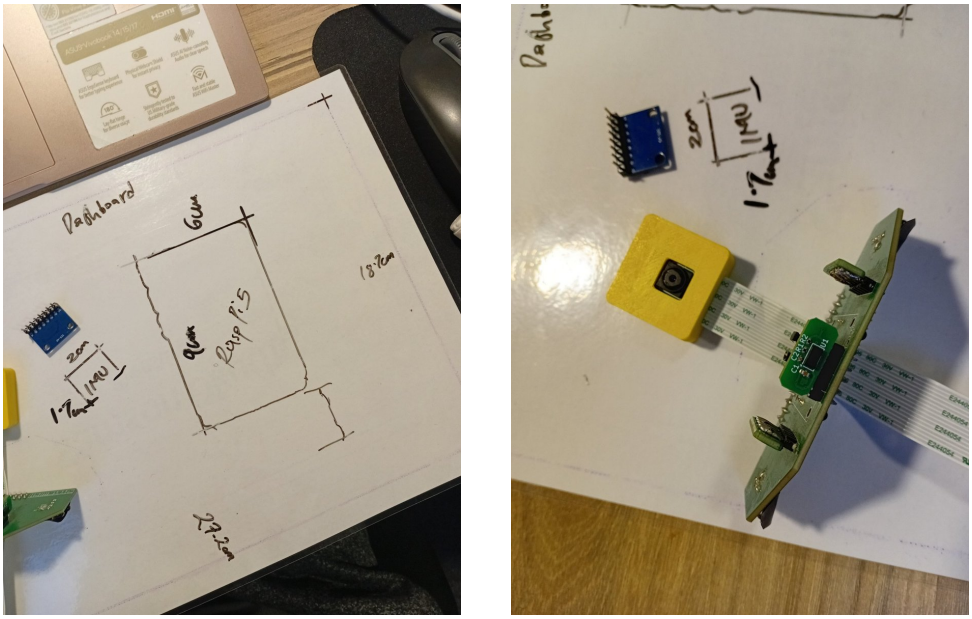
Table F.1 lists the inputs. Where a value was measured it was used; where it was not, it is marked as an estimate and listed as outstanding in Section F.10. Figure F.1 shows two of the measurement records.

F.2 Design choices

Table F.2 gives each main choice, the alternatives considered and the reason. The deciding constraints were the 180 mm print bed, the need to remove the electronics from the vehicle without tools, the sensors' fields of view, and sun and rain.

Table F.1: Inputs to the enclosure and mount designs.

Item	Value	Source
Display outline (Waveshare 10.1 in HDMI LCD (B) in its case)	272 by 187 mm	Measured; the manufacturer's drawing gives 274.12 mm, so the long side is to be confirmed
Display depth	20 mm	Estimate; the manufacturer gives 18 mm without a computer behind it
Display connector span	43 to 45 mm	Measured
Space at the vehicle's meter	about 305 mm across	Tape photograph, 2 September
Handlebar diameter	22.2 mm	Standard 7/8 in bar; not yet measured on the vehicle
Raspberry Pi 4 and 5	85 by 56 mm, M2.5 holes on 58 by 49 mm	Official mechanical drawings
ESP32-S3 A7670X board	75 by 52 mm, M2 holes on 71 by 48 mm	Manufacturer's board file (V1.2; the fitted board is V1.1)
ToF sensor board outline	86, 41, 22 and 30 mm edges; 14 mm above and 6 mm below the PCB	Measured; mounting holes estimated from a photograph
Camera in its printed case	29.2 by 28.2 by 12.35 mm	Measured from its STL
ToF field of view	25° cone	Datasheet
Camera field of view	53.5 by 41.4°	Raspberry Pi documentation
Printer	Prusa MINI, 180 by 180 by 180 mm	Available printers
GY-521 IMU board	20 by 17 mm	Measured; hole spacing 15.2 mm to be measured



(a) Display, Raspberry Pi 5 and IMU sizes recorded on the whiteboard.

(b) The camera in its case above the ToF sensor board, joined by the ribbon, in the layout the pod keeps.

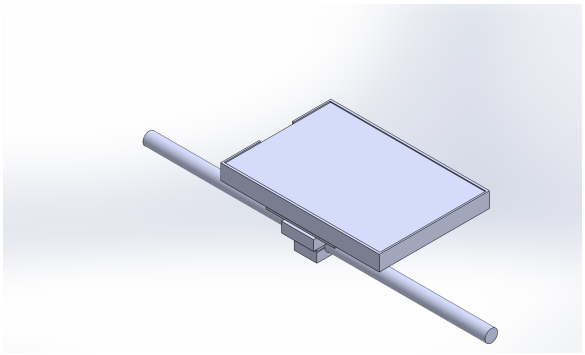
Figure F.1: Measurement records for the enclosure designs.

Table F.2: Main design choices for the enclosures and mounts.

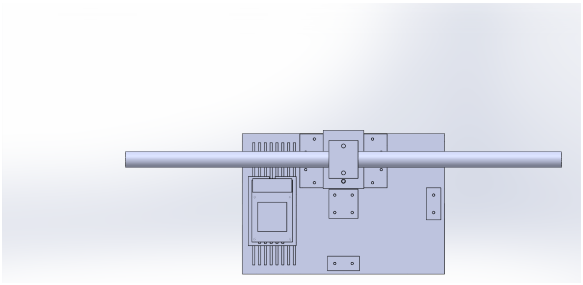
Choice	**Alternatives**	**Reason**
Bar clamp with a dovetail rail and a locking thumb screw	Adhesive pads, straps, a fixed screw mount	The display and pod come off without tools for charging, security and work on the bench. The channel's closed end carries the weight. The thumb screw drops into a detent in the rail, so the part cannot lift out over bumps. One clamp design serves both the display and the pod
Display above the handlebar, like a phone mount	Display centred on the bar	With the bar behind the display's centre, half the display would hang towards the driver's knees. Above the bar it stays in the line of sight at the meter
Display shell printed in four quarters joined by bolted plates	One piece on a larger printer	The shell is 278 by 193 mm, larger than the 180 mm bed in both directions. The plates carry load: the rail plate and the Pi 5 cradle are also joints
Raspberry Pi 5 on a cradle outside the shell	Inside the shell	Inside, the shell would grow from 27 to about 45 mm deep. Outside, the Pi has air for its cooler, and its cables to the display's connector side stay short
Cable exit as an open notch on the connector side	A closed window	It matches the 43 to 45 mm connector span, and it has no roof to print over the plugs
Sensor pod as a head and a box bolted together	One housing; a separate remote box	One housing exceeds the bed. A remote box would exceed the camera's 15 cm ribbon (the ribbon path is about 115 mm). The box also keeps the Pi's heat and the GNSS antenna away from the sensors
Open, flared ToF windows with drip hoods	A window covering the sensors	VL53L0X ranging degrades behind most plastics. The window flares 5° wider than the 25° cone, and a hood keeps rain off
Camera centred above the sensor board	Camera beside the board	This keeps the student's layout. Camera and ahead sensor share a centre line, and the cones show the camera's view clear of the ToF windows
GNSS patch under a 1.2 mm plastic roof	An external antenna	No metal or carbon-filled filament above the patch. One cable fewer
Heat-set inserts for every repeated joint	Screws into printed plastic	The joints survive being opened many times
Faceted outer form with a light-bar groove	A plain box	It follows the TUKZIE concept renders without costing function
IMU pods on the handlebar stem and the rear chassis, keyed by an arrow	One pod; loose boards	These are the positions in the vehicle integration and test plan (SW7-VIT-001), which follows the vacation-work teams' layout. The arrow fixes each board's axes relative to the vehicle

F.3 Dashboard display mount

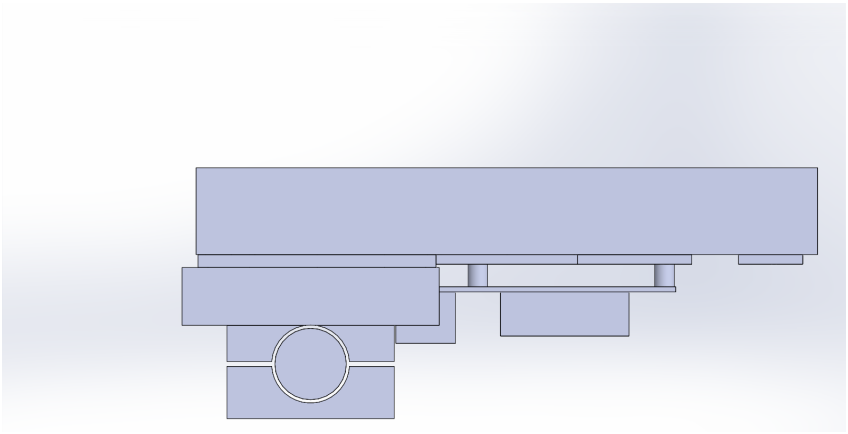
Figure F.2 shows the assembly on a dummy 22.2 mm bar, with stand-in bodies for the display and the Raspberry Pi 5. The model frame is face down: the screen faces up in the pictures. Figure F.3 shows the printed parts, and Table F.3 lists them. To fit the display, the clamp is bolted round the bar over a 1 mm rubber strip and turned to set the screen's tilt. The display slides down onto it, and the thumb screw is turned up into the detent.



(a) Isometric view.



(b) From behind: rail plate, Pi 5 cradle, joiner plates and clamp.



(c) Side view: shell, rail plate, bracket, clamp cap and bar.

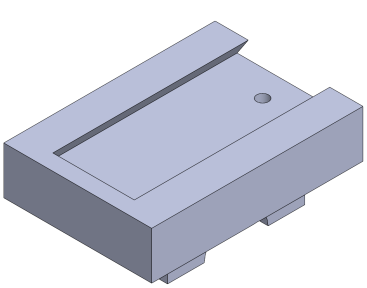
Figure F.2: Dashboard display mount assembly (revision 2).

F.4 Rear sensor pod (Raspberry Pi 4 and telemetry unit)

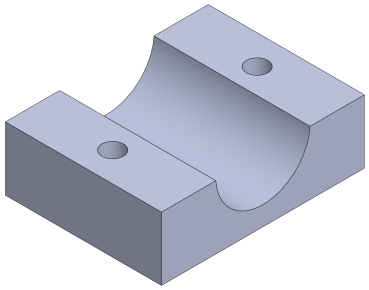
It was designed as the front pod before the two-sided layout of 15 October (Section 4.8.6) and now serves the rear, beside the battery.

The pod (Figures F.4 and F.5) has a sensor head and an electronics box, bolted together by four M3 flange screws.

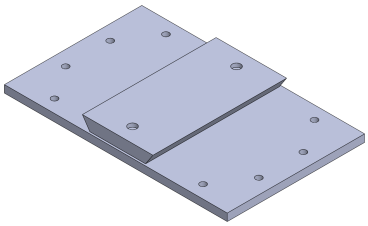
- *Sensor head.* The camera case sits in a cradle above the ToF board, with a foam ring at its front. The three ToF sensors look out through flared windows. The head is an upper shell and a tray that meet at a lip seam.



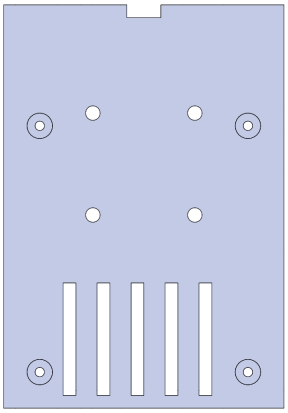
(a) Bracket: dovetail channel and upper clamp half.



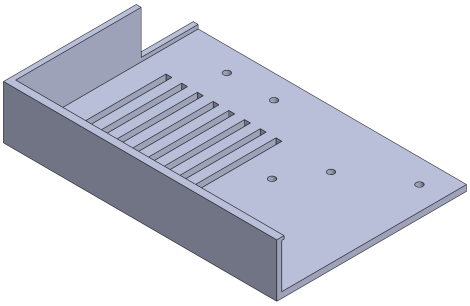
(b) Clamp cap.



(c) Rail plate with the lock detents.



(d) Pi 5 cradle with vent slots.



(e) One of the four shell quarters.

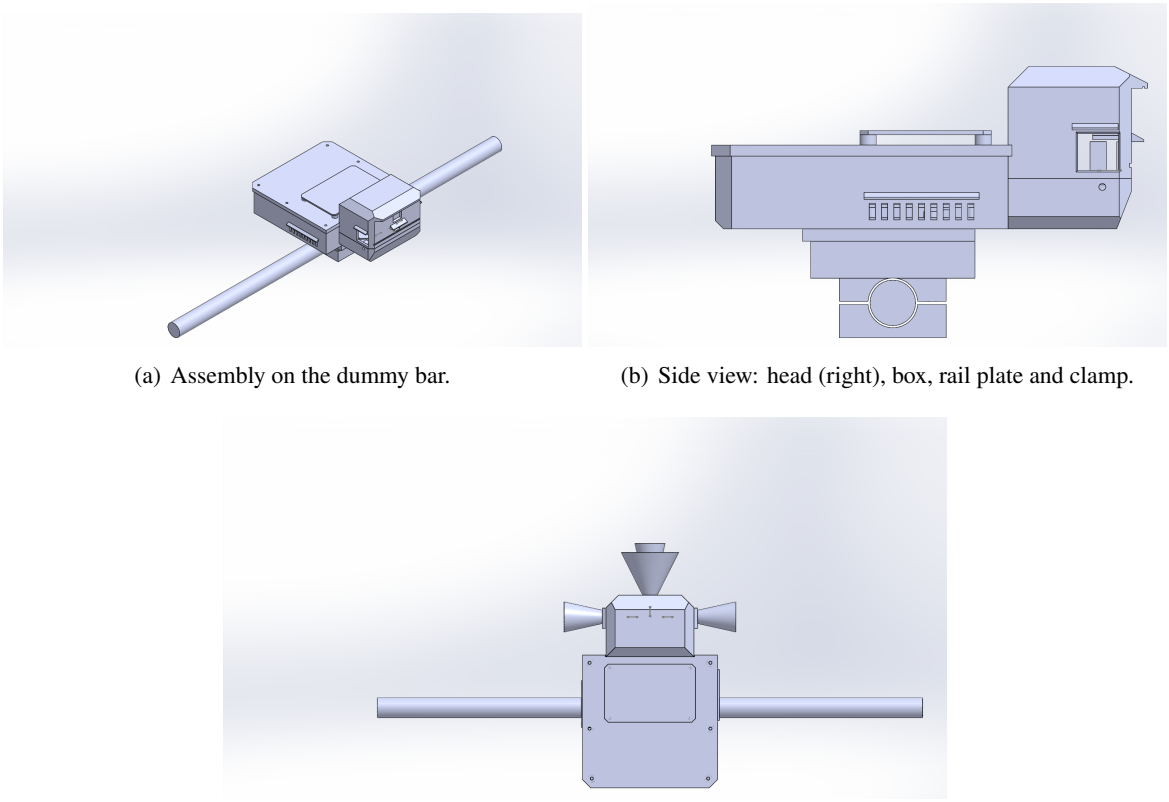
Figure F.3: Printed parts of the dashboard display mount.

Table F.3: Dashboard display mount: printed parts.

Part	**Size (mm)**	**Qty**	**Function**
Shell quarter	139.1 by 96.6 by 27	4	Back shell. The display presses in past 1.2 mm lips on the long walls
Rail plate	120 by 74 by 10	1	Dovetail rail (30 to 38 mm, 72 mm long, two lock detents); joins the bottom quarters
Cross joiner	40 by 40 by 3	1	One bolt into each quarter at the centre
Joiner plate	44 by 20 by 3	2	Across the top and right ends of the joints
Pi 5 cradle	66 by 95 by 10	1	Holds the Pi 5 on 7 mm standoffs; joins the left quarters
Bracket	56 by 80 by 29	1	Dovetail channel (closed end, 0.35 mm parallel clearance), M4 lock insert, upper clamp half with two 10 mm M5 inserts
Clamp cap	40 by 52 by 16	1	Lower clamp half, 4.9 mm under the bore, 1.5 mm squeeze gap

- *Electronics box.* The Raspberry Pi 4 and its fan sit in the front bay. The ESP32 board sits in the rear bay, under the thinned roof. A side hatch gives access to the flash drive. There is an SMA bulkhead for the LTE antenna and a gland for the vehicle supply. A sunshade covers the Pi bay.
- *Mount.* A rail plate under the box takes the same clamp as the display. Its closed end faces forward, so braking pushes the pod against the stop.

The two lower flange bolts sit under the Pi 4, so the head must be bolted on before the Pi goes in. The review found this, and the assembly sequence was corrected. Table F.4 lists the parts.



(a) Assembly on the dummy bar.

(b) Side view: head (right), box, rail plate and clamp.

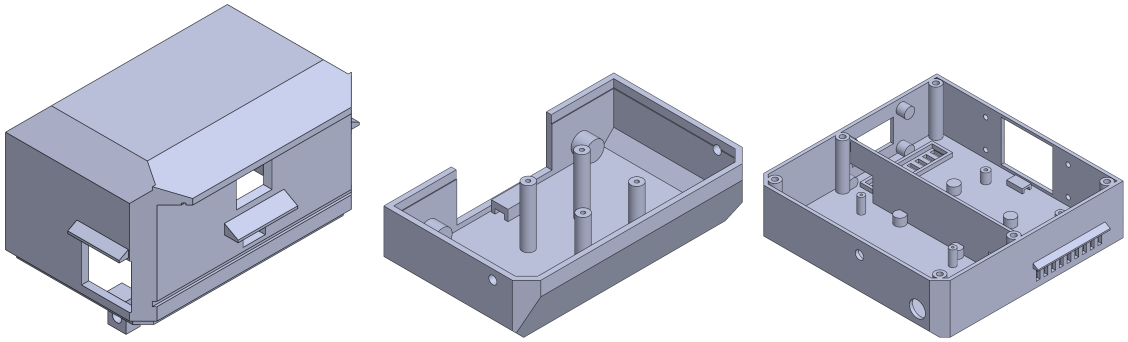
(c) From above, with the view cones of the three ToF sensors (25°) and the camera (53.5°) shown as reference bodies; no cone meets a wall.

Figure F.4: Rear sensor pod assembly (revision 2).

F.5 IMU pods

Each IMU pod (Figure F.6) holds one GY-521 board in a pocket sized to the measured 20 by 17 mm, with 0.6 mm clearance. It is screwed down on its hole edge, and a rib under the lid holds the free edge through a 1 mm foam pad. Cable ties through two tunnels fix the pod to the stem or the chassis rail, with VHB tape under it. An engraved arrow on the lid marks the vehicle's forward direction. The board's wires are soldered directly to the pads, because a header pin would not fit under the lid.

Revision 3 (13 October). The first printed pod showed two faults in a fit test: the boards carry a right-angle header whose Dupont housings stand out past the board edge, so the lid could not close, and



(a) Head upper shell: brow, windows with hoods, flange bosses.

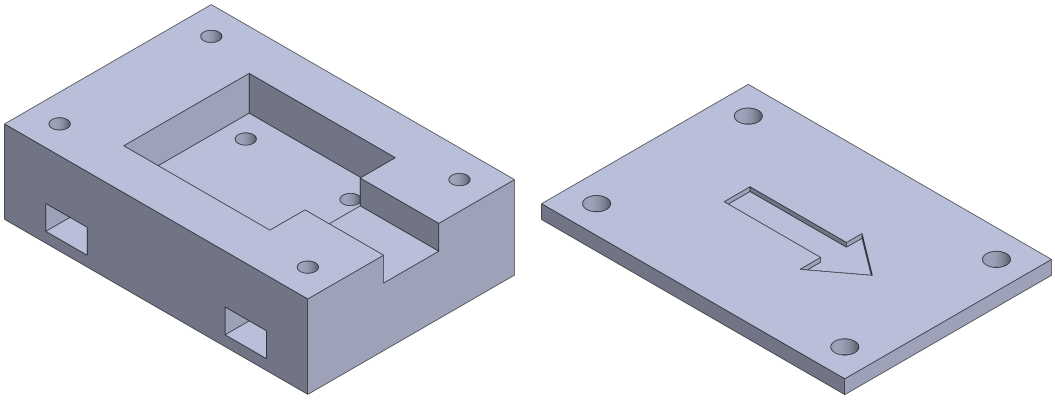
(b) Head tray: board pillars, ribbon guide, jumper channel.

(c) Electronics box base: Pi 4 and ESP32 bays, vents, hatch.

Figure F.5: Main printed parts of the rear sensor pod.

Table F.4: Rear sensor pod: printed parts.

Part	**Size (mm)**	**Function**
Head upper shell	104.8 by 67.9 by 63.1	Camera cradle, ToF windows with drip hoods, brow over the camera, flange bosses
Head tray	96.8 by 60.1 by 26.8	Board pillars (estimated hole positions), ribbon guide, jumper channel and tie anchor
Camera retainer	37.8 by 16.2 by 2	Holds the camera case in its cradle
Box base	148.8 by 142.5 by 38.4	Pi 4 and ESP32 standoffs, 5 V converter space, vents with hoods, hatch, gland, SMA hole, insert bosses
Box lid	148.6 by 146.3 by 5.4	Tongue-and-skirt seam, thinned roof over the GNSS patch
Sunshade	100 by 64 by 7	Second roof over the Pi bay, on four posts
USB hatch cover	52 by 33.5 by 5.4	Closes the side hatch
Rail plate	84 by 60 by 12	Dovetail rail with lock detents



(a) Base: pocket, board and lid pilot holes, strap tunnels, cable exit.

(b) Lid: clearance holes and forward arrow.

Figure F.6: IMU pod (revision 2).

no screws were to hand. Revision 3 follows the push-fit style of the camera case (Figure F.7):

- 1.6 mm walls instead of a solid block; 24 by 39 by 11.6 mm with the lid;
- the board on ledges 2.5 mm above the floor, clear of the header's solder tails, located by two pegs in its holes;
- clearance reduced to 0.4 mm a side at the student's request;
- tubes under the lid press the board onto the pegs, so no screws are used;
- the lid pushes onto a half-lap rim with a 0.1 mm gap;
- an 18 mm bay for the Dupont housings, with a wire slot closed by the lid;
- four cable-tie ears.

Two pods print in 1 h 19 min from 13.7 g, against 2 h 28 min for revision 2. The geometry is scripted (`CAD/IMU_Pod/rev3/imu_pod_rev3.py`), and the script checks that lid and base do not overlap. The peg spacing uses the unmeasured 15.2 mm hole spacing, with about 1 mm of play.

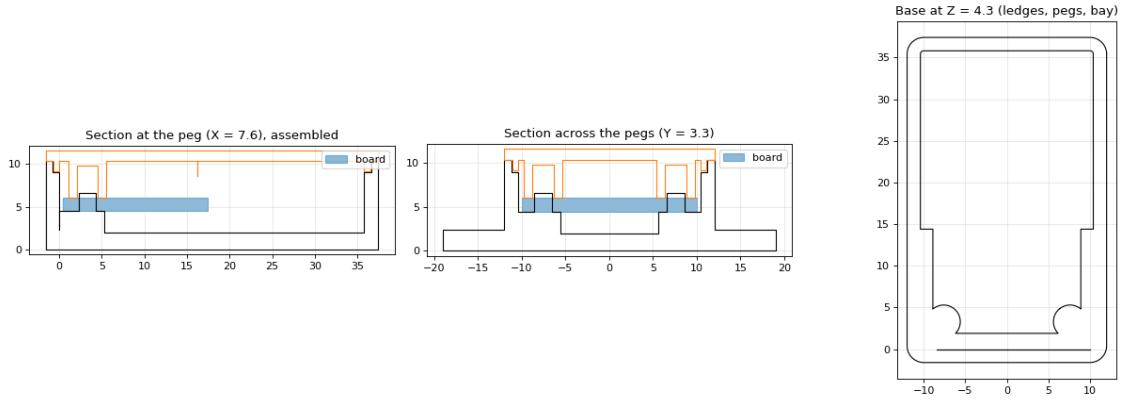


Figure F.7: IMU pod revision 3, sections: board (blue) on its ledges and pegs, lid (orange) with the tubes over the pegs, connector bay to the right.

F.6 Front sensor pod for the Raspberry Pi 5

The two-sided layout (Section 4.8.6) needs a second sensor pod at the front for the Raspberry Pi 5, which carries the AI HAT+ (Hailo accelerator) on its own cooler. It keeps the original head (camera, time-of-flight board, windows and hoods) on a new box for the computer alone (`CAD/Sensor_Pod_Front`; Figure F.8).

- *Reused parts.* An identity audit compared volume, area, centre of mass, bounding box and face count. The head tray, camera retainer and rail plate match the original parts, and the head upper matches the sliced original once a drip lip that the original script adds, but the saved part lacks, is left out. So beds 08, 09 and 12 serve both pods.

- *The stack.* The box was raised by 16 mm to 153 by 117 by 56 mm. It holds, from the bottom: the Raspberry Pi 5 on 10 mm standoffs; the official cooler; the AI HAT+ on 16 mm spacers; and the Hailo with an assumed 8 mm heatsink. The time-of-flight jumpers plug onto the stacking pins above the HAT, and the camera ribbon drops through the HAT's slot (164 mm path, leaving 36 mm of the 200 mm cable).
- *Cooling.* The cooler blows out through low slots in one wall; slots higher in both side walls give air across the HAT and the Hailo. The lid has an exhaust grille under a hood (below).
- *Fixing.* M2.5 by 16 screws from under the floor through the standoffs into the HAT's spacers. A 22 mm gap lets a hex key reach the head's lower flange bolts with the computer fitted, which removes an assembly-order problem of the original pod.

The six audit findings were fixed: thin insert walls, a 0.32 mm skin under the screw heads, two slivers, a short insert cap, thin drip-ledge tips and bosses that would print in mid-air. The assembly then measured:

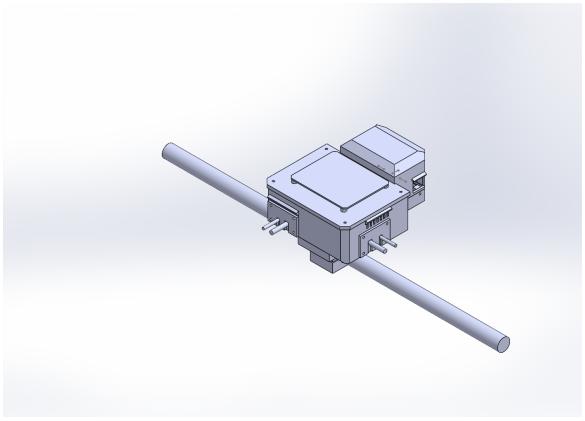
- no interference;
- every fastener hole coaxial;
- every insert wall at least 2.0 mm;
- the Hailo's heatsink 14.8 mm from the lid, and the jumper loop 3.3 mm from it;
- the hex keys clear of everything by 2.7 mm.

A last change, moving two bosses that came within 0.28 mm of a USB plug, was not re-audited, because the CAD program crashed twice during the full-assembly check. The spacer height, the heatsink and the cable length are datasheet values or estimates to be measured. The new parts print in about 20 hours from 250 g of PLA: the box base without supports, the lid and hood, and one bed of rail plate, sunshade and port covers.

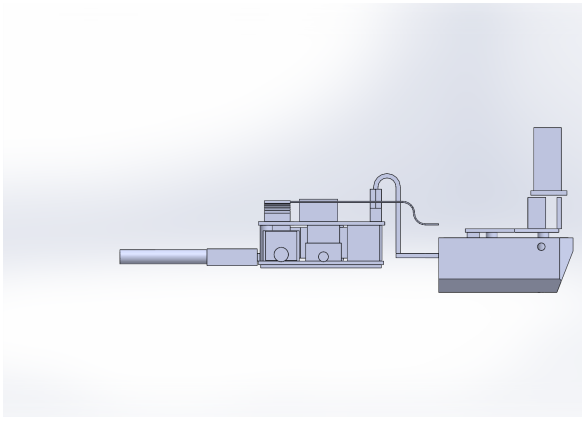
F.7 Ventilated lids

In the closed rear box the Raspberry Pi 4 reached 76.9 °C (Section 4.10.7): the box's only vents were in the floor and low in one wall, and nothing let warm air out at the top. Both pods' lids now have an exhaust grille over the computer, seven 4 mm slots giving about 1,900 mm^2 open. A separate hood stands 10 mm above the grille, closed at the front and sides and open to the rear with a drip lip, so warm air leaves and rain and the head wind stay out (Figure F.9).

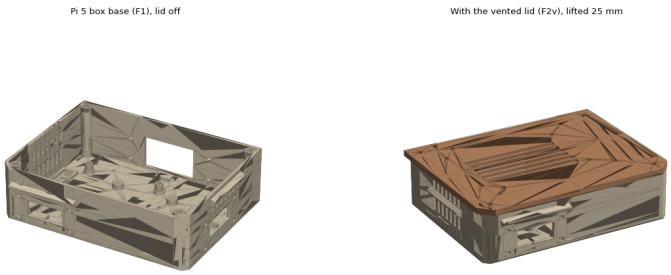
Four pins on the hood push into holes in the lid, so no screws are needed. Both parts print flat without supports: the rear lid in 3 h 39 min, the front lid in 3 h 00 min, and each hood in about 1 h 25 min. The lids were changed from the exported parts with a script (`CAD/Sensor_Pod/vent_rev/lid_vent.py`), which checks that hood and lid do not overlap. The temperature in the vented boxes has not yet been measured.



(a) Assembly on a dummy bar: head, box and lid.

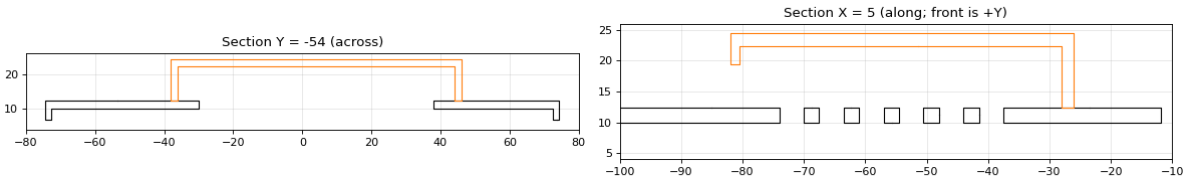


(b) Side view with the walls hidden: computer, cooler, AI HAT+, Hailo heatsink, jumper loop and camera ribbon.

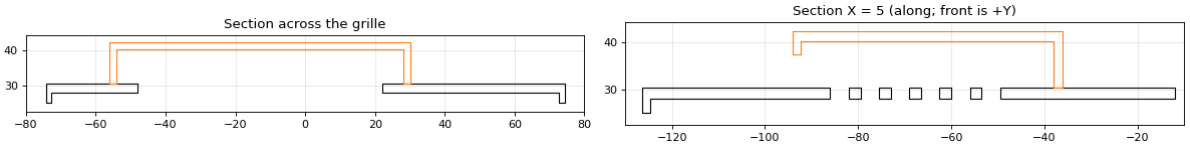


(c) Box base and vented lid, rendered from the print files.

Figure F.8: Front sensor pod for the Raspberry Pi 5 and AI HAT+.



(a) Rear pod (Raspberry Pi 4 and telemetry unit).



(b) Front pod (Raspberry Pi 5 and AI HAT+).

Figure F.9: Ventilated lids, sections: lid (black) with the grille, hood (orange) open to the rear.

F.8 Verification of the geometry

Each design was audited in its saved SOLIDWORKS files by scripts (`CAD/*/build/audit_*.py`). The scripts read the actual three-dimensional geometry, not a rendering, and their results are in each design's `ENGINEERING_REVIEW.md`. Five checks were run:

- *Interference.* SOLIDWORKS interference detection over the whole assembly. Before each result was trusted, a deliberately overlapping probe part was added to prove the check reports overlaps.
- *Contacts.* Every part must touch the parts it fastens to, so that nothing floats.
- *Fasteners.* Every screw position must have a hole of the right size in both parts, coaxial within 0.05 mm.
- *Thin walls.* Every pair of opposing faces closer than 2 mm.
- *Parts.* Body count, geometry faults and fit on the print bed.

The first review of the display mount rejected it. Its three critical defects:

- the clamp cap's bore cut right through it, leaving two loose pieces;
- the M5 inserts had 0.2 mm of wall;
- the shell's bolt holes were mirrored, so the rail plate's holes did not line up.

It also had five major defects:

- the cross joiner and rail plate overlapped by 1146 mm^3;
- the side lock screw's head would hit the rail plate;
- the clamp had no squeeze gap;
- there was no room for nuts behind the display;
- the dovetail flanks were not parallel.

The mirrored holes came from a sign error between the sketch plane's axes and the model's. All were corrected in the scripts and the parts rebuilt (revision 2). The audits were then run again; Table F.5 gives the results. Two corrections first introduced new thin walls of 1.0 and 1.75 mm, which the wall audit caught before the parts were saved.

No finite-element analysis was done, because there is no defined load case. The validation route is physical, in four tests:

- a printed dovetail coupon to check the slide fit;

Table F.5: Geometry checks after revision 2 (12 October).

Check	Display mount	Sensor pod	IMU pod
Probe overlap detected	Yes (1564 mm^3)	Yes (79 057 mm^3)	Yes (1303 mm^3)
Hard interference	0	0, including the view cones	0
Fastener holes	All matched (16 shell bolts, M5 clamp)	All 30 joints and 12 board pilots	All 4 lid screws
Walls under 2 mm	Pi 5 standoffs 1.9 mm (minor); lips 1.5 mm (by design)	None in the printed load paths	1.35 mm, pilot bottom to strap tunnel, below the screw's reach
Dovetail clearance	0.350 mm on both flanks, parallel	Same rail and clamp	Not applicable
Components fixed in assembly	11 of 11	16 of 16	Not applicable
Fits the 180 mm bed	All printed parts	All printed parts	Yes
Acceptance	Prototype-ready for the clamp, rail plate and a dovetail coupon	Requires revision (measurements)	Prototype-ready

- a clamp slip test with 1.5 kg hung at 150 mm from the bar;
- an insert pull-out test;
- a short ride with a safety tie, checking the bolt torques afterwards.

F.9 Manufacture

All parts were sliced with PrusaSlicer 2.9.4 for the Prusa MINI at 0.20 mm layers, onto 13 beds (Figure F.10, Table F.6).

- *Load-bearing parts* (clamp, cap and rail plates): 5 perimeters and 50% infill.
- *Enclosures*: 3 perimeters and 25% infill.
- *Supports*: only under the clamp's half bore, the pod's brow and inside the pod tray and box.
- *Orientation*: each part is printed with its largest flat face on the bed and the load-bearing layers across the load, as each review specifies.

The first prints are in PLA, for checking fits. PLA softens at about 55 °C, so the parts that stay on the vehicle are to be reprinted in PETG or ASA. Two faults in the slicer's command line had to be worked round:

- two multi-extruder settings in the saved print profile crash it;
- its `--merge` option crashes it, even with default settings.

For the second, each bed is laid out by the slicing script and sliced as one model, and the script checks that every part is on the bed.

Table F.6: Print plan (Prusa MINI, PLA, 0.20 mm).

Bed	**Prints**	**Time each**	**PLA each**
Display shell quarters (4 beds)	1 each	3 h 47 min to 3 h 51 min	51 to 55 g
Display rail plate and joiner plates	1	4 h 22 min	51 g
Pi 5 cradle and cross joiner	1	1 h 54 min	23 g
Clamp bracket and cap	2	9 h 0 min	95 g
Pod head upper shell	1	10 h 18 min	114 g
Pod tray and camera retainer	1	3 h 2 min	35 g
Pod box base	1	11 h 23 min	130 g
Pod lid	1	3 h 42 min	56 g
Pod rail plate, sunshade and hatch cover	1	5 h 12 min	61 g
Two IMU pods	1	2 h 28 min	27 g
Total		**about 75.5 h**	**about 0.9 kg**

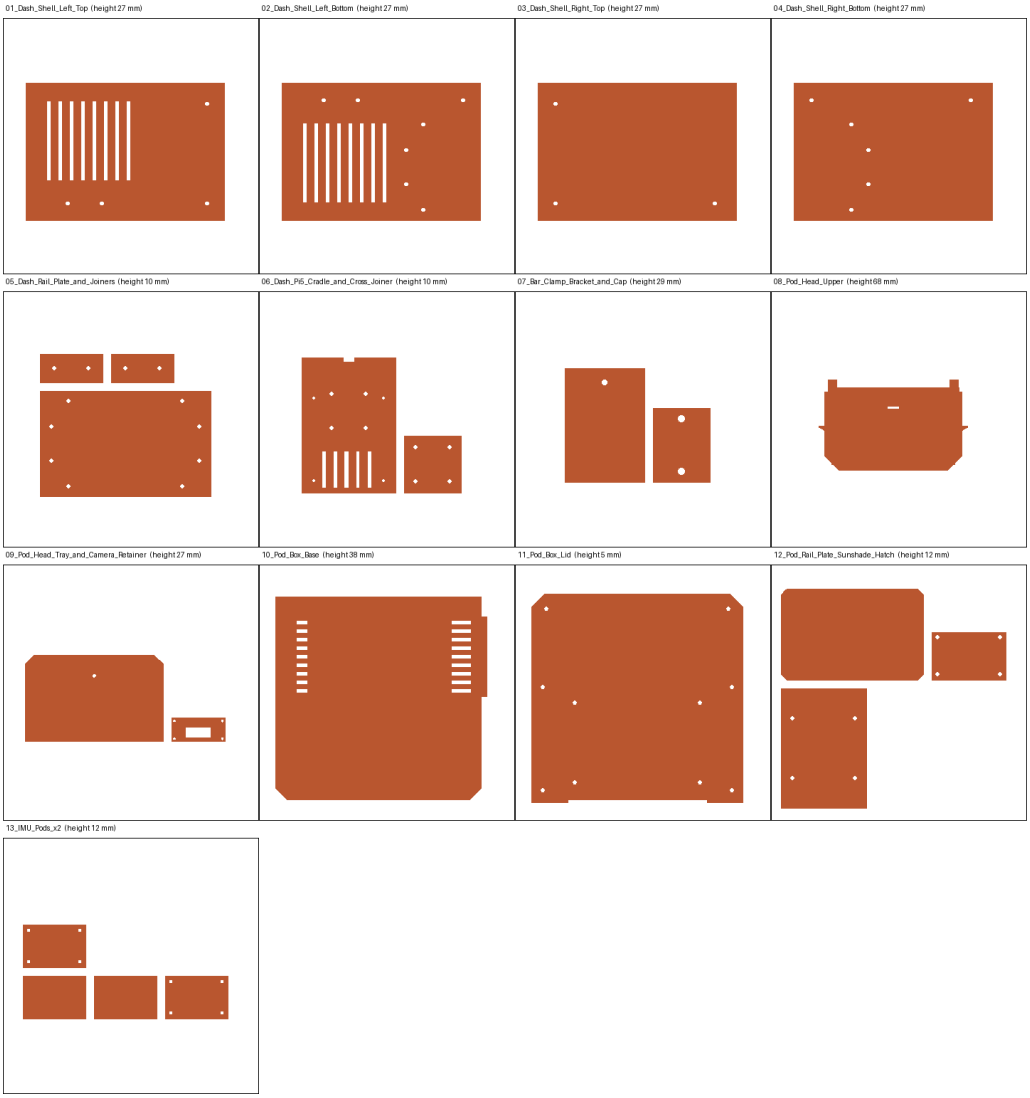


Figure F.10: Top views of the 13 print beds, drawn from the sliced STL data to check that every part is present and none overlaps.

F.10 Outstanding

[Outstanding: printing and fitting: the dovetail coupon, the clamp on the vehicle's bar, the display in its shell, the boards in the pod, and the physical tests of Section F.8.]

[Outstanding: measurements: the display's long side, the bar diameter at the clamp, the ToF board's hole positions, the IMU board's hole spacing, the camera ribbon length, the ESP32 battery and GNSS patch sizes, and the 5 V converter.]

[Outstanding: decision on the side ToF sensors' direction: 90° as built, or 35° as the fusion configuration assumes. The pod and the configuration must agree.]

Appendix G

Dashboard Interface Design

This appendix records how the driver's dashboard on the Raspberry Pi 5 was redesigned between 11 and 12 October: the steps, the rules the design follows, and how each step was checked. The data path behind the pages is described in Section 4.8.4. The refinement was deployed to the Pi 5 on 12 October (commit `5baa5f2`) and the three-planes version on 13 October (commit `864ce27`). *It has not yet been used on the vehicle, and glance times have not been measured.*

G.1 Starting point and constraints

The inherited dashboard (Figure G.1) is the vacation-work teams' PySide6 application. It has eight pages: Driving, Analytics, Diagnostics, Navigation, Front camera, Sensors, Charging and Settings.

Every SW-7 change reaches it as a patch with an exact anchor, applied to a copy (`DashboardIntegration/tests/dashbo` `patches.py`), so the team's own copy is never edited. Each stage of the redesign has an environment switch that restores the previous look exactly, and a test checks that it does.

Four constraints shaped the design:

- it is read at a glance while driving, in sun and at night;
- it is touched on a moving vehicle, possibly with gloves;
- it must keep live data and simulated data visibly apart;
- it must change only the presentation, never the telemetry, routing or sensor logic.

G.2 Design process

The redesign was done in six steps. Each step was written as a brief in `Planning/` and checked by tests and screenshots before the next began.

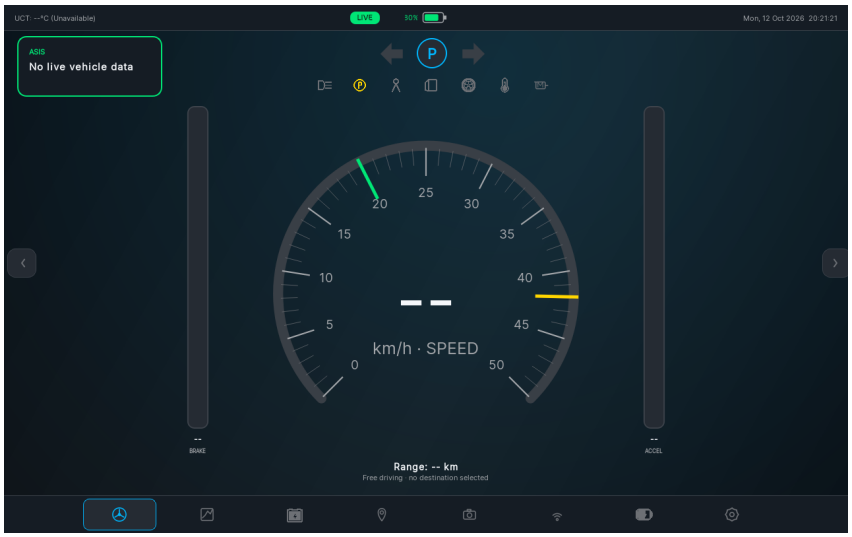


Figure G.1: The inherited Driving page, before the redesign (no live vehicle data).

1. *Layout pass.* A page-by-page review set minimum sizes:

   - 64 px touch targets;
   - driver-facing values of at least 32 px;
   - a 48 px status bar with 16 px text, against 38 and 11 px before.

   The pass also fixed real faults. The Charging page showed 79% while the status bar showed 80%, because one truncated and the other rounded.

2. *Data ownership and page merge.* An audit found the same data shown in several places:

   - battery percentage in seven places;
   - link health on two pages;
   - two different arrival estimates on the map at once;
   - two unlabelled outside temperatures.

   The student chose to keep the pages but remove the duplicates. Each value now has one home page and appears elsewhere only as a glance value that opens it. The Front camera and Sensors pages were merged into one page, Surroundings, leaving seven pages. Drive became the glance page, with tiles for the next turn, the surroundings and the range.

3. *Paper theme.* The student supplied a reference with paper and ink, one terracotta colour for anything that can be tapped, and very large or very small type. Live weather (Open-Meteo) was added, along with a night variant that switches at sunset.

4. *Refinement.* A second brief asked for restraint:

   - IBM Plex Sans for the interface and Plex Mono for readings;
   - fine borders and small radii in place of floating cards;
   - a flat navigation bar;
   - the weather inline in the status bar, with no box;
   - status that never relies on colour alone;
   - a dark mode designed in its own right;

- the team's battery animation kept, in the new colours.

5. *Three planes.* The student found the refined pages too flat and asked for depth again, after a product poster with a huge word behind the product. Each page now has three planes:

   - at the back, one very large word in Archivo Black (OFL), at 14 % ink by day and lifted charcoal at night: DRIVE, CHARGE, SETUP;
   - in the middle, a cut-out of the TUKZIE concept render that overlaps the word;
   - at the front, the readings and controls, unchanged in function.

   The Charging page returned to the team's original composition, a ring around the vehicle outline with rising bubbles, redrawn in the paper colours and animated only while the page is shown. Where no space was free (Map, Diagnostics) the page has no render. A test checks that the whole word stays readable in the open space above the tiles.

6. *Two cameras.* With the second camera and time-of-flight board on the Raspberry Pi 5 (Section 4.8.6), Surroundings gained a Front — Rear switch that moves the picture, alerts, distances and hazards together, a mirrored rear view, and a banner that labels each warning by side and shows the time to contact of an approaching object (Figure 4.5). A test with four fake servers checks the switch, a side going offline, the mirroring and the banner, and that the off switch restores the single-camera page exactly.

G.3 Design rules

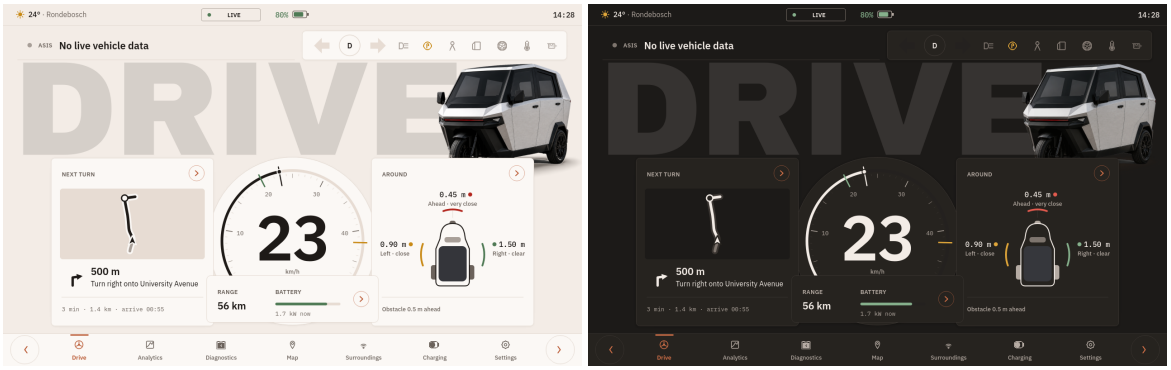
Table G.1 lists the rules the pages follow. Two are checked automatically over every page: no non-interactive element uses the tap colour, and every text size comes from the type scale.

Table G.1: Interface design rules and their reasons.

Rule	**Applied as**	**Reason**
One tap colour	Terracotta only on buttons, toggles, sliders, links and the selected page; checked over every page	The driver can tell what responds to touch without reading
Status not by colour alone	A word with every status mark, for example "Left, close" or "Unavailable"	Readable in glare and by drivers with colour-vision deficiency
Unavailable is not zero	"–" and "Unavailable" in muted grey, never 0	A missing value must not look like a valid reading
Live against simulated	Source chip (LIVE, REPLAY, NO DATA, SIMULATED) and a strip on every page during a simulated drive	Simulated data must never be mistaken for the vehicle's
One hero value per page	Speed, odometer, nearest obstacle, battery, turn distance at 120 to 180 px; the rest at 13, 16 or 22 px	A clear hierarchy at a glance
Tabular figures	IBM Plex Mono for changing numbers	Values do not shift as their digit count changes
Touch targets	At least 64 px	Use on a moving vehicle
One home per value	Other pages show a glance value that opens the home page	Removes duplicated and conflicting figures
Night variant	Charcoal surfaces, the same layout and tap colour, from sunset to sunrise	A white screen dazzles at night

G.4 The pages

Figures G.2 to G.4 show the deployed pages, and Figure G.5 the Pi 5's own screen. The screenshots are from the test suite at the display's native 1280 by 800, with test data.



(a) Drive, day: speed, next turn, surroundings and range tiles; the weather inline at top left.

(b) Drive, night variant.

Figure G.2: The Drive page after the redesign.

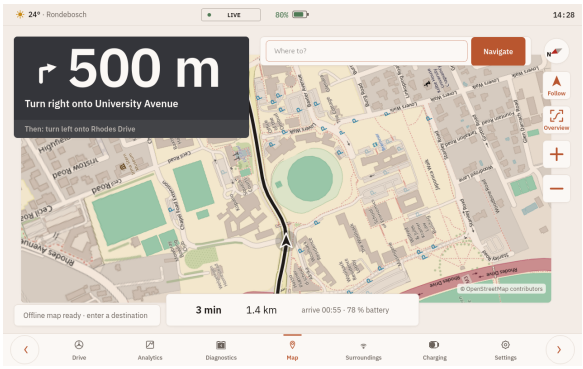
G.5 Verification

The dashboard's test suite runs the patched application off screen against a copy of the team's source. It covers 15 test files:

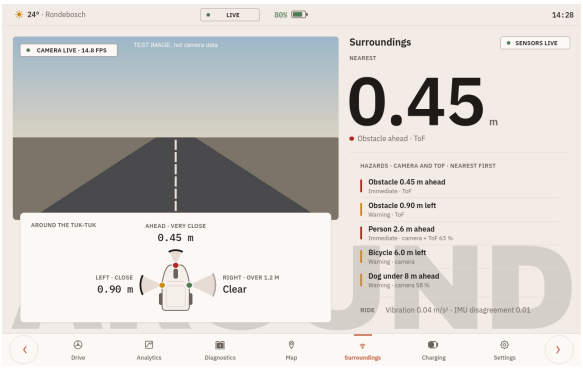
- live data;
- the map;
- simulated-drive synchronisation;
- search;
- the analytics carousel;
- the four layout tests;
- the six style tests, including one that checks the back word stays readable and no render covers a driving value.

All passed on the final merged build on 13 October. The style tests check:

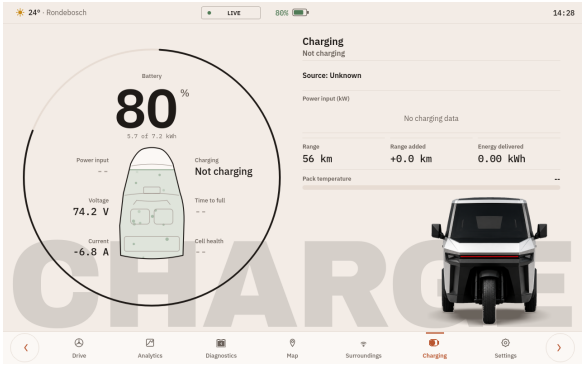
- the tap-colour and type-scale rules;
- that the bundled fonts load;
- that the weather item has no border or background;



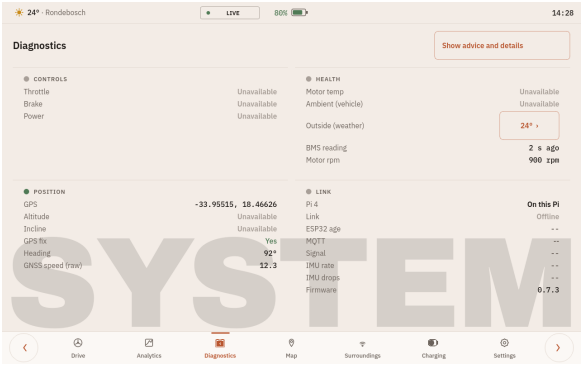
(a) Map: turn banner, single arrival bar, control column.



(b) Surroundings before the Front and Rear switch of 15 October: one camera, the three ToF distances, one combined hazard list (the two-sided version is in Figure 4.5).

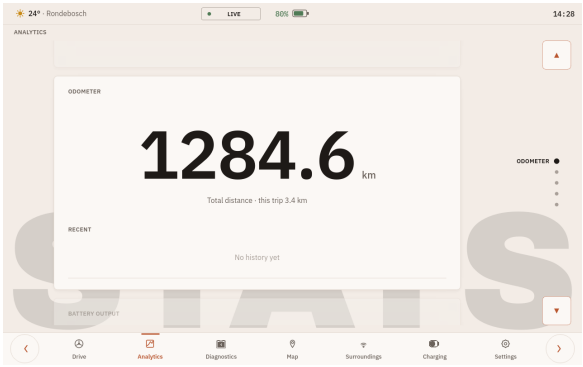


(c) Charging, with the team's battery animation restyled.

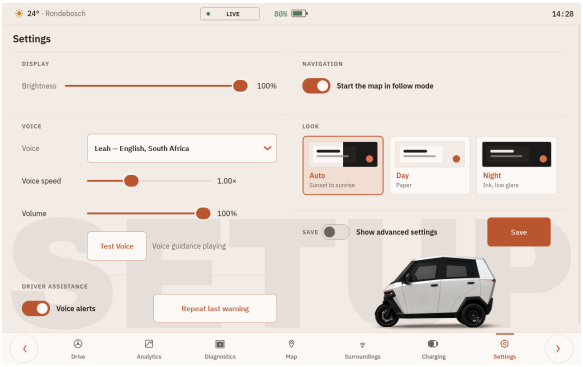


(d) Diagnostics: four groups; unavailable values in words.

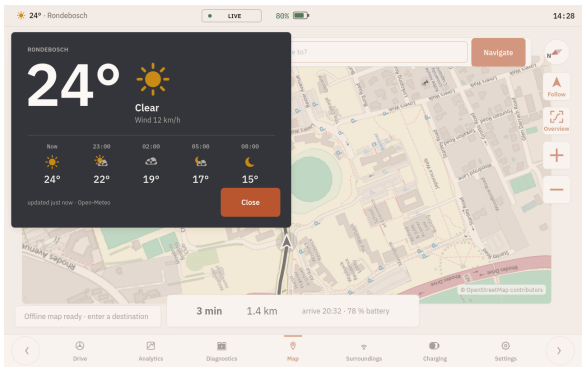
Figure G.3: Map, Surroundings, Charging and Diagnostics pages.



(a) Analytics: one value per card in a vertical carousel.



(b) Settings, including the Auto, Day and Night look.



(c) Weather card, opened from the status bar.

Figure G.4: Analytics, Settings and the weather card.



Figure G.5: Screen grab from the Raspberry Pi 5 after the deployment on 13 October, at night, with live time-of-flight readings from the bench (ahead 0.33 m, left 0.67 m, right not reading) and no vehicle data.

- every page in both themes, with no clipped labels;
- that the battery animation runs.

Recorded widget trees and pixels show that the previous look is unchanged when the redesign is switched off. Paint times measured on the laptop were below 15 ms per frame. The charging animation takes 1.0 to 1.5 ms per frame, and runs only while its page is visible.

Two of the checks caught faults during the redesign:

- an off-screen test that reset the status bar's height hid the obstacle pill;
- a test-cleanup step had been deleting the shared copy of the team's source between runs.

[Outstanding: on the Raspberry Pi 5: paint times, touch response and the fonts on the panel; the 2 s glance measurement on the vehicle; daylight contrast.]

Appendix H

Ethics Approval

The approval letter from the Engineering and Built Environment Research Ethics Committee, reference EBE/03305/2026, dated 6 October 2026, issued under the project's original title. Its conditions and how they are handled are in Section 3.7.

Engineering & Built Environment Committee (EBE REC)
EBE Faculty Office, New Engineering Building, Upper Campus, UCT
021 650 4234
zita.jemaar@uct.ac.za
https://ebe.uct.ac.za/research/ethics-research

2026/10/06

EBE REC REF : EBE/03305/2026
Samson Okuthe
samsn.okuthe@uct.ac.za

Dear Samson Okuthe, (Principal Investigator)

STUDY TITLE: Development of an Embedded High-Performance Computing Telemetry Unit and Windshield Head-Up Display for the TUKZIE Rev 0 Platform

We appreciate the submission of your study to the UCT Engineering & Built Environment Committee (EBE REC) for review and approval. We are pleased to announce that the EBE REC has formally **Approved with Conditions** the aforementioned study, with the following conditions:

The application indicates that no human subjects are to be involved and the study is approved on this basis. However, no full proposal was submitted with the application, yet there may be physical risks associated with the equipment to be used. A full proposal must be given to the supervisor, and a physical risk assessment must be completed to the satisfaction of the responsible laboratory owner.

Approval is only granted for one year and expires on **2026/12/31.**

Applications for renewal must be made prior to the expiration date, per Engineering & Built Environment Committee processes and any research conducted while renewals are being processed must conform to the approved research protocols.

Progress reports and/or close-out reports are required as specified in our Standard Operating Procedures, available on our website: https://ebe.uct.ac.za/research/ethics-research.

Kindly be aware that for all studies approved by the EBE REC, the principal investigator must acquire suitable institutional authorisation, if required, prior to conducting the research. The procedure for requesting access to UCT staff and students, or their data, is available here: https://uct.ac.za/research-support-hub/integrity/accessing-uct-staff-or-students-research-population.

Additionally, please note that the principal investigator bears the ongoing responsibility for the ethical conduct of study, and that this approval extends only to the named investigators conducting the study. The following people have been approved to participate in this study:

- No additional people conducting this study.

Please quote the EBE REC Reference in all your correspondence.

Yours sincerely

Harro Von Blottnitz

Chairperson
Engineering & Built Environment Committee

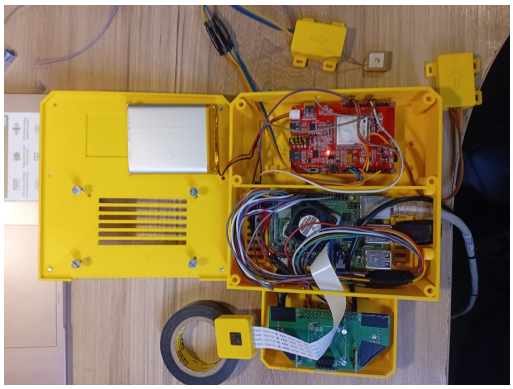
Appendix I

Photographs of the First Road Session

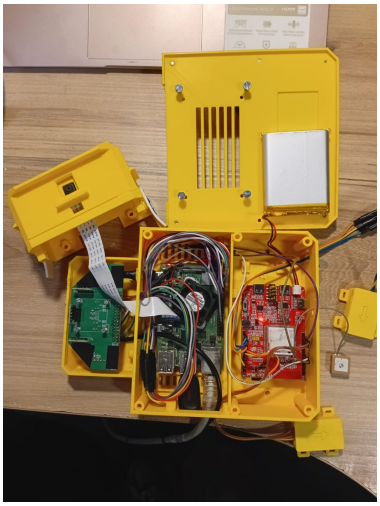
These photographs and video stills were taken by the team around the first road session of 19 October 2026 (Section 3.4.3) and shared on 21 October. The originals, twelve photographs and five short videos, are kept in the project's `Roadtest1` folder and not in the repository. In line with Section 3.7, the faces of other people have been cropped away or blurred and the vehicle's number plate blurred; the videos show people, so they are listed in Table I.1 rather than reproduced.

I.1 Rear sensor pod

Figure I.1 shows the rear sensor pod of Appendix F as built: the camera head with the time-of-flight board, the Raspberry Pi 4 with its fan and the ESP32 telemetry unit in their bays, and the vented lid with its push-on hood.



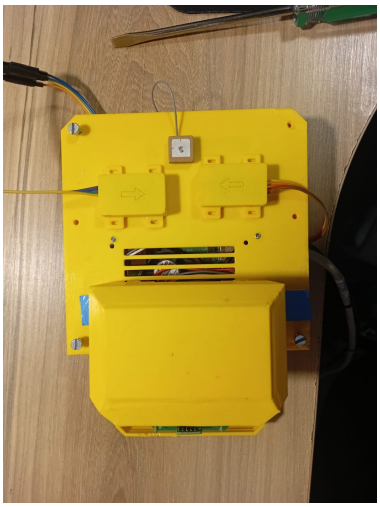
(a) Open, from above: the bays with the Raspberry Pi 4 and the ESP32, a battery pack held in the lid, and the head tray with the time-of-flight board below.



(b) Open, with the camera head lifted on its ribbon cable beside the time-of-flight board.



(c) Closed, with the hood over the lid's exhaust grille and the head tray open at the front.



(d) The lid from above, with the GNSS antenna on top and the grille under the hood.

Figure I.1: The rear sensor pod, printed and assembled.

I.2 The system on the vehicle

Figure I.2 shows the units mounted over the rear frame of the TUKZIE Rev 0, outdoors and in the laboratory, with the removable mounting of R13: a board laid over the frame and the units on it, with no drilling.



(a) From behind: the rear sensor pod (yellow) at the back edge of the board, the small IMU pods, and the power strip at the front of the bed.



(b) Closer, with a clear box holding further electronics and the units on the board.



(c) The whole vehicle from behind, outdoors.



(d) From the side in the laboratory (video still).

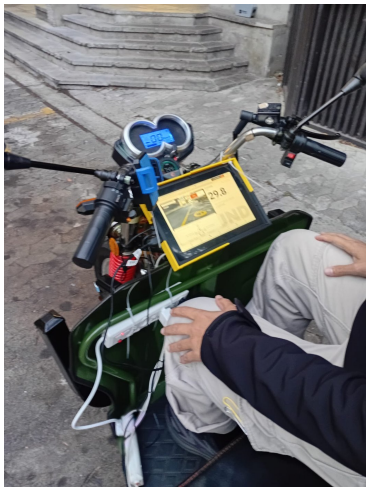
Figure I.2: The system mounted on the vehicle. Faces and the number plate are blurred or cropped.

I.3 The dashboard on the handlebar

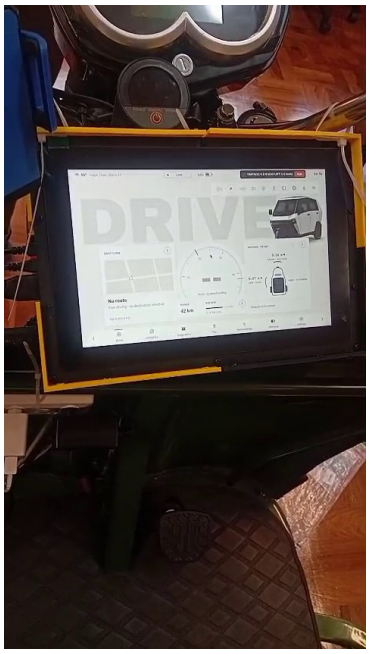
Figure I.3 shows the display in its printed holder on the handlebar, with the dashboard running.



(a) The display on the handlebar, outdoors.



(b) From the seat.



(c) The DRIVE page (video still).

Figure I.3: The dashboard display mounted on the vehicle.

Table I.1: Videos from the first road session, kept with the originals (file names as shared on 21 October 2026).

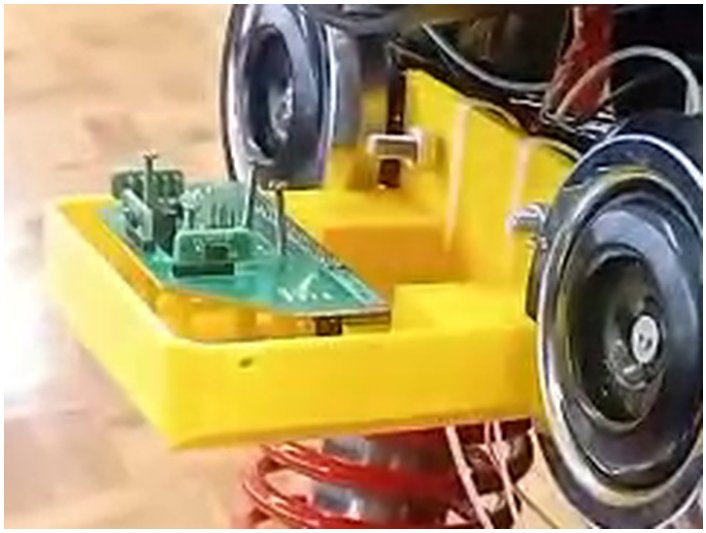
File	Length	What it shows
WhatsApp Video 2026-10-21 at 21.07.44	45 s	From a corridor to a doorway as the vehicle is ridden out of the building
WhatsApp Video 2026-10-21 at 21.07.53	19 s	A toy RC car and its remote shown to the camera, then the vehicle pushed through a doorway into a lecture venue
WhatsApp Video 2026-10-21 at 21.08.18	8 s	A walk round the rear frame with the units mounted, indoors
WhatsApp Video 2026-10-21 at 21.08.19	16 s	The vehicle ridden slowly inside the lecture venue
WhatsApp Video 2026-10-21 at 21.08.20	43 s	The wiring at the bed, then the display on the handlebar showing the DRIVE page

I.4 Front sensor board on the vehicle

Figure I.4 shows the front time-of-flight sensor board of the Raspberry Pi 5 unit (Section 4.8.6) fitted on the vehicle, on its yellow printed mount below the headlight and between the fork legs, facing forward.



(a) The front of the vehicle with the sensor board on its mount (video still).



(b) The mount and board, enlarged from the same frame.

Figure I.4: The front sensor board fitted on the TUKZIE Rev 0 (from the video of 21.08.20, at 28 s).